A COMPANION BOOK FOR STUDENTS

ROBOTS, MATHS AND MACHINES THAT THINK

From the first sensor to the first search algorithm

Basics of Robotics and Artificial Intelligence

$F = ma$

$\tau = J^T F$



Mr. Sourbh Kumar

Educator · Robotics · Artificial Intelligence

Livid Design Inc.

The way we build, makes difference.

PUBLISHED BY LIVID DESIGN LLP

www.lividdesign.in

Robots, Maths and Machines That Think

A Companion Book to the Lecture "Basics of Robotics and Artificial Intelligence"

Author: Mr. Sourbh Kumar
Edition: First edition, version 1.0, October 2026

Published by: Livid Design LLP (Livid Design Inc.)
Web: www.lividdesign.in Email: info@lividdesign.in Phone: +91-8348827197

© 2026 Livid Design LLP. All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior written permission of Livid Design LLP, except for brief quotations with clear attribution, used for review, criticism or teaching. Requests for permission may be sent to info@lividdesign.in.

The author's right to be identified as the author of this work is asserted. "Livid Design" and the Livid Design logo are brand marks of Livid Design LLP and may not be used without its written consent.

Educational use and safety. This book is written for teaching and learning. The programs, circuits and experiments are examples and are provided as they are, without warranty of any kind. Always work with a teacher or a responsible adult when you build or run a robot, follow the safety rules of your laboratory, and check voltage, current and mechanical limits before connecting hardware. Livid Design LLP and the author are not liable for loss or damage arising from the use of this material.

All figures were created by the author for this book. Product and company names mentioned belong to their respective owners and are used for identification only.

Preface

I have taught robotics for a long time, and one thing has stayed the same in every batch: the students who do well are not the ones who memorise the most formulas. They are the ones who can look at a moving machine and ask, "What is it measuring right now, what is it deciding, and what is it pushing against?" If you can ask those three questions, you can learn any robot, old or new.

This book is the written partner of the lecture *Basics of Robotics and Artificial Intelligence*. In the lecture we had animations, and you could watch an arm swing or a controller settle. On paper I cannot move the pictures, so I have done two things instead. First, every figure here was produced by actually running the mathematics, so the curves are not drawings of what I think should happen; they are what the equations give. Second, I have written out the working in full, including the small steps that lecture time never allows. When a calculation looks long, read it with a pencil in your hand and reproduce it. That is how it becomes yours.

How the book is arranged. It follows the five units of the course, but I have split some of them into more chapters than the syllabus does, because a few topics (frames, kinematics, control) deserve patience. Chapters 1 to 3 give you the language of robot bodies. Chapters 4 to 7 cover sensors, control, motors and grippers, which is everything that makes a body useful. Chapters 8 to 10 move into artificial intelligence: agents, learning, search and reasoning under uncertainty. Chapter 11 puts everything together on a small wheeled robot, and Chapter 12 holds the practice problems with complete solutions. The appendices have a formula sheet, short Python programs, a glossary and suggestions for what to read and build next.

How to read it. Boxes with a teal edge are worked examples: cover the solution and try first. Boxes with an amber edge hold the one idea I would like you to leave with. Boxes with a dark edge are remarks from the classroom, usually about a mistake I see again and again. Small "Try it yourself" boxes are exercises with a short hint; the full problem set at the end has worked answers.

A request. Do not treat anything here as finished. Change a number, break an assumption, write a ten-line program and watch what happens. Robotics is learnt by touching it. If you build something from these pages, I would be very happy to hear about it.

Mr. Sourbh Kumar

Contents

Preface iii

1 Meeting the Robot 1

1.1 Why start with a definition . . . 1
1.2 A short walk through history . . . 2
1.3 The anatomy of a robot . . . 2
1.4 Joints, links and degrees of freedom . . . 3
1.5 Types of robot bodies . . . 3
1.6 The laws of robotics, and what they really teach . . . 4
1.7 Accuracy, repeatability and resolution . . . 5
1.8 Where robots work today . . . 5
1.9 Chapter summary . . . 6

2 Describing Motion: Frames and Transformations 7

2.1 The problem of “where” . . . 7
2.2 Rotation in a plane . . . 7
2.3 Rotations in three dimensions . . . 9
2.4 Translation and homogeneous transforms . . . 10
2.5 Describing orientation: Euler angles and a warning . . . 11
2.6 Frames in a real work cell . . . 12
2.7 Chapter summary . . . 12

3 Arm Kinematics 13

3.1 Forward kinematics of a two-link arm . . . 13
3.2 The workspace . . . 14
3.3 Inverse kinematics of the two-link arm . . . 15
3.4 The Denavit–Hartenberg convention . . . 17
3.5 The Jacobian: from joint speed to hand speed . . . 18

3.6 Singularities . . . 19
3.7 Moving smoothly: trajectories . . . 20
3.8 Chapter summary . . . 22

4 Sensors and Signals **23**
4.1 What a sensor does . . . 23
4.2 Incremental encoders . . . 23
4.3 Ultrasonic distance sensing . . . 25
4.4 LiDAR . . . 26
4.5 From voltage to number: the ADC . . . 26
4.6 Noise and filtering . . . 28
4.7 A first look at the Kalman filter . . . 29
4.8 Sensor fusion in one sentence . . . 30
4.9 Chapter summary . . . 31

5 Making Machines Obey: Feedback Control **32**
5.1 Open loop versus closed loop . . . 32
5.2 The proportional controller . . . 32
5.2.1 A model to calculate with . . . 32
5.3 Adding the integral term . . . 34
5.4 Adding the derivative term . . . 35
5.5 Describing a response . . . 35
5.6 The second-order picture . . . 36
5.7 Stability in plain language . . . 37
5.8 Tuning in practice . . . 38
5.9 A discrete-time controller in code . . . 38
5.10 Cascade control in a servo . . . 39
5.11 Chapter summary . . . 39

6 Actuators and Drives **40**
6.1 Comparing the drive families . . . 40
6.2 The permanent-magnet DC motor . . . 40
6.3 Stepper motors . . . 42
6.4 Servo motors . . . 43
6.5 Gears and why robots need them . . . 43
6.6 Hydraulic and pneumatic drives . . . 45

6.7 Choosing an actuator: a way of thinking . . . 46
6.8 Chapter summary . . . 46

7 Grippers and Grasping **47**
7.1 Families of grippers . . . 47
7.2 Gripping by friction . . . 47
7.3 Form closure and force closure . . . 49
7.4 Vacuum grippers . . . 49
7.5 Choosing a gripper . . . 50
7.6 Chapter summary . . . 50

8 Intelligence for Robots **51**
8.1 What do we mean by AI? . . . 51
8.2 The agent model . . . 51
8.2.1 Properties of environments . . . 52
8.2.2 Types of agent programs . . . 52
8.3 Learning from examples . . . 53
8.4 The artificial neuron . . . 53
8.5 How a neuron learns: gradient descent . . . 54
8.6 Layers of neurons . . . 56
8.7 Generalisation: the real goal . . . 57
8.8 Where learning enters a robot . . . 57
8.9 Chapter summary . . . 58

9 Search, Games and Evolution **59**
9.1 Formulating a search problem . . . 59
9.2 Uninformed search . . . 60
9.3 Informed search and A* . . . 60
9.4 Local search and the idea of optimisation . . . 61
9.5 Adversarial search: minimax . . . 62
9.5.1 Alpha–beta pruning . . . 62
9.6 Genetic algorithms . . . 63
9.7 Choosing among methods . . . 64
9.8 Chapter summary . . . 65

10 Knowledge and Reasoning Under Uncertainty **66**

10.1 Representing knowledge . . . 66
10.2 Propositional logic in practice . . . 67
10.2.1 Inference rules . . . 67
10.2.2 Forward and backward chaining . . . 67
10.3 The trouble with certainty . . . 67
10.4 Probability in one page . . . 68
10.5 Bayes' rule . . . 68
10.5.1 A neat way to see it: odds form . . . 68
10.6 Localisation with a discrete Bayes filter . . . 69
10.7 Deciding with probabilities: expected utility . . . 71
10.8 Putting probabilities to work . . . 71
10.9 Chapter summary . . . 71

11 Putting It Together: A Mobile Robot **72**
11.1 Kinematics of a differential-drive robot . . . 72
11.2 Odometry: where have I been? . . . 73
11.3 Sizing a delivery robot, step by step . . . 75
11.4 Software architecture: layers of behaviour . . . 77
11.5 A control loop for following a line or a path . . . 77
11.6 Testing: simulate, then trust . . . 78
11.7 Chapter summary . . . 78

12 Practice Problems with Worked Solutions **79**
12.1 The lecture homework . . . 79
12.2 Extra practice . . . 81
12.3 Challenge projects . . . 83

A A Maths Toolkit: What the Equations Assume **84**
A.1 Angles and trigonometry . . . 84
A.2 Vectors and matrices . . . 84
A.3 Calculus in five lines . . . 85
A.4 Exponentials and logarithms . . . 86
A.5 Averages, variance and noise . . . 86
A.6 Complex numbers, in one paragraph . . . 86

B Formula Sheet **87**

C Programs to Run and Change **89**

C.1 Forward and inverse kinematics 89

C.2 A PID loop on a simulated motor 89

C.3 A neuron network that learns XOR 90

C.4 A* on a grid 90

C.5 Differential-drive odometry 91

D Glossary **92**

E Where to Go Next **94**

E.1 Books I recommend 94

E.2 Things to build 94

E.3 A closing word 95

List of Figures

1.1 The sense–think–act loop. The environment closes the loop: whatever the robot does changes what it will sense next. . . . 1

1.2 The three laws as a priority check. A command is passed to Law 1 first; Law 2 and Law 3 are consulted only if the higher laws raise no objection. . . . 4

2.1 Frame 1 is frame 0 turned by 30°. The same physical point P has two different sets of coordinates. . . . 8

3.1 Two-link arm with $l_1 = 0.5$ m, $l_2 = 0.3$ m, $\theta_1 = 30°$, $\theta_2 = 45°$. . . . 13

3.2 Reachable points of the 0.5 m/0.3 m arm with limited joint ranges. The dashed circles mark the maximum reach 0.8 m and the minimum reach 0.2 m. . . . 15

3.3 Two different joint settings put the tip on the same target. Which one a controller picks depends on obstacles and joint limits. . . . 17

3.4 The magnitude of $\det J$ against the elbow angle. It touches zero at the stretched and folded poses. . . . 20

3.5 A 90° joint move in 2 s. The quintic profile starts and ends with zero acceleration, so the motor torque begins and ends gently. . . . 21

4.1 Quadrature signals. Whichever channel rises first tells you which way the shaft is turning; counting all four edges in each slot period multiplies the resolution by four. . . . 25

4.2 Timing of a typical ultrasonic module: after the trigger, the echo line stays high for a time proportional to distance. . . . 25

4.3 The same sine wave digitised with 3 bits and with 6 bits. More bits give a finer staircase and a smaller quantisation error. . . . 27

4.4 A noisy reading of a signal that steps from 1 to 2. Both filters reduce the wobble; notice how they trail the step. . . . 29

4.5 A robot moving at constant speed, measured with noisy sonar (grey dots). The Kalman estimate (red) follows the truth much more closely than the raw data. . . . 31

5.1 Final value of a P-controlled plant with unit gain as K_p grows. The curve approaches 1 but never reaches it. . . . 33

5.2 Responses of a three-stage plant to a unit step under P, PI and PID control. P settles low; PI reaches the target with an overshoot; PID reaches it smoothly. . . . 35

5.3 Step response for $\omega_n = 4$ rad/s and four damping ratios. Light damping rings; critical damping ($\zeta = 1$) is the fastest without overshoot; heavy damping is slow. 36

6.1 Torque (blue) and mechanical power (red) against speed for the motor of the worked example. Power peaks at half the no-load speed. 41

6.2 Top: half-step drive patterns for a two-phase stepper. Bottom: microstepping replaces on/off currents with sine and cosine currents. 42

6.3 A 3000 rpm, 0.1 N m motor behind a gearbox of efficiency 0.8: speed falls and torque rises in proportion to the ratio. 44

6.4 A hydraulic lift. A 50 N push on a 4 cm^2 piston lifts 1250 N on a 100 cm^2 piston. . 45

7.1 Left: forces on a part held between two fingers. Right: the grip force each finger needs for a safety factor of 2. 48

8.1 Left: the artificial neuron. Right: three common activation functions. 54

8.2 Gradient descent on $f(x, y) = \frac{1}{2}(x^2 + 10y^2)$ for three learning rates. The last, 0.215, is just past the stability limit and the path blows up. 55

8.3 A network with 2 inputs, 4 hidden sigmoid neurons and 1 output trained on XOR. Left: the loss falls over 6000 epochs. Right: the learned decision regions separate the two classes with a bent boundary. 56

9.1 Planning on a 20×20 map with walls. Both methods find a shortest path of length 26, but breadth-first search expands 320 cells while A* with a Manhattan heuristic expands 176 (light blue dots). 62

9.2 Alpha–beta pruning on the example tree. After the left branch guarantees 2, a single leaf below 2 in the right branch makes the rest irrelevant. 63

9.3 A GA with 20 individuals optimising the function on the left. The best solution found (red star, $x = 0.455$) is within 0.006 of the true peak ($x = 0.461$) after about ten generations. 65

10.1 Belief that an obstacle is present after $0, 1, \ldots, 5$ consecutive echoes. 69

11.1 Geometry of a differential-drive robot. The body has heading θ; the wheels are a distance b apart. 72

11.2 Left: a path with a speed change. Right: six simulated runs of the same straight command with 3% random wheel error show the drift of dead reckoning. 75

Chapter 1

Meeting the Robot

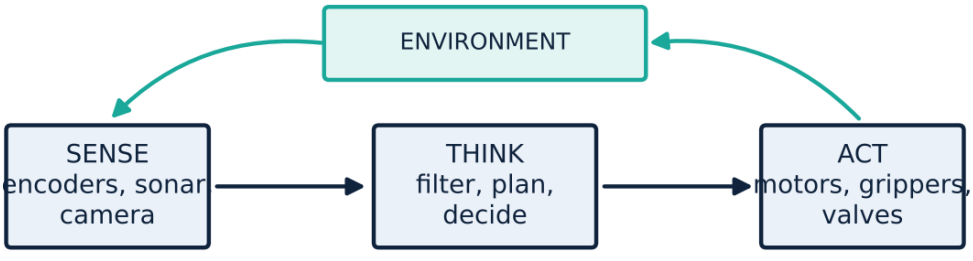
1.1 Why start with a definition

Ask ten people what a robot is and you will hear ten answers: a metal man, a vacuum cleaner, a car that drives itself, a factory arm. All of them are right, and that is exactly the trouble. If the word covers everything, it explains nothing. So let me give you a working definition that I will use for the rest of this book.

The idea to keep

A robot is a machine that **senses** its surroundings, **decides** what to do using a program, and then **acts** on the world with motors or other actuators, and it repeats this cycle by itself without a person guiding every move.

Notice that each of the three verbs eliminates something. A washing machine acts and has a little deciding inside it, but it hardly senses the world, so we do not usually call it a robot. A remote-controlled toy car senses nothing and decides nothing; the person holding the remote does both, so it is just a vehicle. A robot vacuum cleaner has a bump switch, a few distance sensors, a program that chooses where to turn, and wheels, so it passes all three tests.



Every robot, from a toy to a factory arm, repeats this loop many times per second

Figure 1.1: The sense–think–act loop. The environment closes the loop: whatever the robot does changes what it will sense next.

1.2 A short walk through history

People have built self-moving machines for a very long time. Ancient workshops made water clocks with figures that struck bells, and medieval craftsmen made mechanical birds driven by cams and gears. These were automata: they repeated one fixed motion and could not react to anything. The word *robot* itself came much later, from a stage play of the early twentieth century, where it meant a forced worker. The writers of science fiction then gave us the idea of the machine that thinks, and engineers have been trying to catch up with the stories ever since.

The turning point for industry came in the middle of the twentieth century, when a programmable arm was installed on a factory line to lift hot metal parts. After that, arms spread through car factories because welding, painting and lifting are dull, dangerous and repetitive for people but ideal for machines. In the decades that followed, cheap microcontrollers, better sensors and finally machine learning moved the robot from the cage on the factory floor into warehouses, hospitals, farms and homes.

I mention this history for one reason. Each step was made possible not by one clever robot but by a cheaper or better *component*: the motor controller, the encoder, the processor, the camera. When you study the parts in this book, you are studying the real engine of progress.

1.3 The anatomy of a robot

Every robot, whatever it looks like, can be pulled apart into the same handful of subsystems. When I meet a new machine I always walk through this list.

1. **Mechanical structure.** The links, joints and frame. This is the body, and it decides what motions are physically possible.
2. **Actuators.** The muscles: electric motors, hydraulic cylinders, pneumatic valves.
3. **Sensors.** The senses. Some look inward (how far has this joint turned?) and some look outward (how far away is that wall?).
4. **Controller.** The part that closes the loop. It reads the sensors, compares reality with the goal and commands the actuators.
5. **Power supply.** Batteries, regulators, drivers. Beginners ignore this and then wonder why their robot resets whenever the motor starts.
6. **End effector.** Whatever touches the world: a gripper, a welding torch, a suction cup, a pen.
7. **Software and intelligence.** Everything from a ten-line control loop to a planner that searches for a route.

A word from the classroom

When a student tells me “my robot does not work,” I ask which of these seven is the suspect. Nine times out of ten it is power or a loose wire, not the clever algorithm they were worried about. Debug from the bottom of the list upwards.

1.4 Joints, links and degrees of freedom

A *link* is a rigid piece. A *joint* connects two links and allows one kind of relative motion. The two joints you will meet most are the **revolute** joint, which rotates about an axis like a door hinge, and the **prismatic** joint, which slides along an axis like a drawer.

The number of independent motions a mechanism can make is called its **degrees of freedom** (DOF). A free object in space has six: three translations (along x, y, z) and three rotations (about x, y, z). To place a tool at any position *and* at any orientation in space you therefore need at least six well-arranged joints. That is why so many industrial arms have six axes.

For a quick count of planar mechanisms there is a helpful rule due to Grübler. If a planar mechanism has n rigid bodies including the fixed ground, j_1 joints that allow one degree of freedom each and j_2 joints that allow two, then

$$\text{DOF} = 3(n-1) - 2j_1 - j_2. \tag{1.1}$$

The logic is simple. Each free body in a plane has three freedoms; the ground is fixed so it contributes none, giving $3(n-1)$. Every one-freedom joint removes two of them (it ties two of the three relative motions), and every two-freedom joint removes one.

Worked example: A two-link planar arm

Take the arm of Chapter 3: ground plus two moving links gives $n = 3$. There are two revolute joints, each with one freedom, so $j_1 = 2$ and $j_2 = 0$. Then $\text{DOF} = 3(3-1) - 2(2) - 0 = 6 - 4 = 2$. The arm has two degrees of freedom, which matches what we know: two joint angles are enough to describe its pose.

Worked example: A four-bar linkage

Four bodies (ground included) joined in a loop by four revolute joints. Here $n = 4$, $j_1 = 4$. So $\text{DOF} = 3(3) - 2(4) = 9 - 8 = 1$. A four-bar linkage has a single freedom: turn one crank and the whole mechanism responds in one definite way. That is why it is used for windscreen wipers and simple grippers; one motor is enough.

Try it yourself

A planar mechanism has five bodies (ground included) joined by five revolute joints in a closed loop. How many degrees of freedom does it have? (Hint: $n = 5$, $j_1 = 5$. Is the answer what you expected for a loop? What happens if you add a second loop?)

1.5 Types of robot bodies

Engineers classify arms by how the first three joints are arranged, because those three decide where the hand can go (the last three usually decide how it is oriented).

A Cartesian machine, like a 3D printer or a CNC router, is the easiest to program because each joint moves exactly one coordinate. The articulated arm is the most flexible but needs the trigonometry of Chapter 3. A SCARA arm is rigid vertically and compliant horizontally, which is exactly what is wanted for pushing a component into a circuit board.

Table 1.1: Common arm geometries and the shape of the space they can reach.

Type	First three joints	Reachable region
Cartesian (gantry)	slide, slide, slide	a box
Cylindrical	turn, slide, slide	a thick cylindrical shell
Spherical	turn, turn, slide	a thick spherical shell
Articulated (anthropomorphic)	turn, turn, turn	a roughly spherical region
SCARA	turn, turn, slide (vertical)	a flat ring, stiff in the vertical direction

Robots are also classified by what they do. *Industrial manipulators* work in fixed cells. *Mobile robots* move on wheels, tracks or legs. *Service robots* help people in homes and hospitals. *Humanoids* and *legged robots* imitate living bodies and are hard because balance is an active control problem. *Collaborative robots* (cobots) are built to share space with workers safely, with limited force and speed.

1.6 The laws of robotics, and what they really teach

You will have seen the three laws proposed in fiction by Isaac Asimov. In the lecture we animated them. Stated plainly:

1. A robot may not injure a human being or, through inaction, allow a human being to come to harm.
2. A robot must obey orders given by humans, except where such orders would conflict with the First Law.
3. A robot must protect its own existence, as long as this does not conflict with the First or Second Law.

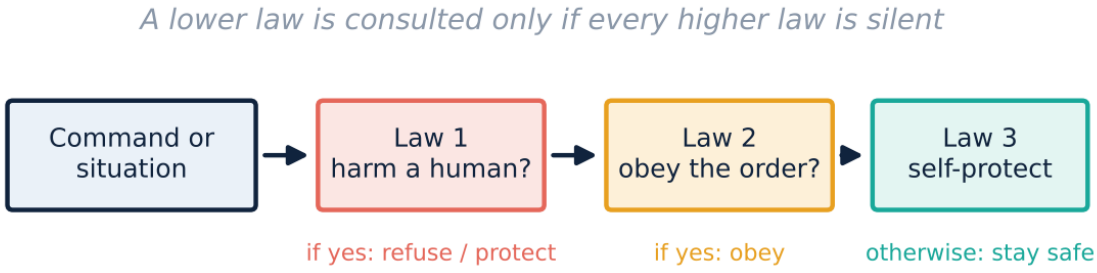


Figure 1.2: The three laws as a priority check. A command is passed to Law 1 first; Law 2 and Law 3 are consulted only if the higher laws raise no objection.

What matters for an engineer is not the wording but the structure: they form a **strict priority order**. In programming language this is just an if–else chain.

```
def decide(action, ordered):
    if harms_human(action) or harms_human(not_acting()):
        return "Law 1: refuse the action, or act to protect"
    elif ordered:
        return "Law 2: obey"
    else:
        return "Law 3: protect myself"
```

The hard part is hidden in the function `harms_human`. How does a camera and a processor decide that a push will hurt someone? A program cannot read "harm" from pixels. This is why real robots are protected by physical and measurable limits instead: speed limits near people, force limits at the joints, safety laser scanners that slow or stop a machine when someone enters its zone, and emergency-stop circuits that do not depend on software at all. Fiction gave us the principle; standards and sensors give us the practice. Please remember that distinction when someone tells you robots "follow the three laws". They do not, and no commercial robot is built that way.

1.7 Accuracy, repeatability and resolution

Three words are often mixed up, and they matter when you read a robot's data sheet.

- **Resolution** is the smallest change the system can *detect or command*. An encoder with 2000 counts per turn has an angular resolution of 0.18°.
- **Accuracy** is how close the robot gets to a point you *asked* for. It is limited by calibration, link lengths that are not exactly what the model says, and flexing under load.
- **Repeatability** is how close the robot gets to the *same* point when sent there again and again. A machine can be wonderfully repeatable and still inaccurate (it always lands 2 mm to the left).

Industrial arms usually quote repeatability of a few hundredths of a millimetre but accuracy that is several times worse, because repeatability only needs a consistent mechanism while accuracy needs a perfect model. If a task needs absolute accuracy, a camera or a touch probe is added to measure and correct. We will return to this idea whenever we talk about feedback.

1.8 Where robots work today

A short tour will help you see why the rest of the book matters.

- **Factories**: welding, painting, assembling, packing. Here the arm's repeatability and its speed are the selling points.
- **Warehouses**: mobile robots carry shelves to pickers, and arms with suction cups pick parcels. Perception (finding the parcel) and grasping (holding it) are the hard problems.
- **Medicine**: surgical systems scale down a surgeon's hand motion and filter out tremor.

- **Agriculture**: machines that identify ripe fruit or weeds by camera.
- **Exploration**: rovers and underwater vehicles, where communication delay forces a degree of autonomy.
- **Education**: low-cost kits, like those you used in class, where the same ideas run at small scale.

1.9 Chapter summary

- A robot senses, decides and acts in a loop.
- Seven subsystems appear in every robot: structure, actuators, sensors, controller, power, end effector and software.
- Degrees of freedom count independent motions; six are needed for full position and orientation in space.
- The three laws are a priority structure from fiction; real safety uses physical limits and standards.
- Resolution, accuracy and repeatability are different, and data sheets rely on the difference.

Chapter 2

Describing Motion: Frames and Transformations

2.1 The problem of “where”

Suppose I tell you a screw is at the point $(1.2,\ 0.6)$ metres. Is that useful? Only if you know where the origin is and which way the axes point. The same screw has completely different numbers if I measure from the base of the arm, from the centre of the table or from the camera mounted on the wrist. In robotics we cannot avoid this, because a single task involves many of these viewpoints at once: the world, the robot base, each link, the tool and the camera. Each viewpoint is called a **coordinate frame**, and the central skill of this chapter is converting a point from one frame to another without making a sign error.

2.2 Rotation in a plane

Begin in two dimensions. Frame 0 has axes x_0, y_0. Frame 1 shares the same origin but is turned anticlockwise by an angle θ. A point P has coordinates (p_x^1, p_y^1) when measured along the axes of frame 1. What are its coordinates along frame 0?

Look at the unit vectors. The axis x_1, seen from frame 0, is the vector $(\cos\theta,\ \sin\theta)$. The axis y_1 is $(-\sin\theta,\ \cos\theta)$. The point is p_x^1 steps along x_1 plus p_y^1 steps along y_1, so

$$\begin{pmatrix} p_x^0 \\ p_y^0 \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}}_{R(\theta)} \begin{pmatrix} p_x^1 \\ p_y^1 \end{pmatrix}. \tag{2.1}$$

The matrix $R(\theta)$ is a **rotation matrix**. Its columns are the new axes written in the old frame, and that is the easiest way to remember it.

Worked example: Converting a point between frames

Frame 1 is rotated by 30° relative to frame 0. A point has coordinates $(1.2,\ 0.6)$ in frame 1. Find its coordinates in frame 0.

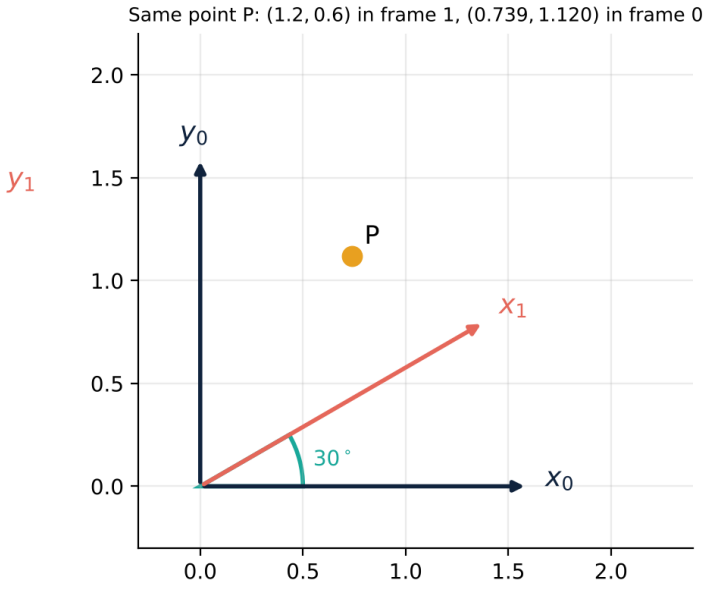


Figure 2.1: Frame 1 is frame 0 turned by 30°. The same physical point P has two different sets of coordinates.

$\cos 30^\circ = 0.8660$ and $\sin 30^\circ = 0.5$. Then

$$p_x^0 = 0.8660(1.2) - 0.5(0.6) = 1.0392 - 0.3 = 0.7392,$$
$$p_y^0 = 0.5(1.2) + 0.8660(0.6) = 0.6 + 0.5196 = 1.1196.$$

So $P = (0.739, 1.120)$ in frame 0. A good check: the distance from the origin must not change. $\sqrt{1.2^2 + 0.6^2} = 1.3416$ and $\sqrt{0.7392^2 + 1.1196^2} = 1.3416$. Rotation never stretches anything.

Two properties of $R(\theta)$ are worth keeping in your head. First, $R(\theta)^{-1} = R(\theta)^T = R(-\theta)$: to undo a rotation, just turn the other way, and the inverse is found by flipping rows and columns, which is much cheaper than a general inverse. Second, $\det R = \cos^2\theta + \sin^2\theta = 1$, which is the algebraic way of saying that shapes are not distorted. Also, two successive planar rotations add: $R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2)$. This single identity is the reason the angle of the second link of an arm is simply $\theta_1 + \theta_2$ in the world frame.

Derivation: why $R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2)$, $R^{-1} = R^T$ and $\det R = 1$

Where the columns come from. In frame 1 the unit vector along x_1 is $(1, 0)^T$. Multiplying by $R(\theta)$ picks out the first column, $(\cos\theta, \sin\theta)^T$, which is exactly where the x_1 axis points as seen from frame 0. The second column does the same for y_1. This is why the columns of R are the new axes.

Two rotations add. Multiply the two matrices, row by row against column:

$$R(\theta_1)R(\theta_2) = \begin{pmatrix} \cos\theta_1\cos\theta_2 - \sin\theta_1\sin\theta_2 & -\cos\theta_1\sin\theta_2 - \sin\theta_1\cos\theta_2 \\ \sin\theta_1\cos\theta_2 + \cos\theta_1\sin\theta_2 & -\sin\theta_1\sin\theta_2 + \cos\theta_1\cos\theta_2 \end{pmatrix}.$$

Now use the two addition formulas, $\cos(a+b) = \cos a\cos b - \sin a\sin b$ and $\sin(a+b) = \sin a\cos b + \cos a\sin b$. The top-left entry is $\cos(\theta_1 + \theta_2)$, the bottom-left is $\sin(\theta_1 + \theta_2)$,

and the right column is the same pair with the signs of a rotation matrix. The product is therefore $R(\theta_1 + \theta_2)$.
The inverse is the transpose. Write $R^T R$ with entries:

$$R^T R = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

$$= \begin{pmatrix} \cos^2\theta + \sin^2\theta & -\cos\theta\sin\theta + \sin\theta\cos\theta \\ -\sin\theta\cos\theta + \cos\theta\sin\theta & \sin^2\theta + \cos^2\theta \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Since $R^T R = I$, the matrix R^T is the inverse of R. The determinant is $\cos\theta \cdot \cos\theta - (-\sin\theta)(\sin\theta) = \cos^2\theta + \sin^2\theta = 1$.

2.3 Rotations in three dimensions

In space we need three elementary rotations, one about each axis:

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{pmatrix}, \; R_y(\beta) = \begin{pmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{pmatrix}, \; R_z(\gamma) = \begin{pmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{2.2}$$

Each is the planar rotation placed inside a three-by-three identity so that the chosen axis stays fixed. Rotations are combined by matrix multiplication, and here something new and important happens: **the order matters**. Turning a book 90° about the vertical axis and then 90° about a horizontal one leaves it in a different place than doing the same two turns in the reverse order. Try it with a book; no equation convinces as well as that. In symbols, $R_x R_z \neq R_z R_x$ in general.

Derivation: multiplying two 3×3 rotations by hand

To find the entry in row i and column j of a product, take row i of the left matrix and multiply it, element by element, with column j of the right matrix, then add. For $R_z(90°)R_x(90°)$ the left matrix has rows $(0,-1,0)$, $(1,0,0)$ and $(0,0,1)$. The right matrix has columns $(1,0,0)^T$, $(0,0,1)^T$ and $(0,-1,0)^T$. Then

$$\begin{aligned} &\text{row 1: } (0)(1) + (-1)(0) + 0 = 0, \quad 0 + 0 + 0 = 0, \quad 0 + (-1)(-1) + 0 = 1, \\ &\text{row 2: } (1)(1) + 0 + 0 = 1, \quad 0, \quad 0, \\ &\text{row 3: } 0, \quad (1)(1) = 1, \quad 0, \end{aligned}$$

which gives the matrix with rows $(0,0,1)$, $(1,0,0)$, $(0,1,0)$ quoted in the example below. Doing the other order the same way gives a different array, and that is the whole point.

Worked example: Order of rotations

Compute $R_z(90°)R_x(90°)$ and $R_x(90°)R_z(90°)$.

With $\cos 90^\circ = 0$, $\sin 90^\circ = 1$:

$$R_z(90^\circ) = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad R_x(90^\circ) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}.$$

$$R_z R_x = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad R_x R_z = \begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{pmatrix}.$$

The two products are different matrices, so the two orders give different final orientations.

A caution that I give every class: the product $R_1 R_2$ can be read two ways. If the second rotation is about the *moving* axes it is a post-multiplication, $R = R_1 R_2$. If the second rotation is about the *fixed* world axes, it is a pre-multiplication, $R = R_2 R_1$. Most mistakes in 3D kinematics come from mixing these two readings.

2.4 Translation and homogeneous transforms

A frame can also be displaced. Moving the origin by a vector $\mathbf{d}$ adds $\mathbf{d}$ to every coordinate, so a general change of frame is

$$\mathbf{p}^0 = R\,\mathbf{p}^1 + \mathbf{d}. \tag{2.3}$$

Rotation is a multiplication and translation is an addition, which is untidy when you chain many frames. A neat trick fixes this. Append a 1 to each point and enlarge the matrix:

$$\begin{pmatrix} \mathbf{p}^0 \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} R & \mathbf{d} \\ \mathbf{0}^T & 1 \end{pmatrix}}_{T} \begin{pmatrix} \mathbf{p}^1 \\ 1 \end{pmatrix}. \tag{2.4}$$

The matrix T is called a **homogeneous transformation**. Now a rotation and a translation together are a single multiplication, and a chain of frames is a chain of multiplications:

$$T_n^0 = T_1^0\, T_2^1 \cdots T_n^{n-1}. \tag{2.5}$$

This is the backbone of robot kinematics. Notice how it reads: to find the tool in the world, multiply the transform from world to base, then base to link 1, then link 1 to link 2, and so on to the tool.

The inverse has a closed form that avoids a numerical matrix inverse:

$$T^{-1} = \begin{pmatrix} R^T & -R^T\mathbf{d} \\ \mathbf{0}^T & 1 \end{pmatrix}. \tag{2.6}$$

You can verify it by multiplying T and T^{-1} and checking you obtain the identity.

Worked example: A camera on a wrist

A planar arm has its tool frame located at $(0.50,\ 0.40)$ in the base frame, rotated 90° from it. A camera, mounted in the tool frame at $(0.10,\ 0)$, sees a bolt at $(0.20,\ 0.05)$ in the camera frame (assume the camera axes are parallel to the tool axes). Where is the bolt in the base

frame?
Step 1. Bolt in tool frame: camera offset plus bolt: $(0.10 + 0.20,\ 0 + 0.05) = (0.30,\ 0.05)$.
Step 2. Rotate by 90°: $R(90^\circ) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, so $R(0.30, 0.05)^T = (-0.05,\ 0.30)$.
Step 3. Add the tool origin: $(0.50 - 0.05,\ 0.40 + 0.30) = (0.45,\ 0.70)$.
The bolt is at $(0.45,\ 0.70)$ in the base frame. The arm can now be commanded to go there. This is precisely the calculation a vision-guided pick-and-place robot does every second.

Derivation: the homogeneous trick and the inverse of T

Why the extra 1 works. Multiply the block matrix by the extended point, block by block:

$$\begin{pmatrix} R & \mathbf{d} \\ \mathbf{0}^T & 1 \end{pmatrix} \begin{pmatrix} \mathbf{p} \\ 1 \end{pmatrix} = \begin{pmatrix} R\mathbf{p} + \mathbf{d} \cdot 1 \\ \mathbf{0}^T\mathbf{p} + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} R\mathbf{p} + \mathbf{d} \\ 1 \end{pmatrix}.$$

The top block is exactly the rotate-then-translate rule, and the bottom is just the number 1 again, so the trick can be repeated.
Why chains multiply. If $\mathbf{p}^0 = T_1^0\mathbf{p}^1$ and $\mathbf{p}^1 = T_2^1\mathbf{p}^2$ (extended points), substitute the second into the first: $\mathbf{p}^0 = T_1^0T_2^1\,\mathbf{p}^2$. So $T_2^0 = T_1^0T_2^1$, and the pattern continues for any number of frames.
The inverse. Start from $\mathbf{p}^0 = R\mathbf{p}^1 + \mathbf{d}$ and solve for $\mathbf{p}^1$. Subtract $\mathbf{d}$: $R\mathbf{p}^1 = \mathbf{p}^0 - \mathbf{d}$. Multiply both sides on the left by R^T and use $R^TR = I$: $\mathbf{p}^1 = R^T\mathbf{p}^0 - R^T\mathbf{d}$. In block form this is the matrix $\begin{pmatrix} R^T & -R^T\mathbf{d} \\ \mathbf{0}^T & 1 \end{pmatrix}$, which is the quoted formula.
Numerical check with the wrist camera example. There $R = R(90^\circ)$ and $\mathbf{d} = (0.50, 0.40)^T$, so $R^T = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ and $-R^T\mathbf{d} = -(0.40,\ -0.50)^T = (-0.40,\ 0.50)^T$. Apply it to the bolt at $(0.45, 0.70)$ in the base frame:

$$R^T \begin{pmatrix} 0.45 \\ 0.70 \end{pmatrix} + \begin{pmatrix} -0.40 \\ 0.50 \end{pmatrix} = \begin{pmatrix} 0.70 \\ -0.45 \end{pmatrix} + \begin{pmatrix} -0.40 \\ 0.50 \end{pmatrix} = \begin{pmatrix} 0.30 \\ 0.05 \end{pmatrix},$$

which is the bolt's position in the tool frame found before. The inverse has taken us back.

2.5 Describing orientation: Euler angles and a warning

Nine numbers in a rotation matrix are redundant, because a 3D orientation has only three independent freedoms. People like to describe orientation with three angles, for instance *roll, pitch and yaw*, i.e. rotations about x, y and z in some agreed order. This is easy to picture, but there is a catch called **gimbal lock**: when the middle angle reaches $\pm 90^\circ$, two of the axes line up and one degree of freedom disappears, so different angle triples give the same orientation and small physical motions need enormous angle changes. Aircraft and camera gimbals suffer from it. Robots avoid it by using a different representation, the **unit quaternion**, which has four numbers, no singularity and cheap composition. You do not need quaternion algebra today. It is enough to know why software stores orientation that way.

2.6 Frames in a real work cell

Let me show you how many frames can appear in something as ordinary as a pick-and-place cell. There is a *world* frame fixed to the floor, a *base* frame at the foot of the arm, a frame for each link, a *tool* frame at the gripper tip, a *camera* frame, and an *object* frame on the part. The question "can the gripper reach the part?" is answered by chaining

$$T_{\text{object}}^{\text{base}} = \left(T_{\text{base}}^{\text{world}}\right)^{-1} T_{\text{camera}}^{\text{world}}\, T_{\text{object}}^{\text{camera}},$$

and then asking whether the arm can bring its tool frame to that place. Calibration, which every robot cell needs, is nothing more than measuring the fixed transforms (such as $T_{\text{camera}}^{\text{world}}$) accurately.

A word from the classroom

When your answer is wrong by a mirror image or by a quarter turn, check three things: did you use R when you needed R^T, did you multiply in the right order, and did you add the translation after the rotation instead of before? That list solves most of the doubts I get.

2.7 Chapter summary

- A coordinate frame is an origin plus axes; every robot task involves several.
- The columns of a rotation matrix are the new axes written in the old frame; $R^{-1} = R^T$ and $\det R = 1$.
- In three dimensions the order of rotations matters.
- Homogeneous transforms combine rotation and translation, and chains of frames become products of matrices.
- Quaternions are used instead of Euler angles in software to avoid gimbal lock.

Chapter 3

Arm Kinematics

Kinematics is the geometry of motion without asking about forces. For a robot arm it answers two questions that are mirror images of each other. *Forward kinematics*: given the joint angles, where is the hand? *Inverse kinematics*: given where I want the hand, what should the joint angles be? The first has exactly one answer and is easy. The second may have several answers or none, and it is the one that actually matters in practice.

3.1 Forward kinematics of a two-link arm

Consider the planar arm of Figure 3.1. The first link of length l_1 turns by θ_1 measured from the horizontal base line. The second link of length l_2 is attached at the end of the first and turns by θ_2 *relative to the first link*. Its absolute direction is therefore $\theta_1 + \theta_2$.

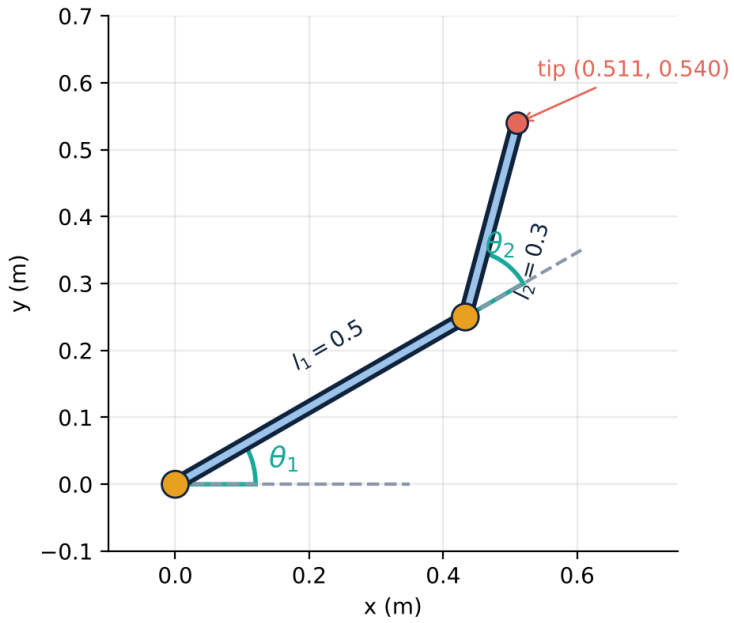


Figure 3.1: Two-link arm with $l_1 = 0.5$ m, $l_2 = 0.3$ m, $\theta_1 = 30°$, $\theta_2 = 45°$.

The elbow sits at $(l_1 \cos\theta_1,\ l_1 \sin\theta_1)$. The tool tip is a further l_2 along the absolute direction

$\theta_1 + \theta_2$:

$$x = l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2), \tag{3.1}$$
$$y = l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2). \tag{3.2}$$

These two equations are the forward kinematics. There is nothing else to it.

Derivation: the forward equations from the geometry

Draw the first link alone. It has length l_1 and makes angle θ_1 with the horizontal, so it forms the hypotenuse of a right triangle. The horizontal side is $l_1 \cos\theta_1$ (adjacent over hypotenuse is the cosine) and the vertical side is $l_1 \sin\theta_1$. That is the position of the elbow. The second link is a vector of length l_2 that points in the absolute direction $\theta_1 + \theta_2$. The same triangle argument gives horizontal part $l_2 \cos(\theta_1 + \theta_2)$ and vertical part $l_2 \sin(\theta_1 + \theta_2)$. Vectors add head to tail, so the tip is the elbow plus this second vector, which is the pair of equations above.

The reason the absolute direction is $\theta_1 + \theta_2$ is the rule of the previous chapter: turning by θ_2 about the elbow, after the whole arm has been turned by θ_1, is the product $R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2)$. In matrix language the tip is $R(\theta_1)\left[\begin{pmatrix} l_1 \\ 0 \end{pmatrix} + R(\theta_2)\begin{pmatrix} l_2 \\ 0 \end{pmatrix}\right]$, and multiplying out gives the same result.

Worked example: Locating the tip

With $l_1 = 0.5$ m, $l_2 = 0.3$ m, $\theta_1 = 30^\circ$ and $\theta_2 = 45^\circ$ we have $\theta_1 + \theta_2 = 75^\circ$.

$$x = 0.5\cos 30^\circ + 0.3\cos 75^\circ = 0.5(0.8660) + 0.3(0.2588) = 0.4330 + 0.0776 = 0.5107 \text{ m},$$
$$y = 0.5\sin 30^\circ + 0.3\sin 75^\circ = 0.5(0.5) + 0.3(0.9659) = 0.2500 + 0.2898 = 0.5398 \text{ m}.$$

The tip is at $(0.511, 0.540)$ m. Distance from the base: $\sqrt{0.5107^2 + 0.5398^2} = 0.743$ m, which lies between the minimum reach 0.2 m and the maximum reach 0.8 m, so the answer is plausible.

3.2 The workspace

The **workspace** is the set of all points the tip can reach. For our two-link arm it is easy to find without any heavy algebra. The distance from the base to the tip is largest when the arm is fully stretched, $\theta_2 = 0$, giving $l_1 + l_2$. It is smallest when the elbow is folded back as far as possible, $\theta_2 = \pm 180^\circ$, giving $|l_1 - l_2|$. Between those two extremes every distance is possible. So the workspace is a ring (an annulus) with inner radius $|l_1 - l_2|$ and outer radius $l_1 + l_2$. With real joints that cannot rotate through 360° only part of the ring is reachable, as the scatter in Figure 3.2 shows.

The idea to keep

Before computing any inverse kinematics, check that the target lies inside the workspace. For the planar two-link arm the test is simply $|l_1 - l_2| \le r \le l_1 + l_2$ where r is the distance of the target from the base.

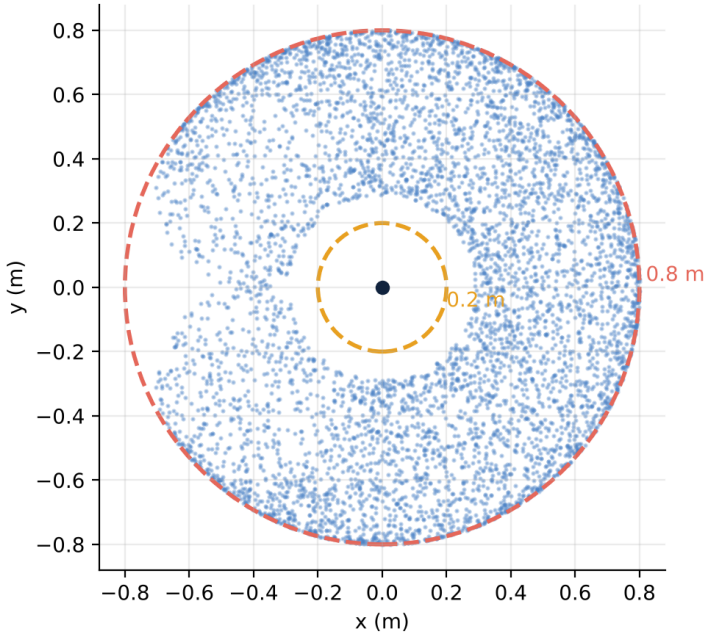


Figure 3.2: Reachable points of the 0.5 m/0.3 m arm with limited joint ranges. The dashed circles mark the maximum reach 0.8 m and the minimum reach 0.2 m.

3.3 Inverse kinematics of the two-link arm

Now the harder direction. We are given a target (x, y) and must find θ_1 and θ_2. A pleasant feature of the planar two-link arm is that it has a clean closed-form solution, which I derive in two steps so you can see where each term comes from.

Step 1: the elbow angle from the distance alone. Let $r^2 = x^2 + y^2$. The base, the elbow and the tip form a triangle with sides l_1, l_2 and r. The law of cosines gives

$$r^2 = l_1^2 + l_2^2 + 2 l_1 l_2 \cos\theta_2 \quad \Longrightarrow \quad \cos\theta_2 = \frac{x^2 + y^2 - l_1^2 - l_2^2}{2\, l_1 l_2}. \tag{3.3}$$

(The plus sign appears because θ_2 is the relative angle at the elbow measured from the straight-out direction, not the interior angle of the triangle.) If the right-hand side is outside $[-1, 1]$ the target is out of reach. Otherwise

$$\theta_2 = \pm \arccos\left(\frac{x^2 + y^2 - l_1^2 - l_2^2}{2\, l_1 l_2} \right). \tag{3.4}$$

The two signs are the two "elbow" configurations.

Derivation: the elbow angle, with no triangle sketch needed

Write the tip position with the forward equations and square and add them, since $r^2 = x^2 + y^2$:

$$\begin{aligned} r^2 &= \big[l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2)\big]^2 + \big[l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2)\big]^2 \\ &= l_1^2(\cos^2\theta_1 + \sin^2\theta_1) + l_2^2\big(\cos^2(\theta_1 + \theta_2) + \sin^2(\theta_1 + \theta_2)\big) \\ &\quad + 2 l_1 l_2 \big[\cos\theta_1 \cos(\theta_1 + \theta_2) + \sin\theta_1 \sin(\theta_1 + \theta_2)\big]. \end{aligned}$$

Each bracket of squares equals 1. The last bracket has the form $\cos a \cos b + \sin a \sin b =$

$\cos(a-b)$ with $a=\theta_1+\theta_2$ and $b=\theta_1$, so it equals $\cos\theta_2$. Therefore

$$r^2 = l_1^2 + l_2^2 + 2\,l_1 l_2 \cos\theta_2,$$

and solving for $\cos\theta_2$ gives the formula in the text. Notice that θ_1 has disappeared: the distance to the tip depends only on how much the elbow is bent. This is also why the workspace limits are l_1+l_2 (when $\cos\theta_2=1$) and $|l_1-l_2|$ (when $\cos\theta_2=-1$).

Step 2: the shoulder angle. The direction to the target is $\operatorname{atan2}(y,x)$. The first link is tilted away from that direction by the angle $\operatorname{atan2}\big(l_2\sin\theta_2,\ l_1+l_2\cos\theta_2\big)$, which is the small angle that the second link adds at the shoulder. So

$$\theta_1 = \operatorname{atan2}(y,x) - \operatorname{atan2}\big(l_2\sin\theta_2,\ l_1+l_2\cos\theta_2\big). \tag{3.5}$$

Use `atan2` and not a plain arctangent: it knows which quadrant the point is in.

Derivation: where the shoulder formula comes from

Expand $\cos(\theta_1+\theta_2)=\cos\theta_1\cos\theta_2-\sin\theta_1\sin\theta_2$ and $\sin(\theta_1+\theta_2)=\sin\theta_1\cos\theta_2+\cos\theta_1\sin\theta_2$ inside the forward equations and collect terms in $\cos\theta_1$ and $\sin\theta_1$:

$$x = (l_1+l_2\cos\theta_2)\cos\theta_1 - (l_2\sin\theta_2)\sin\theta_1,$$
$$y = (l_1+l_2\cos\theta_2)\sin\theta_1 + (l_2\sin\theta_2)\cos\theta_1.$$

Give the two bracketed numbers names, $k_1=l_1+l_2\cos\theta_2$ and $k_2=l_2\sin\theta_2$. Both are known once θ_2 has been found. The equations now read

$$\begin{pmatrix}x\\y\end{pmatrix} = \begin{pmatrix}\cos\theta_1 & -\sin\theta_1\\ \sin\theta_1 & \cos\theta_1\end{pmatrix}\begin{pmatrix}k_1\\k_2\end{pmatrix} = R(\theta_1)\begin{pmatrix}k_1\\k_2\end{pmatrix}.$$

The target is the point (k_1,k_2) turned through the angle θ_1. The direction of the target is therefore the direction of (k_1,k_2) plus θ_1, that is $\operatorname{atan2}(y,x)=\theta_1+\operatorname{atan2}(k_2,k_1)$, which rearranges to the stated formula. As a check, $k_1^2+k_2^2 = l_1^2+2l_1l_2\cos\theta_2+l_2^2 = r^2$, in agreement with Step 1. In the worked example $k_1 = 0.5+0.3(0.2333) = 0.5700$ and $k_2 = 0.3\sin 76.5^\circ = 0.2917$, the two numbers that appeared inside the atan2.

Worked example: Reaching a point

Same arm, target $(0.5,\ 0.4)$ m.

$x^2+y^2 = 0.25+0.16 = 0.41$. Then $\cos\theta_2 = \dfrac{0.41-0.25-0.09}{2(0.5)(0.3)} = \dfrac{0.07}{0.30} = 0.2333$, so $\theta_2 = \pm 76.5^\circ$.

Elbow-down ($\theta_2 = +76.5^\circ$): $\operatorname{atan2}(0.4, 0.5) = 38.66^\circ$. The correction is $\operatorname{atan2}(0.3\sin 76.5^\circ,\ 0.5+0.3\cos 76.5^\circ) = \operatorname{atan2}(0.2917,\ 0.5700) = 27.10^\circ$. So $\theta_1 = 38.66-27.10 = 11.56^\circ$.

Elbow-up ($\theta_2=-76.5^\circ$): the correction becomes -27.10° and $\theta_1 = 38.66+27.10 = 65.76^\circ$.

Check the first solution by forward kinematics: $x = 0.5\cos 11.56^\circ + 0.3\cos(11.56^\circ+76.5^\circ) = 0.4899+0.0102 = 0.5001$ and $y = 0.5\sin 11.56^\circ + 0.3\sin 88.06^\circ = 0.1002+0.2998 = 0.4000$.

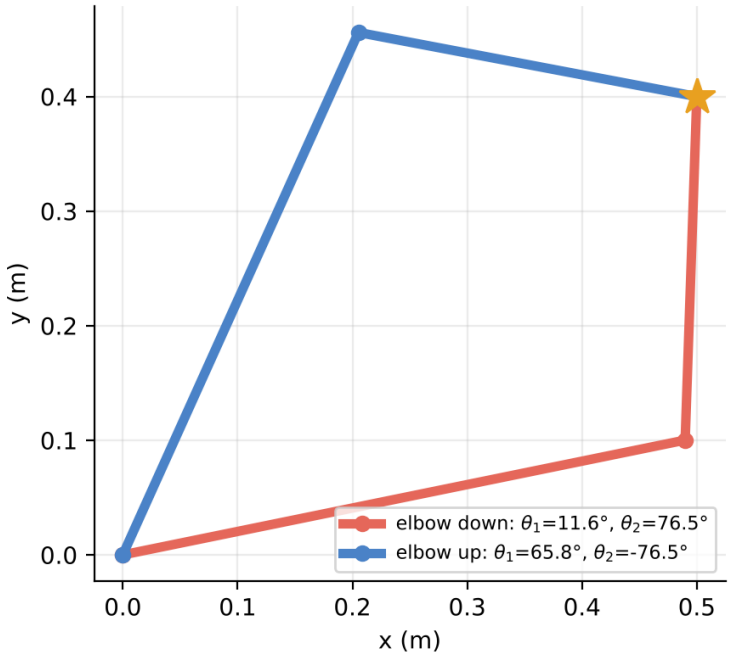


Figure 3.3: Two different joint settings put the tip on the same target. Which one a controller picks depends on obstacles and joint limits.

A word from the classroom

Always verify an inverse kinematics result by putting the angles back into the forward equations. It takes thirty seconds and it has saved more than one demonstration day. If the check fails by a small amount, suspect rounding; if it fails by a mirror image, you probably chose the wrong sign of θ_2.

For arms with six joints the algebra is longer, but the idea survives. If the last three axes intersect at a point (a *spherical wrist*, as on most industrial arms), the problem splits: the first three joints place the wrist point, and the last three orient the tool. That decoupling is the reason spherical wrists are so common.

3.4 The Denavit–Hartenberg convention

For longer chains we want a recipe that works for any serial robot. The Denavit–Hartenberg (DH) convention gives each link four numbers and turns every joint into the same standard transformation. For link i the numbers are:

- a_i: the length of the common perpendicular between consecutive joint axes,
- α_i: the twist angle between those axes,
- d_i: the offset along the joint axis,
- θ_i: the rotation about the joint axis.

Of these, one is the joint variable (it is θ_i for a revolute joint and d_i for a prismatic one) and the other three are constants. The transform from frame $i-1$ to frame i is

$$A_i = \begin{pmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{3.6}$$

and the full arm is $T_n^0 = A_1 A_2 \cdots A_n$. For our planar two-link arm, $\alpha_i = 0$, $d_i = 0$, $a_1 = l_1$, $a_2 = l_2$, and multiplying $A_1 A_2$ reproduces exactly the forward equations above. You can confirm this in a few lines of algebra using $\cos\theta_1\cos\theta_2 - \sin\theta_1\sin\theta_2 = \cos(\theta_1+\theta_2)$ and its sine counterpart.

Derivation: where the DH matrix comes from, and the two-link check

The DH transform is four simple moves made one after the other: turn by θ_i about the joint axis (z), slide d_i along that axis, slide a_i along the new x axis, then twist by α_i about x. As a product, $A_i = R_z(\theta_i)\,T_z(d_i)\,T_x(a_i)\,R_x(\alpha_i)$. The rotation part is $R_z(\theta_i)R_x(\alpha_i)$. Multiplying row by column,

$$\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta\cos\alpha & \sin\theta\sin\alpha \\ \sin\theta & \cos\theta\cos\alpha & -\cos\theta\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{pmatrix},$$

which is the top-left block of A_i. The two slides together move the origin by $(a_i, 0, d_i)$ in the turned frame, and turning that vector by $R_z(\theta_i)$ gives $(a_i\cos\theta_i,\ a_i\sin\theta_i,\ d_i)$, the last column.

For the planar arm put $\alpha = 0$ and $d = 0$. Then $A_i = \begin{pmatrix} c_i & -s_i & 0 & a_i c_i \\ s_i & c_i & 0 & a_i s_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ and the last column of A_1A_2 is A_1 applied to $(a_2c_2,\ a_2s_2,\ 0,\ 1)^T$:

$$\begin{aligned} x &= c_1(a_2c_2) - s_1(a_2s_2) + a_1c_1 = a_1\cos\theta_1 + a_2\cos(\theta_1+\theta_2), \\ y &= s_1(a_2c_2) + c_1(a_2s_2) + a_1s_1 = a_1\sin\theta_1 + a_2\sin(\theta_1+\theta_2). \end{aligned}$$

With $a_1 = l_1$ and $a_2 = l_2$ these are the forward equations again.

3.5 The Jacobian: from joint speed to hand speed

Position is only half of the story. Robots move, and we often need to know how fast the hand moves when the joints turn at certain rates. Differentiate the forward kinematics with respect to time. For the two-link arm,

$$\dot{x} = -\big(l_1\sin\theta_1 + l_2\sin(\theta_1+\theta_2)\big)\dot{\theta}_1 - l_2\sin(\theta_1+\theta_2)\,\dot{\theta}_2, \tag{3.7}$$

$$\dot{y} = \ \ \big(l_1\cos\theta_1 + l_2\cos(\theta_1+\theta_2)\big)\dot{\theta}_1 + l_2\cos(\theta_1+\theta_2)\,\dot{\theta}_2. \tag{3.8}$$

In matrix form $\dot{\mathbf{p}} = J(\dot{\theta})$ with the **Jacobian**

$$J = \begin{pmatrix} -l_1s_1 - l_2s_{12} & -l_2s_{12} \\ l_1c_1 + l_2c_{12} & l_2c_{12} \end{pmatrix}, \qquad s_1 = \sin\theta_1,\ c_{12} = \cos(\theta_1+\theta_2) \text{ etc.} \tag{3.9}$$

The Jacobian is the local conversion factor between the joint world and the hand world, and it changes with the pose.

Derivation: the Jacobian by differentiation

Time enters only through the angles, which change as the joints move, so we need the chain rule. The inside of $\cos(\theta_1+\theta_2)$ has derivative $\dot\theta_1+\dot\theta_2$. Therefore

$$\dot x = \frac{d}{dt}\left[l_1\cos\theta_1 + l_2\cos(\theta_1+\theta_2)\right] = -l_1\sin\theta_1\,\dot\theta_1 - l_2\sin(\theta_1+\theta_2)\,(\dot\theta_1+\dot\theta_2).$$

Now collect the terms that multiply $\dot\theta_1$ and those that multiply $\dot\theta_2$:

$$\dot x = \underbrace{-\big(l_1\sin\theta_1 + l_2\sin(\theta_1+\theta_2)\big)}_{\partial x/\partial\theta_1}\,\dot\theta_1 + \underbrace{\big(-l_2\sin(\theta_1+\theta_2)\big)}_{\partial x/\partial\theta_2}\,\dot\theta_2.$$

The same steps on y give $\partial y/\partial\theta_1 = l_1\cos\theta_1 + l_2\cos(\theta_1+\theta_2)$ and $\partial y/\partial\theta_2 = l_2\cos(\theta_1+\theta_2)$. So the Jacobian is simply the table of partial derivatives

$$J = \begin{pmatrix} \partial x/\partial\theta_1 & \partial x/\partial\theta_2 \\ \partial y/\partial\theta_1 & \partial y/\partial\theta_2 \end{pmatrix}.$$

Each entry answers a small question: if only this joint moves a little, how much does the hand move in this direction? That reading works for any robot, however many joints it has.

Worked example: Hand speed

At $\theta_1 = 30^\circ$, $\theta_2 = 45^\circ$ the joints turn at $\dot\theta_1 = 0.4$ rad/s and $\dot\theta_2 = 0.2$ rad/s. With $s_1 = 0.5$, $c_1 = 0.866$, $s_{12} = \sin 75^\circ = 0.9659$, $c_{12} = 0.2588$:

$$J = \begin{pmatrix} -0.25 - 0.2898 & -0.2898 \\ 0.4330 + 0.0776 & 0.0776 \end{pmatrix} = \begin{pmatrix} -0.5398 & -0.2898 \\ 0.5106 & 0.0776 \end{pmatrix}.$$

$$\dot x = -0.5398(0.4) - 0.2898(0.2) = -0.2159 - 0.0580 = -0.2739 \text{ m/s},$$
$$\dot y = 0.5106(0.4) + 0.0776(0.2) = 0.2042 + 0.0155 = 0.2197 \text{ m/s}.$$

Speed of the tip: $\sqrt{0.2739^2 + 0.2197^2} = 0.351$ m/s.

The same matrix works backwards when you want the hand to move at a chosen velocity: $\dot{} = J^{-1}\dot{\mathbf{p}}$, provided J can be inverted. Which brings us to the dangerous place.

3.6 Singularities

For the two-link arm, $\det J = l_1 l_2 \sin\theta_2$. When $\theta_2 = 0$ (arm fully stretched) or $\theta_2 = \pm 180^\circ$ (arm folded) the determinant is zero and J cannot be inverted. These poses are called **singularities**.

Derivation: the determinant and the inverse Jacobian

For a 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ the determinant is $ad - bc$ and the inverse is $\dfrac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Insert the entries of J:

$$\begin{aligned} \det J &= (-l_1 s_1 - l_2 s_{12})(l_2 c_{12}) - (-l_2 s_{12})(l_1 c_1 + l_2 c_{12}) \\ &= -l_1 l_2 s_1 c_{12} - l_2^2 s_{12} c_{12} + l_1 l_2 s_{12} c_1 + l_2^2 s_{12} c_{12} \\ &= l_1 l_2 \left(s_{12} c_1 - c_{12} s_1\right) = l_1 l_2 \sin\big((\theta_1 + \theta_2) - \theta_1\big) = l_1 l_2 \sin\theta_2 . \end{aligned}$$

The l_2^2 terms cancel and the rest is the sine subtraction formula. The inverse Jacobian contains the factor $1/\det J$, so every entry grows without limit as $\sin\theta_2 \to 0$. A feel for the size: for $l_1 = 0.5$ and $l_2 = 0.3$, $\det J = 0.15 \sin\theta_2$. At $\theta_2 = 45^\circ$ this is 0.106, at $\theta_2 = 5^\circ$ it is 0.013. The joint speeds needed for the same hand velocity are about eight times larger in the second pose.

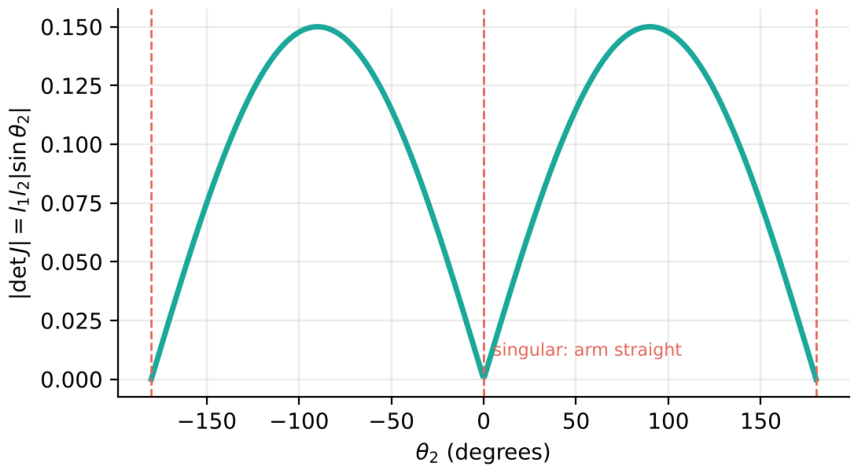


Figure 3.4: The magnitude of $\det J$ against the elbow angle. It touches zero at the stretched and folded poses.

What is happening physically? When the arm is fully stretched, the hand cannot move further outward no matter how the joints turn, so one direction of motion is lost. Worse, to move the hand along a direction that is almost impossible, the inverse Jacobian demands enormous joint speeds. A robot following a straight line that passes near a singularity will therefore either slow down sharply or have its joints race at their limits. Programmers avoid the problem by planning paths that stay away from stretched or folded poses, and controllers include a safeguard that limits joint speed near singularities.

3.7 Moving smoothly: trajectories

Knowing where to start and finish is not enough; a robot must also travel between the two. If a joint is told to jump instantly from 0° to 90° the motor would need infinite acceleration, the gears would be shocked and the load would swing. We prescribe a smooth **trajectory** $q(t)$ instead.

A cubic polynomial $q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$ can satisfy four conditions: start position, end position, and zero speed at both ends. For a move from q_0 to q_f in time T it works out to

$$q(t) = q_0 + (q_f - q_0)\left[3\left(\tfrac{t}{T}\right)^2 - 2\left(\tfrac{t}{T}\right)^3\right]. \tag{3.10}$$

Differentiating gives a speed that starts and ends at zero, with a peak $\dot{q}_{\max} = \frac{3}{2}(q_f - q_0)/T$ in the middle. A quintic polynomial adds zero acceleration at the ends as well, which removes the sudden jerk in torque.

Derivation: solving for the polynomial coefficients

Let $\Delta = q_f - q_0$ and $s = t/T$. For the cubic, $q = a_0 + a_1 t + a_2 t^2 + a_3 t^3$ and $\dot{q} = a_1 + 2a_2 t + 3a_3 t^2$. The four conditions give four equations:

$$q(0) = q_0 \Rightarrow a_0 = q_0, \qquad \dot{q}(0) = 0 \Rightarrow a_1 = 0,$$
$$\dot{q}(T) = 0 \Rightarrow 2a_2 T + 3a_3 T^2 = 0 \;\Rightarrow\; a_2 = -\tfrac{3}{2}a_3 T, \qquad q(T) = q_f \Rightarrow a_2 T^2 + a_3 T^3 = \Delta.$$

Substituting a_2 into the last equation gives $-\frac{3}{2}a_3 T^3 + a_3 T^3 = -\frac{1}{2}a_3 T^3 = \Delta$, so $a_3 = -2\Delta/T^3$ and $a_2 = 3\Delta/T^2$. Hence $q = q_0 + \Delta\,(3s^2 - 2s^3)$, the formula in the text. The speed is $\dot{q} = \Delta(6t/T^2 - 6t^2/T^3)$. It is greatest where the acceleration $\ddot{q} = \Delta(6/T^2 - 12t/T^3)$ is zero, that is at $t = T/2$, and there $\dot{q} = \Delta(3/T - 1.5/T) = 1.5\,\Delta/T$.
For the quintic there are six conditions (position, speed and acceleration at both ends). The three at $t = 0$ force $a_0 = q_0$, $a_1 = 0$, $a_2 = 0$. The three at $t = T$ become

$$a_3 T^3 + a_4 T^4 + a_5 T^5 = \Delta, \quad 3a_3 T^2 + 4a_4 T^3 + 5a_5 T^4 = 0, \quad 6a_3 T + 12a_4 T^2 + 20a_5 T^3 = 0.$$

Solving this small linear system gives $a_3 = 10\Delta/T^3$, $a_4 = -15\Delta/T^4$, $a_5 = 6\Delta/T^5$. Check by substitution: $10 - 15 + 6 = 1$, $30 - 60 + 30 = 0$, $60 - 180 + 120 = 0$. So $q = q_0 + \Delta(10s^3 - 15s^4 + 6s^5)$, with peak speed $1.875\,\Delta/T$ at the middle.
Numbers for a 90° *move in* 2 *s, at* $t = 0.5$ *s* ($s = 0.25$). Cubic: $3(0.0625) - 2(0.015625) = 0.15625$, so $q = 14.06^\circ$. Quintic: $10(0.015625) - 15(0.003906) + 6(0.000977) = 0.1035$, so $q = 9.32^\circ$. The quintic starts more gently, and its peak speed is $84.4^\circ/\text{s}$ against $67.5^\circ/\text{s}$ for the cubic.

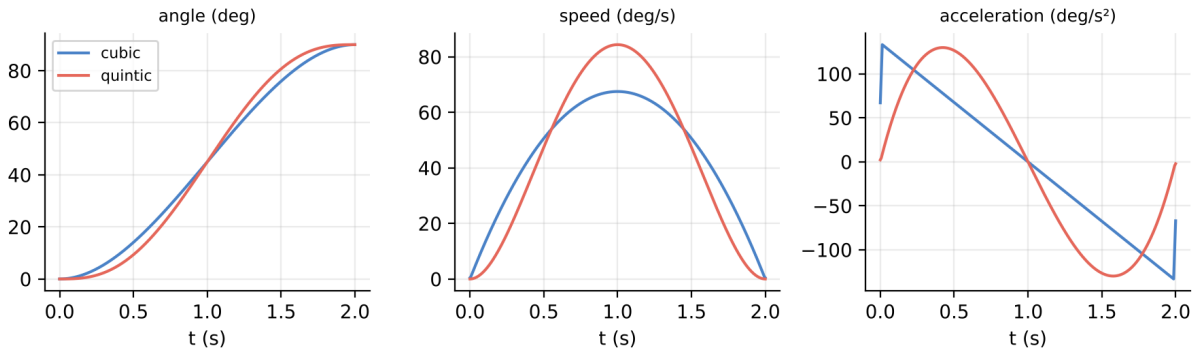


Figure 3.5: A 90° joint move in 2 s. The quintic profile starts and ends with zero acceleration, so the motor torque begins and ends gently.

Worked example: Peak speed

A joint must move 90° in 2 s with a cubic profile. Peak speed is $1.5 \times 90/2 = 67.5^\circ/\text{s}$. If the joint's motor can only do $60^\circ/\text{s}$, the move must be stretched: $T \geq 1.5 \times 90/60 = 2.25$ s.

3.8 Chapter summary

- Forward kinematics is a direct calculation; inverse kinematics may have several solutions or none.
- The two-link planar arm has workspace $|l_1 - l_2| \leq r \leq l_1 + l_2$ and a closed-form inverse using the law of cosines.
- DH parameters give a uniform recipe for any serial chain.
- The Jacobian maps joint rates to hand velocity, and its determinant vanishes at singular poses.
- Smooth polynomial trajectories protect the machine.

Chapter 4

Sensors and Signals

A robot that cannot sense is a very expensive toy that repeats itself. This chapter walks through what sensors are, how a few important ones work in detail, how their raw signals become numbers a processor can use, and how we deal with the fact that every measurement lies a little.

4.1 What a sensor does

A sensor converts some physical quantity (angle, distance, temperature, light, force) into an electrical signal. Most of the time that signal is a voltage that changes smoothly with the quantity, or a stream of pulses. The processor then needs to read it, calibrate it, filter it and use it. Think of every sensor as a small translator with three properties you should always ask about: its **range** (what it can measure), its **resolution** (the smallest change it can show) and its **noise** (how much the reading wobbles when nothing changes).

Sensors are grouped in two ways. By *what they look at*: **proprioceptive** sensors observe the robot itself (joint encoders, motor current, battery voltage, a gyroscope), while **exteroceptive** sensors observe the outside world (cameras, sonar, LiDAR, bump switches). By *how they produce a signal*: **passive** sensors only receive (a camera in daylight, a thermistor) and **active** sensors emit energy and listen for the effect (sonar, LiDAR, infrared proximity).

4.2 Incremental encoders

An incremental encoder is a disc with evenly spaced slots fixed to a shaft. A light source on one side and a detector on the other produce a pulse every time a slot passes. Counting pulses tells you how far the shaft has turned.

Resolution. If the disc has N slots, one pulse per slot gives N counts per turn, and each count represents $360^\circ / N$. With two detectors placed so that their signals are a quarter of a cycle apart (called channels A and B), we can count every edge of both channels, which gives four times as many counts. This is *quadrature decoding*.

Direction. When the shaft turns one way, A rises while B is low. Turned the other way, A rises while B is high. A tiny state machine compares the two channels at every edge and increments or decrements the count.

Table 4.1: A few sensors a student robot might carry and what they report.

Sensor	Measures	Typical use
Incremental encoder	shaft angle, speed	joint position, wheel odometry
Potentiometer	angle (absolute, coarse)	hobby servo feedback
Ultrasonic module	distance by echo time	obstacle detection, parking aid
Infrared reflective	near-range reflectance	line following, edge detection
LiDAR	range at many angles	mapping and localisation
IMU (accelerometer + gyro)	acceleration, turn rate	tilt, balance, dead reckoning
Force or load cell	force	detecting a grasp, weighing
Current sensor	motor current	estimating torque, detecting stall
Camera	light intensity image	recognition, tracking, inspection

Worked example: Encoder arithmetic

A disc has 500 slots and the controller decodes all four edges per slot.

- Counts per turn: $500 \times 4 = 2000$.
- Angle per count: $360^\circ / 2000 = 0.18^\circ$.
- The controller reads 3500 counts since power-up. Turned angle $= 3500 \times 0.18^\circ = 630^\circ = 1.75$ turns.
- If the shaft turns at 2 turns per second, the pulse rate is $2 \times 2000 = 4000$ counts per second. If the processor samples every 1 ms, it will see on average 4 counts per sample, which is quite coarse; estimating speed from so few counts is noisy, so we usually average over a longer window or time the interval between pulses.

Derivation: turning counts into angle and speed

Let the decoder give C counts per turn ($C = 4N$ for quadrature). One count is $360^\circ / C$ of rotation. If the count changes by Δn during a time Δt, the shaft has turned $\Delta n / C$ revolutions and its speed is

$$\omega = \frac{\Delta n}{C\,\Delta t} \text{ rev/s} = \frac{2\pi\,\Delta n}{C\,\Delta t} \text{ rad/s}.$$

Because Δn is a whole number, the smallest speed step you can see is $\Delta n = 1$, that is $1/(C\Delta t)$. For $C = 2000$: with $\Delta t = 1$ ms the speed resolution is $1/(2000 \times 0.001) = 0.5$ rev/s, which is coarse, and with $\Delta t = 50$ ms it is $1/(2000 \times 0.05) = 0.01$ rev/s, which is fine. This is the arithmetic behind the advice to use a longer window, and it shows the usual trade-off: a longer window gives a better number but an older one.

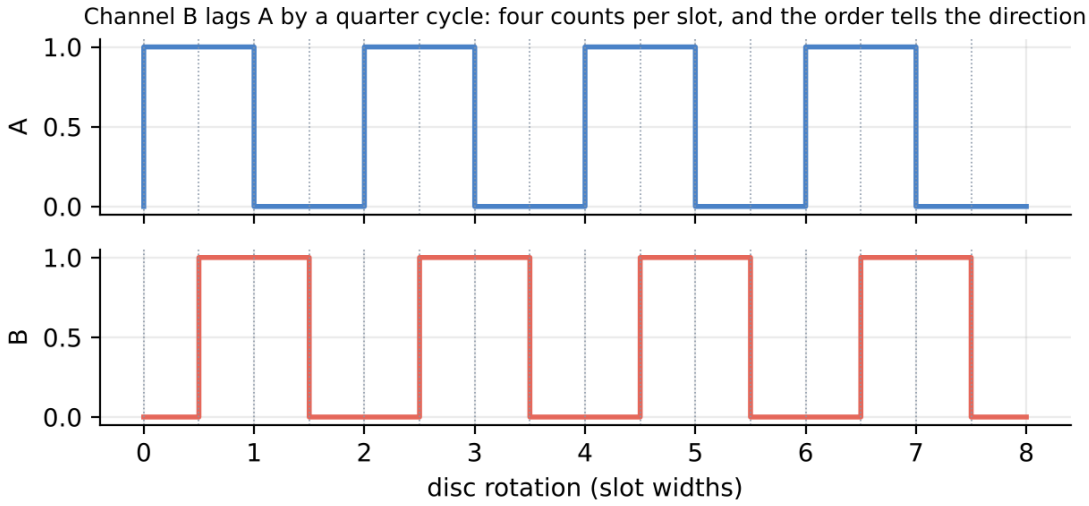


Figure 4.1: Quadrature signals. Whichever channel rises first tells you which way the shaft is turning; counting all four edges in each slot period multiplies the resolution by four.

The weakness of an incremental encoder is that it only knows change. At power-up it has no idea where the shaft is, so robots with such encoders must find a *home* position (touching a limit switch, for instance) before they start. Absolute encoders, which give a unique code for every angle, do not need this, but they cost more.

4.3 Ultrasonic distance sensing

An ultrasonic module sends a short burst of sound at about 40 kHz and listens for the echo. The time between the burst and the echo is t. The sound travels to the object and back, covering $2d$, so

$$d = \frac{v\,t}{2}, \tag{4.1}$$

where v is the speed of sound. In air $v \approx 331.3 + 0.606\,T_C$ metres per second, with T_C the temperature in degrees Celsius, which gives about 343 m/s at 20°C.

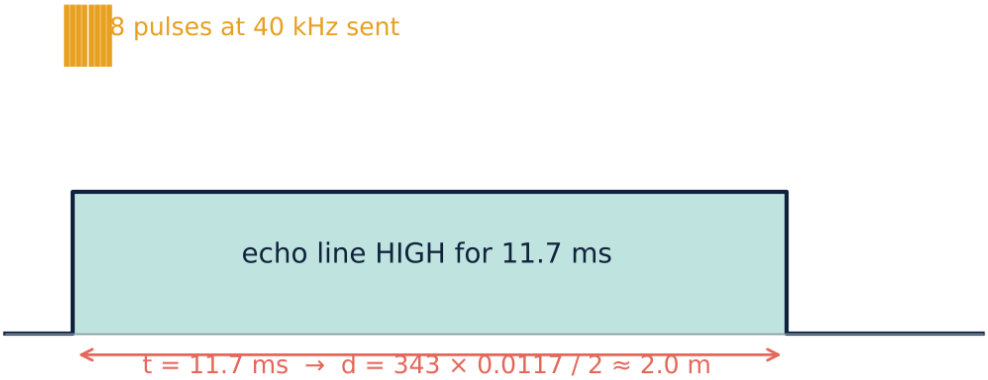


Figure 4.2: Timing of a typical ultrasonic module: after the trigger, the echo line stays high for a time proportional to distance.

Worked example: Reading a distance

The echo pin stays high for 11.7 ms. Then $d = 343 \times 0.0117/2 = 2.006$ m, about 2.0 m. If the room warmed to 30°C, $v = 331.3 + 18.2 = 349.5$ m/s and the same pulse would mean 2.04 m, so temperature causes a 2% error unless compensated.

Sonar has a few well-known troubles. The beam is a cone, not a line, so a narrow object off to the side can appear in front. Soft, angled or very small surfaces reflect little sound and may be missed. Two robots using the same frequency can confuse each other. And there is a minimum distance, of a couple of centimetres, below which the echo returns before the sensor has stopped transmitting. Knowing these limits is worth more than the formula.

4.4 LiDAR

A LiDAR fires a very short pulse of laser light and measures the time of flight. The idea is the same as sonar, but light is about a million times faster than sound, so the timing electronics must be about a million times finer: a target 3 m away returns the light after only 20 nanoseconds. Many low-cost LiDARs instead measure the position of the returning spot on a tiny sensor, a method called triangulation. A spinning mirror or a rotating head sweeps the beam around, so one revolution gives a ring of range readings, a *scan*. A scan of, say, 360 points can be converted to Cartesian coordinates by

$$x_k = r_k \cos \phi_k, \qquad y_k = r_k \sin \phi_k, \tag{4.2}$$

where r_k is the measured range at bearing ϕ_k. The cloud of points is the raw material for building maps and for spotting walls, doors and people.

Derivation: light timing and the polar-to-Cartesian step

Light travels at $c \approx 3 \times 10^8$ m/s and the pulse must go out and come back, so the delay for a target at distance d is $t = 2d/c$. For $d = 3$ m, $t = 6/(3 \times 10^8) = 2 \times 10^{-8}$ s $= 20$ ns, as stated. A timing error of just 1 ns corresponds to a distance error of $c\,\Delta t/2 = 0.15$ m, which tells you why such electronics are demanding.

For the conversion, the point sits at distance r_k along a direction making angle ϕ_k with the x axis, so the right-triangle definitions of cosine and sine give $x_k = r_k \cos \phi_k$ and $y_k = r_k \sin \phi_k$. Example: a return of $r = 2.0$ m at $\phi = 30°$ is the point $(2.0 \times 0.866,\ 2.0 \times 0.5) = (1.732,\ 1.000)$ m. Converting every one of the 360 readings in this way gives the cloud of points.

4.5 From voltage to number: the ADC

Many sensors output a smoothly varying voltage. A microcontroller works with integers, so an **analog-to-digital converter** (ADC) measures the voltage at chosen instants and rounds it to one of 2^n levels, where n is the number of bits. If the reference voltage is V_{ref}, the size of one step (the least significant bit, LSB) is

$$\text{LSB} = \frac{V_{\text{ref}}}{2^n}, \qquad \text{code} = \text{round}\left(\frac{V}{\text{LSB}}\right). \tag{4.3}$$

Rounding means the digital value is wrong by up to half a step. This unavoidable error is called **quantisation error**, and it is at most $\pm\text{LSB}/2$.

Derivation: why LSB $= V_{\text{ref}}/2^n$ and how big the rounding noise is

n bits can form 2^n different codes, from 0 to $2^n - 1$. The span 0 to V_{ref} is divided among these levels, so each step is $V_{\text{ref}}/2^n$. A reading is rounded to the nearest level, so the error e is anywhere between $-\text{LSB}/2$ and $+\text{LSB}/2$, and each value is about equally likely. For a uniform spread over a width $\Delta = \text{LSB}$ the mean square error is

$$\frac{1}{\Delta}\int_{-\Delta/2}^{\Delta/2} e^2\,de = \frac{1}{\Delta}\cdot\frac{2}{3}\left(\frac{\Delta}{2}\right)^3 = \frac{\Delta^2}{12},$$

so the root-mean-square quantisation noise is $\Delta/\sqrt{12} \approx 0.29\,\text{LSB}$. For the 10-bit example, that is $0.29 \times 3.223\ \text{mV} = 0.93\ \text{mV}$. Each extra bit halves Δ and so halves the noise.

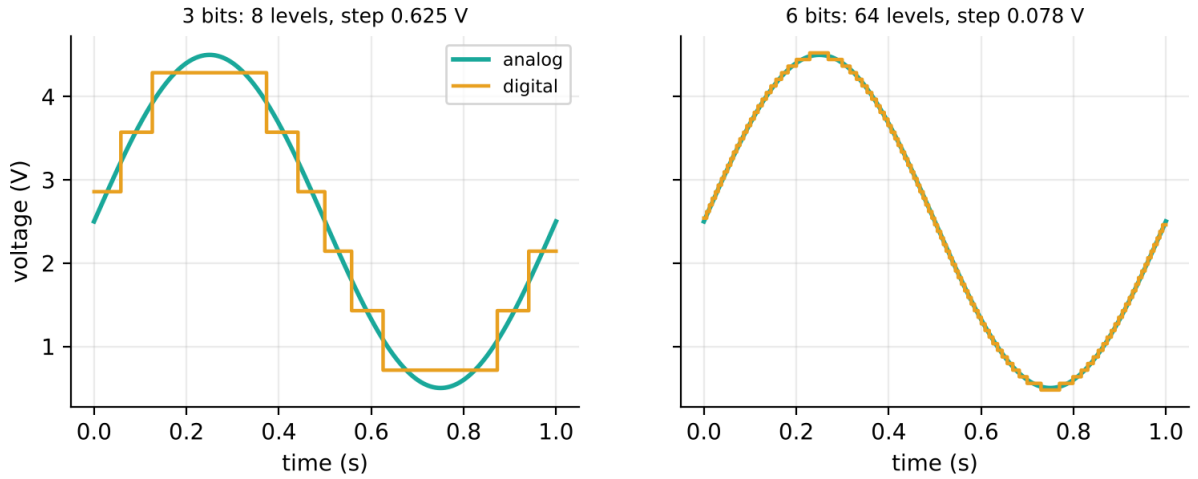


Figure 4.3: The same sine wave digitised with 3 bits and with 6 bits. More bits give a finer staircase and a smaller quantisation error.

Worked example: A 10-bit converter

A 10-bit ADC with $V_{\text{ref}} = 3.3$ V has $2^{10} = 1024$ levels. LSB $= 3.3/1024 = 3.223$ mV. An input of 1.65 V gives $1.65/0.003223 = 512$ so the code is 512 (binary 1000000000). The maximum error is ± 1.6 mV. Suppose the sensor changes by 10 mV per degree of temperature: then one LSB corresponds to 0.32°, so the resolution of the thermometer is limited by the converter to about a third of a degree no matter how good the sensor is.

Sampling. The converter also reads at discrete times. If you sample too slowly you can be fooled completely: a fast vibration may appear as a slow wave. The rule, from the sampling theorem of Nyquist and Shannon, is to sample at more than twice the highest frequency present in the signal,

$$f_s > 2f_{\max}. \tag{4.4}$$

In practice, engineers choose five to ten times $f_{\max}$ and put a simple low-pass filter in front of the converter to remove anything faster than needed. A well-known example is the wagon wheel

that seems to rotate backwards in a film; the camera's frame rate is too slow for the wheel's speed.

Derivation: why more than twice the highest frequency

Sample the wave $\sin(2\pi f t)$ at times $t = n/f_s$. The samples are $\sin(2\pi f n/f_s)$. Now consider a second wave of frequency $f - f_s$. Its samples are $\sin\big(2\pi(f - f_s)n/f_s\big) = \sin(2\pi f n/f_s - 2\pi n)$, and because subtracting a whole number of turns changes nothing, these samples are identical to the first set. After sampling, the two frequencies cannot be told apart, and the high one *masquerades* as a low one. This is called aliasing. The apparent frequency is $|f - k f_s|$ for the integer k that brings it nearest to zero.
Example: a 70 Hz vibration sampled at 100 Hz appears at $|70 - 100| = 30$ Hz. To avoid ambiguity, every real frequency must be below $f_s/2$, which is the same as $f_s > 2 f_{\max}$.

4.6 Noise and filtering

Real signals are never clean. Electrical interference, vibration and the sensor's own imperfection add random variation. We deal with it by **filtering**, which means combining several measurements so that random errors partly cancel while the genuine signal remains.

Moving average. Average the last M samples:

$$\bar{z}_k = \frac{1}{M} \sum_{i=0}^{M-1} z_{k-i}. \tag{4.5}$$

If the noise on each reading is independent with standard deviation σ, the average has standard deviation $\sigma/\sqrt{M}$. Averaging 16 readings cuts the noise to a quarter. The price is delay: the average responds late to a genuine change.

Derivation: why the noise falls as $1/\sqrt{M}$

Let each reading be $z = s + \varepsilon$, the true value plus an independent error ε with variance σ^2. The variance of a sum of independent quantities is the sum of the variances, so the sum of M readings has variance $M\sigma^2$. Dividing by M multiplies the variance by $1/M^2$:

$$\operatorname{Var}(\bar{z}) = \frac{M\sigma^2}{M^2} = \frac{\sigma^2}{M} \quad \Longrightarrow \quad \sigma_{\bar{z}} = \frac{\sigma}{\sqrt{M}}.$$

For $M = 16$ the standard deviation is $\sigma/4$, as claimed. Quadrupling the number of samples only halves the noise, so there are rapidly diminishing returns. The average of the last M samples is on average $(M-1)/2$ samples old, which is the delay mentioned.

Exponential filter. It needs no memory of past samples, only one stored number:

$$y_k = (1 - \alpha)\, y_{k-1} + \alpha\, z_k, \qquad 0 < \alpha \le 1. \tag{4.6}$$

A small α smooths strongly but reacts slowly; a large α reacts fast but passes more noise. The time constant is roughly $\tau \approx T_s/\alpha$ where T_s is the sample interval.

Derivation: the time constant of the exponential filter

Feed in a step, $z_k = 1$ for all $k \geq 0$, with $y_{-1} = 0$. Then $y_0 = \alpha$, $y_1 = (1-\alpha)\alpha + \alpha = 1 - (1-\alpha)^2$, and by repeating this, $y_k = 1 - (1-\alpha)^{k+1}$. The output closes the gap to the target by the fraction α at every step, so the remaining gap shrinks as $(1-\alpha)^k$. A continuous first-order system has a gap that shrinks as $e^{-t/\tau}$. Matching the two at $t = kT_s$:

$$(1-\alpha)^k = e^{-kT_s/\tau} \;\Rightarrow\; \tau = \frac{-T_s}{\ln(1-\alpha)} \approx \frac{T_s}{\alpha} \quad \text{for small } \alpha,$$

because $\ln(1-\alpha) \approx -\alpha$. Example: $\alpha = 0.2$ and $T_s = 10$ ms give exactly $\tau = -10/\ln 0.8 = 44.8$ ms, while the shortcut T_s/α gives 50 ms. After about 3τ the output has covered 95% of the step. Unrolling the formula also shows $y_k = \alpha \sum_{i \geq 0} (1-\alpha)^i z_{k-i}$: a weighted average in which older readings carry geometrically smaller weights.

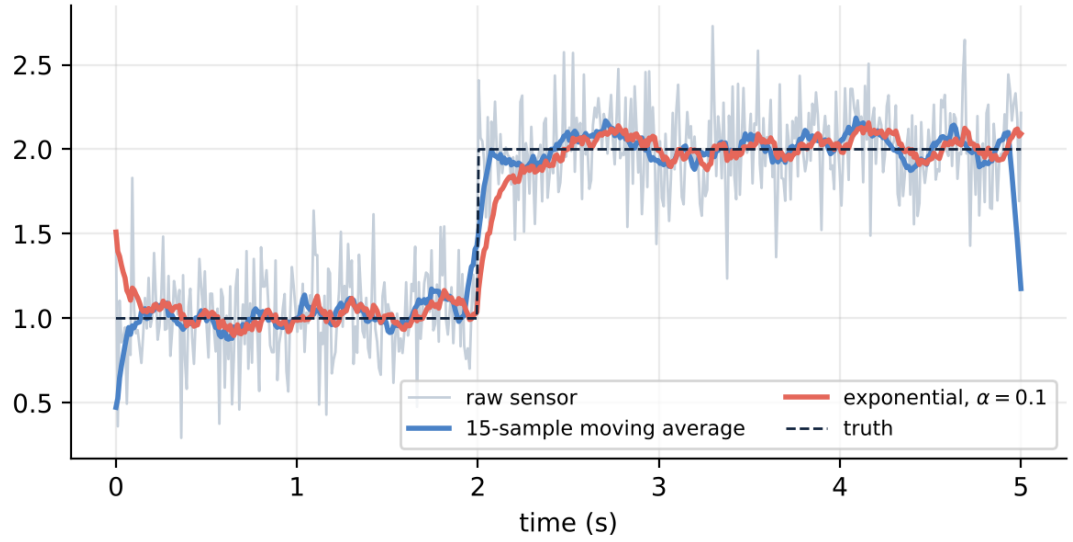


Figure 4.4: A noisy reading of a signal that steps from 1 to 2. Both filters reduce the wobble; notice how they trail the step.

A word from the classroom

Many students try to cure a noisy control loop by filtering harder. That often makes the robot oscillate, because the delay of the filter lands inside the feedback loop. Filter only as much as you must, and prefer to fix the source of noise (shielding, grounding, a better sensor mount) first.

4.7 A first look at the Kalman filter

Averaging treats every measurement as equally valuable. But often we also know something about how the system moves: a robot commanded to travel 0.1 m per step should be about 0.1 m further on after the step. The Kalman filter combines such a **prediction** with the noisy **measurement**, weighting each by how much we trust it.

Take one dimension, estimating a position x. We keep an estimate $\hat{x}$ and the variance P that says

how uncertain we are.

Predict: $\hat{x}^- = \hat{x} + u$ (move by the commanded amount u), $P^- = P + Q$ (Q is the process noise: the motion is never perfect, so uncertainty grows).

Update with measurement z whose noise variance is R:

$$K = \frac{P^-}{P^- + R}, \qquad \hat{x} = \hat{x}^- + K\,(z - \hat{x}^-), \qquad P = (1 - K)\,P^-. \tag{4.7}$$

The number K, the Kalman gain, lies between 0 and 1. If the measurement is very noisy (R large), K is small and we lean on the prediction. If the prediction is very uncertain (P^- large), K is near 1 and we trust the measurement. The estimate is always a weighted blend, and the uncertainty after the update is smaller than either source alone.

Derivation: where the Kalman gain comes from

Suppose the prediction $\hat{x}^-$ has error variance P^- and the measurement is $z = x + v$ with noise variance R, independent of the prediction error. We look for the best blend $\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$, and "best" means smallest error variance. The error after blending is

$$x - \hat{x} = (1 - K)(x - \hat{x}^-) - Kv,$$

so its variance is $P(K) = (1 - K)^2 P^- + K^2 R$. This is a parabola in K. Set the slope to zero:

$$\frac{dP}{dK} = -2(1 - K)P^- + 2KR = 0 \;\Rightarrow\; K = \frac{P^-}{P^- + R}.$$

Put this back: $1 - K = R/(P^- + R)$, so

$$P = \frac{R^2 P^- + P^{-2} R}{(P^- + R)^2} = \frac{P^- R\,(R + P^-)}{(P^- + R)^2} = \frac{P^- R}{P^- + R} = (1 - K)P^-.$$

With the numbers of the example, $P^- R/(P^- + R) = 0.05 \times 0.09/0.14 = 0.0321$, the same as $(1 - 0.357)(0.05)$. The formula also shows why the result is always better than either source: $P < P^-$ and $P < R$.

Worked example: One Kalman step

Estimate $\hat{x} = 2.00$ m with $P = 0.04$. The robot is commanded to move $u = 0.10$ m, process noise $Q = 0.01$. A sonar measures $z = 2.20$ m with $R = 0.09$.
Predict: $\hat{x}^- = 2.10$, $P^- = 0.04 + 0.01 = 0.05$.
Gain: $K = 0.05/(0.05 + 0.09) = 0.357$.
Update: $\hat{x} = 2.10 + 0.357(2.20 - 2.10) = 2.136$ m, $P = (1 - 0.357)(0.05) = 0.0321$.
The new estimate sits between the prediction and the measurement, closer to the prediction because that was the more certain of the two, and the variance fell from 0.05 to 0.032.

4.8 Sensor fusion in one sentence

No single sensor is good at everything. Wheel encoders are smooth but drift, a gyroscope measures turning but drifts too, a camera or LiDAR gives absolute information but arrives

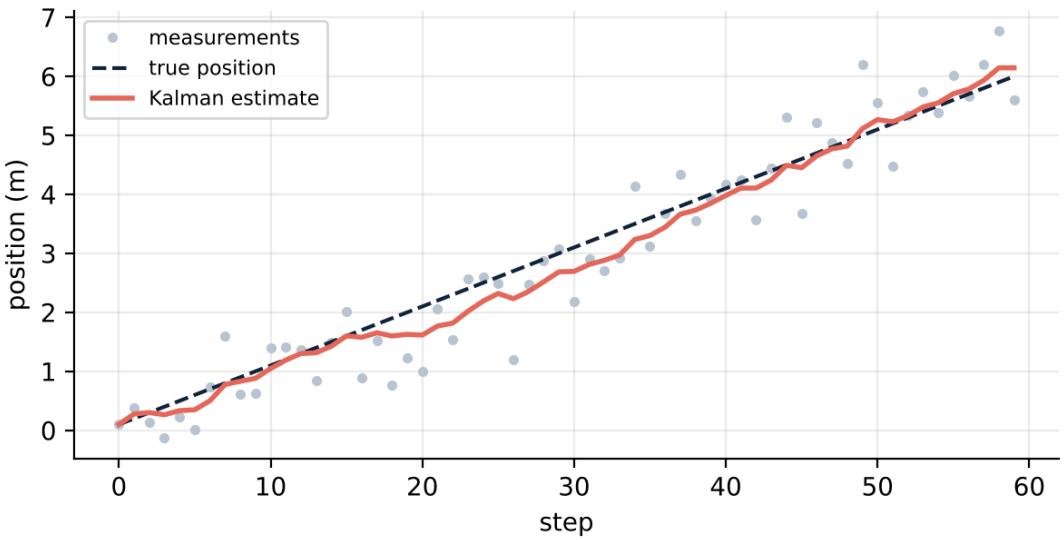


Figure 4.5: A robot moving at constant speed, measured with noisy sonar (grey dots). The Kalman estimate (red) follows the truth much more closely than the raw data.

slowly and noisily. Fusing them, usually with a Kalman-type filter, produces an estimate better than any one could give. The ideas of this chapter, resolution, noise and weighting by confidence, are what sensor fusion is built from.

4.9 Chapter summary

- Sensors are described by range, resolution and noise; proprioceptive and exteroceptive, passive and active are the standard groupings.
- An encoder with N slots and quadrature decoding gives $4N$ counts per turn.
- Ultrasonic distance is $d = vt/2$; temperature changes v.
- An n-bit ADC has LSB $= V_{\text{ref}}/2^n$ and error $\leq$ LSB$/2$; sample faster than $2f_{\max}$.
- Filtering trades noise for delay; a Kalman filter blends prediction and measurement using their uncertainties.

Chapter 5

Making Machines Obey: Feedback Control

5.1 Open loop versus closed loop

Imagine you want a wheeled robot to travel exactly one metre. The simplest idea is to run both motors at a fixed power for a fixed time that you worked out on a good day. That is **open-loop** control: the command is chosen in advance and nobody checks the result. It works while nothing changes. Then the battery sags, the floor is carpeted, or the robot is carrying an extra load, and the distance comes out wrong.

Closed-loop control adds a measurement. The robot reads its encoders, compares the distance travelled with the target, and corrects the motor command according to the difference. That difference is the **error**,

$$e(t) = r(t) - y(t), \tag{5.1}$$

where r is the reference (what we want) and y the measured output (what we have). Feedback is the single most important idea in this book, because it makes a system tolerant of everything we did not model.

5.2 The proportional controller

The most natural rule is: push in proportion to how wrong you are. The controller output is

$$u(t) = K_p\, e(t). \tag{5.2}$$

A large error produces a strong push; a small error produces a gentle one; zero error produces none. K_p is the **proportional gain**.

5.2.1 A model to calculate with

To see exactly what a P controller does, let us give the plant (the thing being controlled) a simple model, a first-order lag such as a motor whose speed responds smoothly to voltage:

$$\tau\, \dot{y} + y = K\, u. \tag{5.3}$$

Here K is the plant's steady gain (output per unit input) and τ its time constant. Connect the P controller, $u = K_p(r - y)$:

$$\tau\dot{y} + y = KK_p(r - y) \implies \tau\dot{y} + (1 + KK_p)\, y = KK_p\, r.$$

This is again a first-order equation. Its steady state (where $\dot{y} = 0$) is

$$y_\infty = \frac{KK_p}{1 + KK_p}\, r, \qquad e_\infty = \frac{1}{1 + KK_p}\, r. \tag{5.4}$$

The idea to keep

A pure proportional controller on a plant with finite gain always leaves a steady-state error. Raising K_p shrinks the error but never removes it, and a very high gain invites oscillation.

Worked example: Steady-state error of a P controller

Plant gain $K = 1$, $K_p = 4$, set-point $r = 1$. Then $y_\infty = \dfrac{4}{1+4} = 0.8$ and the error is 0.2. To bring the final value to 95% we need $\dfrac{K_p}{1+K_p} = 0.95 \Rightarrow K_p = 19$, a gain at which real systems usually become unstable. This is the reason P alone is rarely enough.

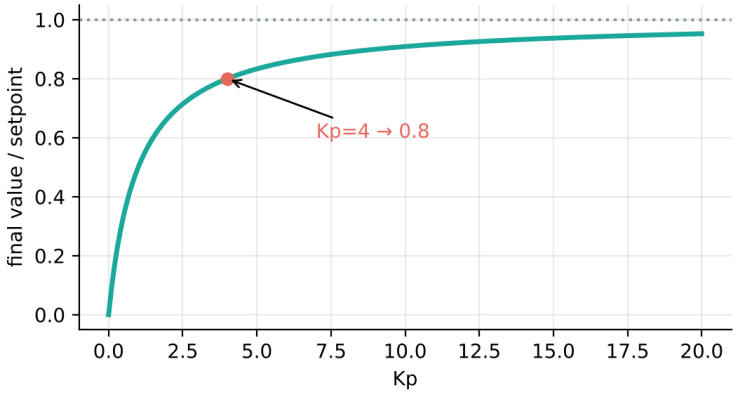


Figure 5.1: Final value of a P-controlled plant with unit gain as K_p grows. The curve approaches 1 but never reaches it.

The time constant of the closed loop is $\tau_{cl} = \tau/(1 + KK_p)$, so a higher gain also makes the response faster. Speed and accuracy improve together, which is why we are tempted to keep increasing the gain.

Derivation: solving the P-controlled first-order loop

Start from $\tau\dot{y} + (1 + KK_p)\, y = KK_p\, r$ and divide by $(1 + KK_p)$ so that the coefficient of y becomes 1:

$$\underbrace{\frac{\tau}{1 + KK_p}}_{\tau_{cl}}\dot{y} + y = \underbrace{\frac{KK_p}{1 + KK_p}}_{y_\infty/r}\, r.$$

This is the original first-order equation with a new time constant $\tau_{cl} = \tau/(1 + KK_p)$ and a

new gain y_∞/r. For a constant set-point and $y(0) = 0$ its solution is $y(t) = y_\infty\big(1 - e^{-t/\tau_{cl}}\big)$, which you can verify by substituting back: $\tau_{cl}\dot{y} = y_\infty e^{-t/\tau_{cl}}$ and $y = y_\infty - y_\infty e^{-t/\tau_{cl}}$, whose sum is y_∞. At steady state, $\dot{y} = 0$ and the formulas of the text follow at once. The error is $e_\infty = r - y_\infty = r\big(1 - \frac{KK_p}{1+KK_p}\big) = \frac{r}{1+KK_p}$.
Numbers. A motor with $K = 1$ and $\tau = 0.5$ s under $K_p = 4$: $\tau_{cl} = 0.5/5 = 0.1$ s, so the speed reaches 95% of its final value after about $3\tau_{cl} = 0.3$ s (against 1.5 s with no feedback), but that final value is only $0.8\,r$.

5.3 Adding the integral term

To remove the leftover error we let the controller remember. The **integral** of the error grows as long as any error persists, so the controller output keeps increasing until the error is exactly zero:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau. \tag{5.5}$$

At steady state the integral stops changing only if $e = 0$. Hence a PI controller removes the constant error of a P controller. The cost is a slower, more oscillatory response, because the integral continues to push even after the output has reached the target, until enough opposite error has accumulated to cancel it.

Derivation: why the integral removes the offset, and how to choose K_p and K_i

The argument. Suppose the loop settles to a steady state with a constant error $e_\infty \neq 0$. Then $\int e\, d\tau$ grows by e_∞ every second, without limit, so u cannot be constant, which contradicts a steady state. The only way out is $e_\infty = 0$.
The calculation. Take the same plant $\tau\dot{y} + y = Ku$ and a PI controller. Differentiating $u = K_p e + K_i \int e$ gives $\dot{u} = K_p\dot{e} + K_i e$. In the Laplace picture (replace d/dt by s, the controller is $K_p + K_i/s$ and the plant is $K/(\tau s + 1)$), the closed-loop relation between output and reference is

$$\frac{Y}{R} = \frac{K(K_p s + K_i)}{\tau s^2 + (1 + KK_p)\, s + KK_i}.$$

Put $s = 0$ for a constant reference: the ratio is $KK_i/KK_i = 1$, so the final value equals the reference exactly.
Choosing the gains. Divide the denominator by τ and compare with $s^2 + 2\zeta\omega_n s + \omega_n^2$:

$$\omega_n^2 = \frac{KK_i}{\tau}, \qquad 2\zeta\omega_n = \frac{1 + KK_p}{\tau} \quad\Longrightarrow\quad K_i = \frac{\tau\omega_n^2}{K}, \quad K_p = \frac{2\zeta\omega_n\tau - 1}{K}.$$

Example: $K = 1$, $\tau = 0.5$ s, and we want $\omega_n = 6$ rad/s with $\zeta = 0.7$. Then $K_i = 0.5 \times 36 = 18$ and $K_p = 2(0.7)(6)(0.5) - 1 = 3.2$. Check: the denominator is $0.5s^2 + 4.2s + 18$, that is $s^2 + 8.4s + 36$, so $\omega_n = \sqrt{36} = 6$ and $2\zeta\omega_n = 8.4$ gives $\zeta = 0.7$. (The numerator zero at $s = -K_i/K_p$ makes the real overshoot a little larger than the 4.6% the second-order formula predicts for $\zeta = 0.7$; this is why the response in the figure overshoots slightly.)

Integral windup. If the actuator is saturated (the motor is already at full voltage) and an error remains, the integral keeps growing without any effect. When the error finally changes sign, the

huge stored value takes a long time to unwind and causes a big overshoot. The remedy is to clamp the integral or stop integrating while the actuator is saturated.

5.4 Adding the derivative term

The **derivative** of the error measures how fast the error is changing, which lets the controller anticipate. If the error is falling quickly the controller can ease off before overshooting:

$$u(t) = K_p\, e(t) + K_i \int_0^t e\, d\tau + K_d \frac{de}{dt}. \tag{5.6}$$

The derivative term behaves like damping. It acts against rapid change and therefore reduces overshoot. Its weakness is that differentiation amplifies noise: a jittery sensor makes a violently jumping derivative. In practice it is applied to a filtered measurement.

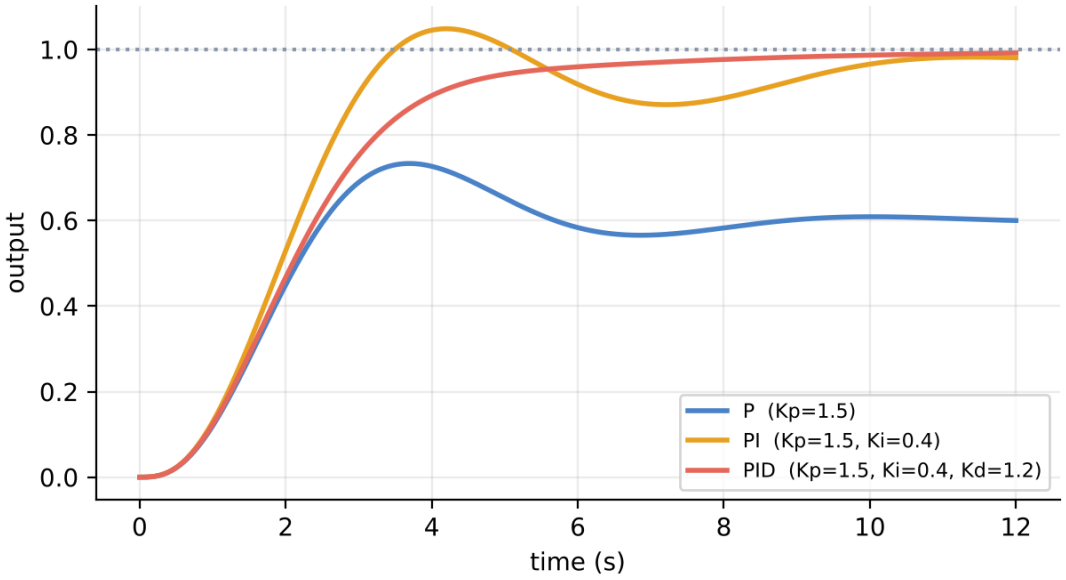


Figure 5.2: Responses of a three-stage plant to a unit step under P, PI and PID control. P settles low; PI reaches the target with an overshoot; PID reaches it smoothly.

In the simulation above the P controller settles at 0.6, which matches $K_p/(1+K_p) = 1.5/2.5$ for a plant of unit DC gain. The integral term removes this offset at the price of a small overshoot; the derivative term brings the overshoot down.

5.5 Describing a response

When we tune a controller we need words for what we see. Take the response to a step command.

- **Rise time**: time to go from 10% to 90% of the final value.
- **Overshoot**: how far the peak exceeds the final value, as a percentage of it.
- **Settling time**: time after which the output stays within, say, 2% of the final value.
- **Steady-state error**: the remaining difference after everything has settled.

These four numbers usually pull against each other. A shorter rise time tends to give more overshoot; a lower overshoot tends to lengthen settling.

5.6 The second-order picture

Many mechanical systems, like a joint with inertia and some friction under position control, behave like a second-order system,

$$\ddot{y} + 2\zeta\omega_n \dot{y} + \omega_n^2 y = \omega_n^2 r. \tag{5.7}$$

Two numbers describe it: the natural frequency ω_n (how fast it wants to swing) and the damping ratio ζ (how strongly the motion is damped).

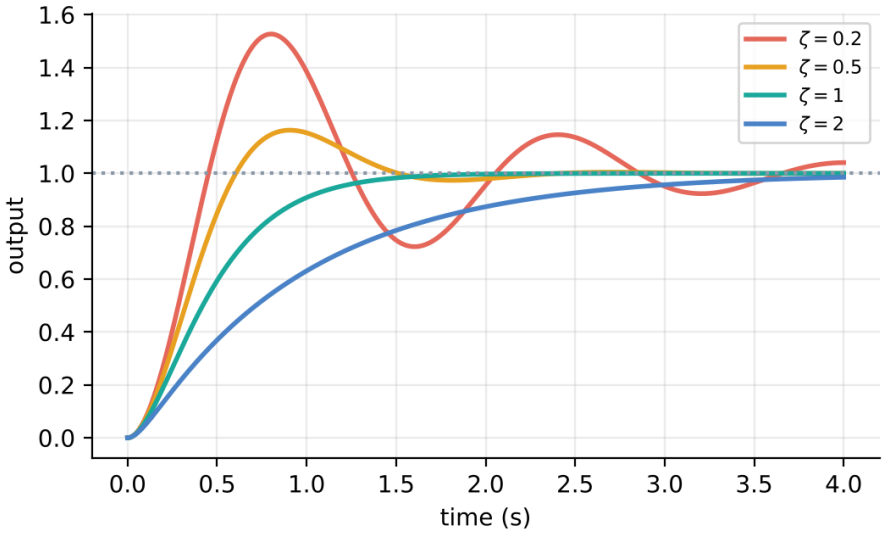


Figure 5.3: Step response for $\omega_n = 4$ rad/s and four damping ratios. Light damping rings; critical damping ($\zeta = 1$) is the fastest without overshoot; heavy damping is slow.

For $\zeta < 1$ the percentage overshoot depends only on ζ:

$$\%\,OS = 100 \exp\left(-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}\right), \tag{5.8}$$

and the 2% settling time is approximately $T_s \approx 4/(\zeta\omega_n)$.

Derivation: the overshoot and settling formulas

For $\zeta < 1$ the unit-step response of $\ddot{y} + 2\zeta\omega_n\dot{y} + \omega_n^2 y = \omega_n^2$ is

$$y(t) = 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1-\zeta^2}} \sin(\omega_d t + \varphi), \qquad \omega_d = \omega_n\sqrt{1-\zeta^2}, \quad \varphi = \cos^{-1}\zeta.$$

Differentiating, the terms combine to $\dot{y} = \dfrac{\omega_n}{\sqrt{1-\zeta^2}}\, e^{-\zeta\omega_n t} \sin(\omega_d t)$. The output peaks when $\dot{y} = 0$, that is when $\sin\omega_d t = 0$; the first such time after 0 is $t_p = \pi/\omega_d$. At that moment $e^{-\zeta\omega_n t_p} = \exp\!\left(-\zeta\omega_n\pi/\omega_d\right) = \exp\!\left(-\pi\zeta/\sqrt{1-\zeta^2}\right)$ and the sine term in y equals

$-\sqrt{1-\zeta^2}$ (its value at $\omega_d t_p = \pi$ with the phase φ), so

$$y(t_p) = 1 + \exp\left(-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}\right),$$

and the overshoot above 1 is the exponential, which is the formula. For the settling time, the oscillation sits inside the envelope $e^{-\zeta\omega_n t}/\sqrt{1-\zeta^2}$. Requiring the envelope to fall below 2% gives approximately $e^{-\zeta\omega_n t} = 0.02$, so $\zeta\omega_n t = \ln 50 = 3.91 \approx 4$ and $T_s \approx 4/(\zeta\omega_n)$.
Check with the worked example. $\zeta = 0.591$: $\pi(0.591)/\sqrt{1-0.349} = 1.857/0.807 = 2.30$, and $e^{-2.30} = 0.100$, a 10% overshoot. With $\omega_n = 6.77$ the damped frequency is $\omega_d = 6.77(0.807) = 5.46$ rad/s and the peak occurs at $t_p = \pi/5.46 = 0.575$ s.

Worked example: Reading a specification

A positioning axis must have no more than 10% overshoot and settle within 1 s.
Overshoot: $0.10 = \exp\left(-\pi\zeta/\sqrt{1-\zeta^2}\right)$ gives $\pi\zeta/\sqrt{1-\zeta^2} = 2.303$. Squaring, $\pi^2\zeta^2 = 5.30(1-\zeta^2)$, so $\zeta^2(9.87+5.30) = 5.30$, $\zeta = 0.591$.
Settling: $T_s = 4/(\zeta\omega_n) \le 1 \Rightarrow \omega_n \ge 4/0.591 = 6.77$ rad/s.
So the controller must be tuned to give damping of at least about 0.6 and a natural frequency above about 6.8 rad/s.

5.7 Stability in plain language

A system is stable if, after a disturbance, it returns to rest. For linear systems this depends on the poles, the roots of the characteristic equation. For the second-order system the poles are

$$s = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}.$$

Derivation: the poles from the quadratic formula

The poles are the roots of the characteristic equation $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$. The quadratic formula gives

$$s = \frac{-2\zeta\omega_n \pm \sqrt{4\zeta^2\omega_n^2 - 4\omega_n^2}}{2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}.$$

When $\zeta < 1$ the number under the root is negative, so the poles are a complex pair $-\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$ and the response rings at ω_d. Example: $\zeta = 0.6$ and $\omega_n = 4$ rad/s give $-2.4 \pm j\,4\sqrt{0.64} = -2.4 \pm j3.2$. The negative real part -2.4 says the ringing dies as $e^{-2.4t}$, and 3.2 rad/s is the ringing frequency. For $\zeta = 1$ both poles meet at $-\omega_n$ (critical damping), and for $\zeta > 1$ both are real and negative (no ringing, slower response).

If both poles have negative real parts the response dies out; if any pole has a positive real part it grows without bound. Raising the gain of a feedback loop that has delay moves the poles to the right, and beyond a certain gain the loop starts to oscillate with growing amplitude. This is why

gains cannot simply be pushed up. A robot arm that starts to buzz loudly when you increase K_p is telling you it has reached the stability limit.

5.8 Tuning in practice

A well-tried procedure for a first guess is the following.

1. Set $K_i = K_d = 0$. Increase K_p until the output responds quickly and shows a small overshoot or slight ringing.
2. Add K_d to calm the ringing. Raise it until the oscillation is gone but the system does not become sluggish or noisy.
3. Add a small K_i to remove the final offset. Increase it until the offset disappears in an acceptable time without a large overshoot.
4. Test with disturbances such as a hand push on the arm and a change in load.

There is also the classic Ziegler–Nichols rule: raise K_p alone until the output oscillates steadily at gain K_u with period T_u, then take $K_p = 0.6K_u$, $K_i = 2K_p/T_u$ and $K_d = K_pT_u/8$. It gives a lively response that usually needs gentle softening, but it is a respectable starting point.

Worked example: Ziegler–Nichols numbers

A joint loop with K_p alone starts to oscillate steadily at $K_u = 10$, and the oscillation period measured on the screen is $T_u = 0.4$ s. Then

$$K_p = 0.6(10) = 6, \qquad K_i = \frac{2K_p}{T_u} = \frac{12}{0.4} = 30, \qquad K_d = \frac{K_pT_u}{8} = \frac{6(0.4)}{8} = 0.3.$$

These are starting values; because the rule aims for a lively response, I usually reduce K_p to about two-thirds and then re-tune K_d by eye.

5.9 A discrete-time controller in code

A microcontroller does not integrate or differentiate continuously. It runs the loop every T_s seconds and approximates:

$$I_k = I_{k-1} + e_kT_s, \qquad D_k = \frac{e_k - e_{k-1}}{T_s}.$$

```
def pid_step(r, y, state, kp, ki, kd, dt, umin=-12, umax=12):
    e = r - y
    state['I'] += e * dt
    d = (e - state['e_prev']) / dt
    u = kp * e + ki * state['I'] + kd * d
    if u > umax: # actuator saturated: stop the integral growing
        u = umax; state['I'] -= e * dt
    elif u < umin:
        u = umin; state['I'] -= e * dt
```

```
    state['e_prev'] = e
    return u
```

Three practical details deserve attention: use a fixed, known T_s (a timer interrupt, not a loop that takes whatever time it takes), clamp the integral to prevent windup, and low-pass filter the derivative.

Derivation: from the formulas to the code, with a hand trace

The integral is the area under the error curve. Over one interval of length T_s the area is approximately the rectangle $e_k T_s$, so adding one rectangle per step gives $I_k = I_{k-1} + e_k T_s$. The derivative is the slope, and the slope between two samples is the rise over the run, $(e_k - e_{k-1})/T_s$. Smaller T_s makes both approximations better.
Trace for $k_p = 2$, $k_i = 1$, $k_d = 0.1$, $T_s = 0.1$ s, reference $r = 1$, starting from rest with $I = 0$ and $e_{\text{prev}} = 0$.
Step 1: $y = 0$, so $e = 1$. $I = 0 + 1(0.1) = 0.1$. $d = (1 - 0)/0.1 = 10$. Then $u = 2(1) + 1(0.1) + 0.1(10) = 3.1$. The large derivative term is a "kick" caused by the sudden jump of the error from 0; this is one reason to differentiate the measurement instead of the error.
Step 2: the output has risen to $y = 0.2$, so $e = 0.8$, $I = 0.1 + 0.08 = 0.18$, $d = (0.8 - 1)/0.1 = -2$, and $u = 2(0.8) + 0.18 + 0.1(-2) = 1.58$. The error fell, so the derivative term turned negative and eased off the push, as intended.

5.10 Cascade control in a servo

Real motor drives nest several loops: an inner *current* loop that is fast (kilohertz) and keeps torque under control, around it a *velocity* loop that is slower, and outermost a *position* loop. Each loop treats the one inside as an almost ideal actuator, which is only legitimate if the inner loop is several times faster than the outer. This ordering explains why servo tuning always starts from the inside.

5.11 Chapter summary

- Feedback acts on the error $e = r - y$.
- P control leaves a steady error $1/(1 + KK_p)$; I removes it; D adds damping.
- Step-response measures: rise time, overshoot, settling time, steady-state error.
- For second-order systems, overshoot depends on ζ and settling time on $\zeta\omega_n$.
- Watch for windup and noise; tune from P to D to I, test with disturbances.

Chapter 6

Actuators and Drives

An actuator turns energy, usually electrical, hydraulic or pneumatic, into motion. Choosing it is a matter of asking how much force or torque, how fast, how precisely, and at what cost in weight, noise and complexity. This chapter builds the models of the three families you must know and the gearbox that so often sits between a motor and its load.

6.1 Comparing the drive families

Table 6.1: Rough comparison of drive types for robots.

	Electric	Hydraulic	Pneumatic
Force/size	moderate	very high	low to moderate
Control	excellent	good, needs valves	difficult (air compresses)
Cleanliness	clean	leaks possible	clean, noisy exhaust
Typical use	most robots	heavy lifting arms, excavators	fast grippers, soft robots

6.2 The permanent-magnet DC motor

The DC motor is the workhorse of small robots. Two relationships describe it completely at the level we need.

Torque from current. The torque is proportional to the armature current i:

$$\tau = K_t \, i. \tag{6.1}$$

Back-EMF from speed. A spinning motor is also a generator. It produces a voltage opposing the supply that is proportional to the angular speed ω:

$$e_b = K_e \, \omega. \tag{6.2}$$

In SI units $K_t = K_e$, which I find reassuring: energy bookkeeping must balance. The electrical

loop with supply voltage V and armature resistance R is

$$V = iR + K_e\omega \implies i = \frac{V - K_e\omega}{R}. \tag{6.3}$$

Substituting into the torque relation gives the **torque–speed line**

$$\tau = \frac{K_t}{R}\left(V - K_e\,\omega\right). \tag{6.4}$$

Two points define it. At standstill ($\omega = 0$) we get the **stall torque** $\tau_s = K_t V/R$, with a stall current V/R which can be large enough to overheat the winding. At zero torque we get the **no-load speed** $\omega_0 = V/K_e$.

Worked example: A small gearmotor

A motor has $K_t = K_e = 0.05$ N m/A (V s/rad), $R = 1.2\ \Omega$, supply $V = 12$ V.

- Stall torque: $0.05 \times 12/1.2 = 0.50$ N m; stall current $12/1.2 = 10$ A.
- No-load speed: $12/0.05 = 240$ rad/s $= 2292$ rpm.
- Mechanical power $P = \tau\omega$ is maximum halfway along the line, at $\omega_0/2$ and $\tau_s/2$: $P_{\max} = \frac{1}{4}\tau_s\omega_0 = \frac{1}{4}(0.5)(240) = 30$ W.
- At a load of 0.2 N m: $i = 0.2/0.05 = 4$ A, $\omega = (12 - 1.2 \times 4)/0.05 = 144$ rad/s $= 1375$ rpm.

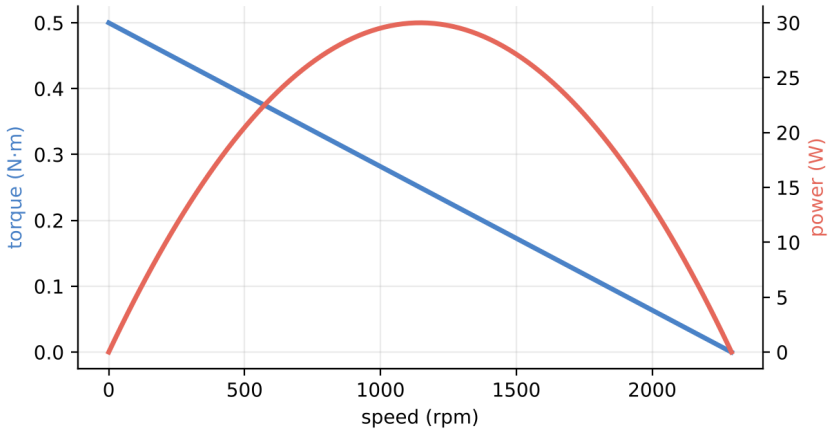


Figure 6.1: Torque (blue) and mechanical power (red) against speed for the motor of the worked example. Power peaks at half the no-load speed.

Because power peaks at mid-speed, motors are most useful there. A frequently misunderstood point is efficiency: at stall the motor produces zero mechanical power but burns $V^2/R = 120$ W as heat. Holding a heavy load with a DC motor against the stall torque is thus very wasteful, and gearboxes with high ratios that do not back-drive, or brakes, are used for holding.

Derivation: power, efficiency and the best operating point of a DC motor

The mechanical power is torque times speed. Insert the torque–speed line:

$$P = \tau\omega = \frac{K_t}{R}\left(V\omega - K_e\omega^2\right).$$

This is a downward parabola in ω. Its slope is $dP/d\omega = \dfrac{K_t}{R}(V - 2K_e\omega)$, which is zero at $\omega = V/(2K_e) = \omega_0/2$. At that speed $\tau = \dfrac{K_t}{R}\left(V - \frac{V}{2}\right) = \frac{1}{2}\tau_s$, so $P_{\max} = \frac{1}{4}\tau_s\omega_0$, as stated in the example.

Why $K_t = K_e$. The electrical power drawn is $Vi = i^2R + e_b i$: the first term is heat in the winding and the second is what is converted into motion. Mechanical power is $\tau\omega = K_t i\omega$ and the converted electrical power is $e_b i = K_e\omega i$. If the conversion has no loss, these must be equal for every i and ω, so $K_t = K_e$.

Efficiency at a working point. In the example at a load of 0.2 N m the current is 4 A and the speed 144 rad/s. Mechanical power $= 0.2 \times 144 = 28.8$ W. Electrical power $= 12 \times 4 = 48$ W. Efficiency $= 28.8/48 = 60\%$. The missing 19.2 W is $i^2R = 16 \times 1.2$, the heat in the winding. At stall the efficiency is 0 and at the no-load speed it is also 0 (no torque), so the efficiency peaks between them, close to the no-load end.

Why the duty cycle sets the voltage. A supply switched between V and 0 with period T and on-time DT has average $\frac{1}{T}\int_0^T v\,dt = \frac{1}{T}(V \cdot DT) = DV$. To obtain 7.5 V from 12 V the duty cycle is $D = 7.5/12 = 62.5\%$.

Speed control with PWM. A microcontroller cannot output 7.5 V on a 12 V supply, but it can switch the supply on and off thousands of times per second. If the fraction of time spent on (the duty cycle) is D, the motor, which has inductance and inertia, responds to the average $V_{\text{avg}} = DV$. A bridge of four transistors (an H-bridge) lets us reverse the polarity for the opposite direction.

6.3 Stepper motors

A stepper motor turns in fixed angular increments. A common type has 200 steps per revolution, a **step angle** of 1.8°. The controller energises its coils in a repeating sequence, and each change of pattern moves the shaft one step.

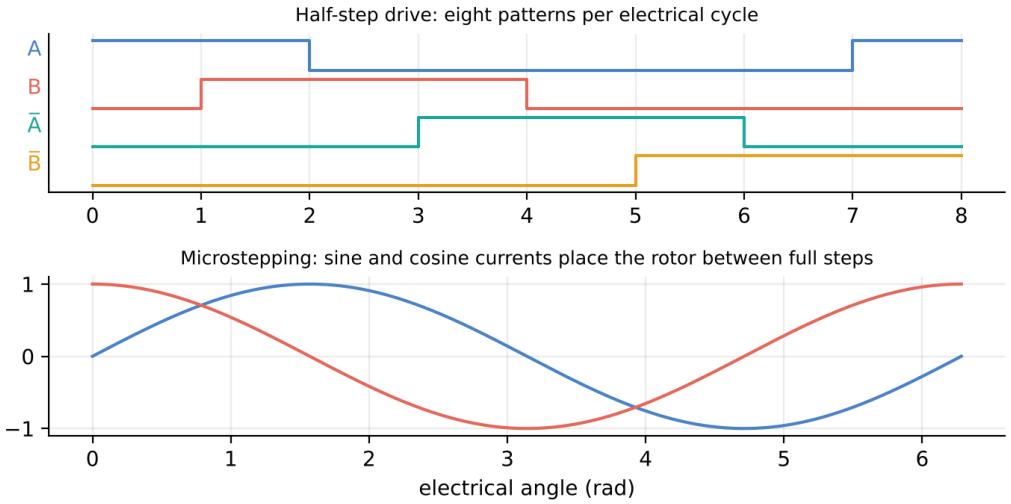


Figure 6.2: Top: half-step drive patterns for a two-phase stepper. Bottom: microstepping replaces on/off currents with sine and cosine currents.

Open-loop position. Provided the motor never misses a step, position is simply the number of steps counted:

$$\theta = n\,\theta_{\text{step}}, \qquad \omega = \frac{2\pi f\,\theta_{\text{step}}}{360^\circ} \ \text{(rad/s for step rate } f\text{)}. \tag{6.5}$$

Microstepping by a factor m divides the step angle by m, though the torque available to resist a position error drops with finer divisions, so the real accuracy is less than the nominal resolution.

Worked example: Stepper speed and resolution

A 1.8° stepper is driven with $1/16$ microstepping at 6400 microsteps per second. Microstep angle $= 1.8/16 = 0.1125^\circ$. Steps per revolution $= 200 \times 16 = 3200$. Speed $= 6400/3200 = 2$ rev/s $= 120$ rpm. To turn a lead screw of pitch 2 mm (the nut moves 2 mm per turn), linear resolution is $2\ \text{mm}/3200 = 0.625\ \mu$m per microstep and speed is 4 mm/s.

Derivation: the stepper speed formula

If the controller sends f step pulses per second and each moves the shaft θ_{step} degrees, the shaft turns $f\,\theta_{\text{step}}$ degrees per second. To change degrees to radians multiply by $2\pi/360$, so $\omega = 2\pi f\theta_{\text{step}}/360^\circ$. In the example, $f = 6400$ and $\theta_{\text{step}} = 0.1125^\circ$ give 720°/s, that is 2 rev/s. The screw moves one pitch (2 mm) per revolution, so the linear speed is $2 \times 2 = 4$ mm/s, and one microstep moves it $2/3200 = 0.000625$ mm.

Steppers are loved for simplicity (no sensor needed) and hated for two behaviours. They lose torque as speed rises, because the coil current cannot build up fast enough, and if the load exceeds the available torque they skip steps silently, so the controller loses its position without knowing it. Where that cannot be accepted, an encoder is added to close the loop, or a servo motor is used instead.

6.4 Servo motors

A hobby servo packages a small DC motor, a gearbox, a potentiometer for position and a little control circuit. You send a pulse every 20 ms whose width, about 1 to 2 ms, encodes the target angle. Industrial servos use the same idea on a larger scale: a motor, an encoder and a drive with nested current, velocity and position loops. The robot's controller supplies targets and the drive does the rest. Servo drives offer good torque over a wide speed range and precise position, which is why they dominate arms.

6.5 Gears and why robots need them

Motors like to spin fast with small torque, and robot joints need slow motion with large torque. A gear train bridges the two. For a gear ratio N (the motor turns N times for one turn of the output):

$$\omega_{\text{out}} = \frac{\omega_{\text{in}}}{N}, \qquad \tau_{\text{out}} = N\,\eta\,\tau_{\text{in}}, \tag{6.6}$$

where η is the efficiency, typically 0.7 to 0.95 per stage. Power is conserved apart from losses: $\tau_{\text{out}}\omega_{\text{out}} = \eta\,\tau_{\text{in}}\omega_{\text{in}}$.

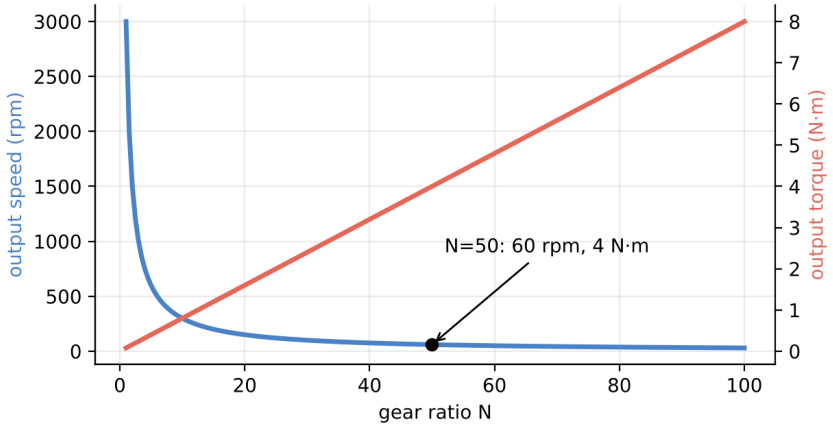


Figure 6.3: A 3000 rpm, 0.1 N m motor behind a gearbox of efficiency 0.8: speed falls and torque rises in proportion to the ratio.

Worked example: Choosing a gearbox

A joint must lift a 2 kg load at the end of a 0.4 m arm (ignore the arm's own mass). The worst-case gravity torque is $2 \times 9.81 \times 0.4 = 7.85$ N m. A motor provides at most 0.1 N m continuously at 3000 rpm. The gearbox has $\eta = 0.8$.

Required ratio: $N = \dfrac{7.85}{0.1 \times 0.8} = 98$. Allowing a margin for acceleration, say $N = 120$. Then the joint speed is at most $3000/120 = 25$ rpm $= 2.6$ rad/s $= 150^\circ$/s, which is acceptable.

The inertia argument. There is a second, less obvious reason for a gearbox. A load with inertia J_L seen through a ratio N appears to the motor as only J_L/N^2. With $N = 100$, the motor feels $1/10000$ of the load inertia, so it can accelerate it quickly and is not upset by changes in the load. This is why a good robot does not use a big direct-drive motor where a small motor and a reducer will do. We can state the rule for maximum acceleration transfer: for a given motor, the load accelerates fastest when the reflected load inertia equals the motor's own, $N_{\text{opt}} = \sqrt{J_L/J_M}$.

Derivation: gear relations, reflected inertia and the optimal ratio

Torque. If τ_{in} turns the input at ω_{in}, the power going in is $\tau_{\text{in}}\omega_{\text{in}}$, and the power coming out is $\tau_{\text{out}}\omega_{\text{out}} = \eta\,\tau_{\text{in}}\omega_{\text{in}}$. With $\omega_{\text{out}} = \omega_{\text{in}}/N$ this gives $\tau_{\text{out}} = \eta\,\tau_{\text{in}}\,\omega_{\text{in}}/\omega_{\text{out}} = \eta N \tau_{\text{in}}$.

Reflected inertia. The load turns at $\omega_L = \omega_m/N$ and has kinetic energy $\tfrac{1}{2}J_L\omega_L^2 = \tfrac{1}{2}\dfrac{J_L}{N^2}\omega_m^2$. A motor-side inertia with the same energy at speed ω_m is J_L/N^2, and that is what the motor "feels".

Best ratio. The motor accelerates its own inertia plus the reflected one: $\dot\omega_m = \tau_m/(J_M + J_L/N^2)$. The load acceleration is $\dot\omega_L = \dot\omega_m/N = \dfrac{\tau_m}{NJ_M + J_L/N}$. To make this as large as possible, make the denominator as small as possible. Its slope with respect to N is $J_M - J_L/N^2$, which vanishes at $N = \sqrt{J_L/J_M}$, where $J_M = J_L/N^2$. Example: $J_M = 2 \times 10^{-5}$ kg m^2 and $J_L = 0.02$ kg m^2 give $N_{\text{opt}} = \sqrt{1000} \approx 32$.

Imperfections. Gears have **backlash**, a little free play, which causes a dead zone and limits accuracy in position control. Precision robots use special gearboxes, such as harmonic drives or planetary reducers with preloaded teeth, which have almost no backlash and high ratios in a compact package. The price is cost and efficiency.

A word from the classroom

Students often ask whether a bigger motor would remove the need for a gearbox. In principle yes (direct drive), and some modern robots do it, but the motor must then be large and heavy to produce the same torque, and the heavy motor at the joint has to be carried by the joints before it. Gears let a small, light, fast motor do the work of a big one.

6.6 Hydraulic and pneumatic drives

Liquids are almost incompressible, and pressure in a closed liquid is the same everywhere (Pascal's law). A small piston pushing on a small area creates a pressure which acts on a large piston and produces a large force:

$$p = \frac{F_1}{A_1} = \frac{F_2}{A_2} \quad \Longrightarrow \quad F_2 = F_1 \, \frac{A_2}{A_1}. \tag{6.7}$$

There is no free lunch: the larger piston moves a shorter distance. Equating work in and work out (ignoring losses), $F_1 d_1 = F_2 d_2$, so $d_2 = d_1 A_1 / A_2$.

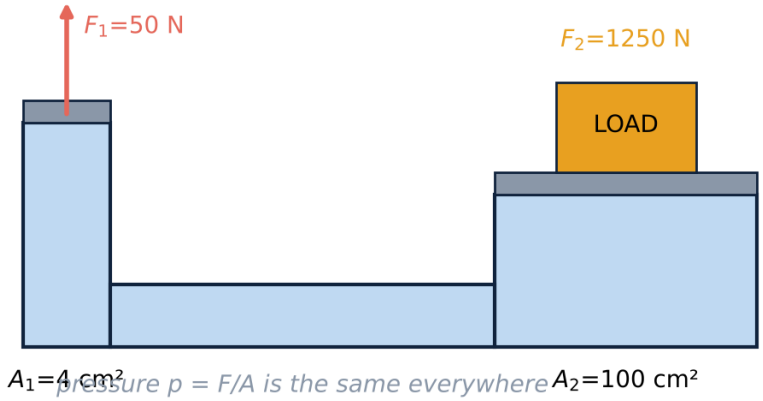


Figure 6.4: A hydraulic lift. A 50 N push on a 4 cm^2 piston lifts 1250 N on a 100 cm^2 piston.

Worked example: Hydraulic lift

$A_1 = 4$ cm^2, $A_2 = 100$ cm^2, push $F_1 = 50$ N over a stroke $d_1 = 50$ mm.
Pressure: $p = 50/(4 \times 10^{-4}) = 125$ kPa. Force out: $F_2 = 125\,000 \times 100 \times 10^{-4} = 1250$ N, which equals $50 \times 100/4$. Distance: $d_2 = 50 \times 4/100 = 2$ mm. Work in: $50 \times 0.05 = 2.5$ J; work out: $1250 \times 0.002 = 2.5$ J. The energy balances.

Hydraulic arms therefore deliver enormous force from compact actuators, which is why excavators and heavy manipulators use them. They need a pump, valves, a reservoir, filters and leak-proof lines, so they are heavier in infrastructure and messier than electric drives.

Pneumatic drives use compressed air. Air is compressible, so a pneumatic cylinder behaves like a spring and cannot hold a position stiffly or be controlled precisely without a lot of effort. But it is clean, quick and cheap, which makes pneumatics ideal for opening and closing simple two-position grippers and for soft robots whose compliance is a feature.

Worked example: Force from an air cylinder

A cylinder of bore 32 mm is supplied with 6 bar gauge (6×10^5 Pa). Area $= \pi(0.016)^2 = 8.04 \times 10^{-4}$ m^2. Force $= 6 \times 10^5 \times 8.04 \times 10^{-4} = 482$ N on the extend stroke. On the retract stroke the rod occupies part of the area, so the force is lower.

6.7 Choosing an actuator: a way of thinking

Start with the job and work backwards.

1. List the worst-case torque or force, including gravity, friction and acceleration of the load.
2. List the maximum speed required at that load.
3. Compute the mechanical power $P = \tau\omega$ at the operating point and compare with the continuous (not peak) rating of the motor.
4. Choose the gear ratio so the motor works near its best speed.
5. Check heating: continuous current should not exceed the rated value, and the stall condition should be protected by current limits.
6. Add a sensor for feedback if position accuracy matters.

6.8 Chapter summary

- DC motor: $\tau = K_t i$, $e_b = K_e \omega$, giving a straight torque–speed line with stall torque $K_t V / R$ and no-load speed V / K_e.
- Steppers move in fixed steps; microstepping refines the step but not the torque; skipped steps go unnoticed without a sensor.
- Gearboxes trade speed for torque by the ratio N and reduce reflected inertia by N^2.
- Hydraulic force gain is A_2 / A_1 paid for by stroke; pneumatics is fast and clean but springy.

Chapter 7

Grippers and Grasping

The end effector is where a robot meets the world, and a surprising share of failed robot projects fail here. A gripper must hold the object firmly enough not to drop it, gently enough not to crush it, and it must do so for objects that are never exactly where or what you expected.

7.1 Families of grippers

- **Mechanical (finger) grippers**: two or more fingers that close on an object. Parallel-jaw designs keep the fingers parallel; angular designs pivot them. They hold the object by squeezing and friction, or by shape.
- **Vacuum (suction) grippers**: a cup connected to a pump. Excellent for smooth, flat, light items such as boxes, glass and sheet metal; useless on porous or very rough surfaces.
- **Magnetic grippers**: electromagnets or permanent magnets for ferrous parts.
- **Soft and adaptive grippers**: flexible fingers, often pneumatically inflated, that conform to irregular shapes such as fruit.
- **Special tools**: welding torches, spray guns, screwdrivers. They are tools more than grippers but occupy the same place on the arm.

Whatever the type, the actuating principle comes from Chapter 6: electric motors with a screw or linkage, or pneumatic pistons for simple open–close motions.

7.2 Gripping by friction

Consider a part of mass m held between two flat fingers, as in Figure 7.1. Each finger presses with a normal force N and so can resist a sliding force up to μN, where μ is the coefficient of friction between finger and part. With two fingers the largest weight they can hold is $2\mu N$. Therefore the condition for holding the part against gravity is

$$2\mu N \geq mg. \tag{7.1}$$

Real systems must also deal with acceleration and uncertainty. If the robot lifts the part upward with acceleration a, the load grows to $m(g+a)$, and a **safety factor** S (2 is common, more for dangerous or valuable parts) is included:

$$N = \frac{S\,m\,(g+a)}{n\,\mu}, \tag{7.2}$$

with n the number of fingers (the contact surfaces that carry friction).

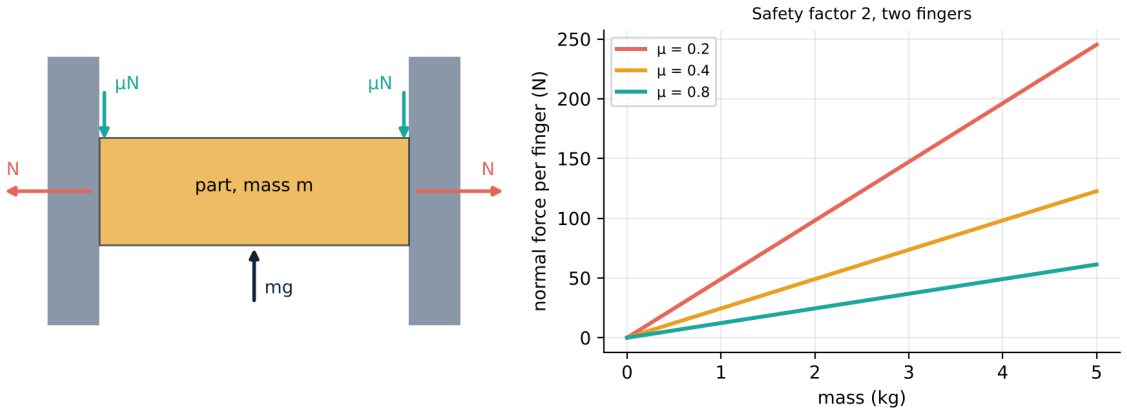


Figure 7.1: Left: forces on a part held between two fingers. Right: the grip force each finger needs for a safety factor of 2.

Worked example: Gripping a 2 kg part

$m = 2$ kg, $\mu = 0.4$, two fingers, $S = 2$, lifted slowly ($a \approx 0$).
Required friction force: $S\,mg = 2 \times 2 \times 9.81 = 39.24$ N. Each finger supplies μN, two fingers give $2\mu N$. So $N = 39.24/(2 \times 0.4) = 49.05$ N per finger.
Now suppose the arm lifts it with $a = 3$ m/s^2. Then $N = 2 \times 2 \times (9.81 + 3)/(2 \times 0.4) = 64.05$ N, about 31% more. A grasp that is perfectly safe in a slow demonstration can drop the part at production speed.

Derivation: the grip-force formula from Newton's second law

Take upward as positive. The part has weight mg pulling down and the friction forces $F_1, F_2, \ldots, F_n$ of the n contacts pulling up. Newton's law gives $\sum F_i - mg = ma$, so the total friction needed is $m(g+a)$. Each contact can supply at most μN before it slips, so the friction available is $n\mu N$. Requiring the available friction to exceed the need by the safety factor S:

$$n\mu N \;\geq\; S\,m(g+a) \quad\Longrightarrow\quad N = \frac{S\,m\,(g+a)}{n\,\mu}.$$

Check the second case of the example: $S\,m(g+a) = 2 \times 2 \times 12.81 = 51.24$ N of friction is needed, and $n\mu = 2 \times 0.4 = 0.8$, so $N = 51.24/0.8 = 64.05$ N. The result varies as $1/\mu$: if oil lowers μ from 0.4 to 0.12 the force must rise by the factor $0.4/0.12 = 3.3$.

What if μ is not known? Then measure it or choose conservatively. Rubber pads on a dry steel part give $\mu \approx 0.5$ to 1; oily steel on steel may be only 0.1 to 0.15, which would raise the required force by a factor of three or four. Oil on a part is an infamous cause of dropped items.

Too much force. Squeezing harder is not always better. A thin plastic bottle, an egg or a circuit board will be damaged by the very force that secures a steel block. The remedy is force control: measure the force with a strain gauge or estimate it from motor current, and stop closing when the target force is reached.

7.3 Form closure and force closure

There are two ways to hold something. With **form closure**, the fingers' *shape* surrounds the part so completely that it cannot move however little friction exists, like a hand cupped around a ball, or a gripper with a V-groove for a shaft. With **force closure**, contact forces and friction together can balance any external disturbance within limits; this is the usual pinch grasp.

Friction contact has a geometric meaning. A point contact can push along the surface normal and can resist tangential force up to μN. Those allowed directions form a cone around the normal, the **friction cone**, whose half-angle is $\arctan \mu$. A contact force that lies outside its cone makes the finger slip. For $\mu = 0.4$ the half-angle is 21.8°; for $\mu = 1$ it is 45°. If a finger presses at an angle to the surface greater than this, the part squirts out. This is why a well-designed gripper has fingers with a high-friction pad and surfaces that meet the object nearly face-on.

Derivation: the friction cone

At a contact the finger pushes on the part with a normal component N and a tangential component F_t. The surface holds without slipping as long as $F_t \le \mu N$. The resultant force makes an angle φ with the normal, where $\tan\varphi = F_t / N$. The condition for no slip is therefore $\tan\varphi \le \mu$, that is $\varphi \le \arctan\mu$. All directions satisfying this form a cone around the normal. Numbers: $\mu = 0.4$ gives $\arctan 0.4 = 21.8^\circ$, $\mu = 1$ gives 45°, and oily steel with $\mu = 0.12$ gives only 6.8°, a very narrow cone.

7.4 Vacuum grippers

A suction cup holds by the difference between the outside pressure and the pressure in the cup. If the pressure inside is lower than atmosphere by Δp over a seal area A, the holding force is

$$F = \Delta p \, A. \tag{7.3}$$

For a safe hold against gravity, with a safety factor S and n cups,

$$\Delta p \, A \, n \;\ge\; S \, m(g + a). \tag{7.4}$$

The effect of orientation is important. When the load hangs below the cup (a vertical lift) the vacuum force must support the weight; when a cup lifts a part with the surface vertical (e.g. the part is picked from a wall) friction carries the weight and the effective force is multiplied by μ.

Worked example: Suction lift of a carton

A carton of mass 3 kg is lifted horizontally-flat by cups of diameter 40 mm. The vacuum level is -60 kPa (pressure 60 kPa below atmosphere) and we use $S = 2$ with $a = 2\ \mathrm{m/s^2}$.

One cup's area: $\pi(0.02)^2 = 1.257 \times 10^{-3}$ m^2. Force per cup: $60\,000 \times 1.257 \times 10^{-3} = 75.4$ N. Required total: $2 \times 3 \times (9.81 + 2) = 70.9$ N. So one cup is barely enough on paper; with leakage on rough cardboard we would use two or three. With $n = 3$ the available force is 226 N, a margin of 3.2 over the load.

Derivation: why $F = \Delta p\, A$, and an upper limit

Pressure is force per area, so a pressure difference Δp across a surface of area A pushes with a force $\Delta p \times A$. The atmosphere presses on the outside of the part with about 101 kPa, and the cup lowers the pressure beneath by Δp. The largest possible Δp is the whole atmosphere, so a perfect vacuum cup holds at most 101 000 N/m^2 = 10.1 N per square centimetre. Real pumps reach -60 to -80 kPa, i.e. 6 to 8 N/cm^2. For the 40 mm cup, $A = \pi(2\text{ cm})^2 = 12.57$ cm^2 and 60 kPa is 6.0 N/cm^2, giving 75.4 N, as in the example. Doubling the cup diameter multiplies the force by four, because the area goes as the square of the diameter.

Because pumps are noisy and leaky, vacuum systems usually monitor the pressure: a pressure sensor tells the program whether the part is held, which is sensing and control inside the gripper.

7.5 Choosing a gripper

Table 7.1: A rough guide.

Object	Good choice	Watch out for
Rigid, known shape	two-finger parallel jaw	jaw alignment, finger wear
Flat sheet, box, glass	vacuum cups	porous or dirty surfaces, leaks
Steel part	electromagnet	residual magnetism, sudden power loss
Fruit, soft items	soft or adaptive fingers	damage by excess force
Mixed items in a bin	adaptive gripper + vision	grasp planning is hard

7.6 Chapter summary

- Gripper types match object types; vacuum suits smooth flat items, fingers suit rigid ones.
- Friction grip: $N = S\, m(g + a)/(n\mu)$.
- The friction cone has half-angle $\arctan \mu$.
- Suction: $F = \Delta p\, A$ per cup; allow for leaks and acceleration.
- Too much force damages; measure and limit it.

Chapter 8

Intelligence for Robots

Up to now our robot has been a very good servant. It knows where its hand is, it senses, it moves smoothly, it grips. What it lacks is judgement: the ability to choose *what* to do. Artificial intelligence is the study of building that judgement. In this chapter I will explain what the field is trying to do, give you the agent model that organises almost everything else, and then take you inside the smallest working piece of machine learning, the artificial neuron.

8.1 What do we mean by AI?

There is no single definition, and I would not trust anyone who gives you one with great confidence. The common approaches can be sorted along two questions: is the goal to *think* or to *act*, and is the yardstick *human behaviour* or *ideal rational behaviour*? For engineering purposes the last combination is the most useful: build a system that acts rationally, which means it picks the action that is expected to give the best outcome according to its goals and the information it has. This definition has a lovely practical side. It lets us measure intelligence by performance in a task rather than by arguing about what thinking is.

AI is also split by ambition. **Narrow AI** does one kind of task, such as recognising objects or planning a path, and works well within its limits. **General AI**, a machine able to learn any intellectual task a person can, remains a research aim and not a product. Every robot you will meet in your career carries narrow AI.

8.2 The agent model

An **agent** is anything that perceives its environment through sensors and acts upon it through actuators. This is exactly our sense–decide–act loop in different words, and it is the reason the AI chapters belong in a robotics course. The agent's behaviour is governed by an **agent function** that maps everything it has perceived so far to an action. The program that implements the function is the **agent program**.

A neat way to specify any agent problem is to fill in four boxes, abbreviated **PEAS**: Performance measure, Environment, Actuators, Sensors.

Writing the PEAS box before writing any code has saved me more time than any other habit. It forces the question "what exactly does success mean?", which is usually the hardest one.

Table 8.1: PEAS descriptions for three robot tasks.

Agent	Performance	Environment	Actuators	Sensors
Warehouse mover	parcels delivered, time, no collisions	aisles, shelves, other robots, people	wheels, lift	LiDAR, encoders, camera
Robot vacuum	area cleaned, battery used	rooms, furniture, pets, stairs	wheels, brush, fan	bumper, cliff, dust sensors
Fruit picker	ripe fruit harvested, damage	orchard rows, trees, weather	arm, soft gripper	camera, force sensor

8.2.1 Properties of environments

The right design depends heavily on the environment. Ask these questions of any task.

- **Fully or partially observable?** Can the sensors see everything that matters? A chess board is fully visible; a corridor with closed doors is not.
- **Deterministic or stochastic?** Does an action always have the same effect? A wheel on carpet slips unpredictably, so real robots live in stochastic worlds.
- **Episodic or sequential?** Does each decision stand alone, or does it affect later ones? Sorting items on a conveyor is nearly episodic. Navigating a building is sequential.
- **Static or dynamic?** Does the world change while the agent thinks?
- **Discrete or continuous?** Finite moves, or real-valued joint angles and times?
- **Single or multiple agents?** Are there other deciders, cooperating or competing?

The hardest case for a robot, and the commonest, is partially observable, stochastic, sequential, dynamic, continuous and multi-agent. The same problems in a simulator are much easier, which is why simulators are teaching tools and not the full story.

8.2.2 Types of agent programs

1. **Simple reflex agents** choose actions from the current percept only through condition–action rules: "if bump switch pressed, back up and turn". Fast and robust, but they cannot remember anything.
2. **Model-based reflex agents** keep an internal state describing the parts of the world they cannot see right now, updated from their percept history and from knowledge of how the world works.
3. **Goal-based agents** compare the predicted results of different actions with a goal and choose the action sequence that reaches it. Planning and search (Chapter 9) live here.
4. **Utility-based agents** go further: when several ways of reaching a goal exist, or goals conflict, they select the option with the highest *utility*, a numerical measure of how desirable an outcome is.

5. **Learning agents** improve their performance from experience. They add a learning element that changes the other components.

Worked example: A reflex rule set for a line-following robot

Two infrared sensors, left (L) and right (R), return 1 when over the dark line. Rules: if L=0, R=0, go straight (line is between the sensors); if L=1, R=0, turn left (the line has drifted left); if L=0, R=1, turn right; if L=1, R=1, stop (junction). This is a simple reflex agent with four rules. Place the robot in a curved track and notice that it works; place it on a track with a gap in the line and it is lost, because it has no memory of where the line was. A model-based agent that remembers the last turn direction can search in the right direction after the gap.

8.3 Learning from examples

So far the rules were written by a person. Machine learning takes a different path: we show the machine examples and let it find the rule. The commonest setting is **supervised learning**. We have a set of inputs $\mathbf{x}$ with the correct outputs y (the labels) and we want a function that predicts y for new $\mathbf{x}$. If y is a number, the task is *regression*; if it is a category, *classification*. In **unsupervised learning** there are no labels and the machine looks for structure, such as groups of similar items. In **reinforcement learning** an agent learns by trying actions and receiving rewards, an approach suited to control and games.

8.4 The artificial neuron

A neuron in the brain collects signals from many other neurons and, if the total is strong enough, fires. The artificial version is a crude but very useful imitation. Given inputs $x_1, \dots, x_m$, weights $w_1, \dots, w_m$ and a bias b, it computes

$$z = \sum_{i=1}^{m} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b, \qquad y = \sigma(z). \tag{8.1}$$

The **weights** say how much each input matters, the **bias** shifts the threshold, and the **activation function** σ decides how the sum becomes an output.

The popular activations are the **sigmoid** $\sigma(z) = 1/(1+e^{-z})$, squeezing any value into $(0,1)$ and interpretable as a probability; the hyperbolic tangent, which does the same into $(-1,1)$; and the **ReLU**, $\max(0,z)$, which is trivial to compute and works very well in deep networks.

Worked example: One neuron by hand

Inputs $\mathbf{x} = (2,\ -1)$, weights $\mathbf{w} = (0.5,\ 1.2)$, bias $b = 0.3$.
$z = 0.5(2) + 1.2(-1) + 0.3 = 1.0 - 1.2 + 0.3 = 0.1$.
Sigmoid: $y = 1/(1+e^{-0.1}) = 1/(1+0.9048) = 0.525$.
The neuron is nearly undecided (about 52%) because the weighted evidence almost cancels.

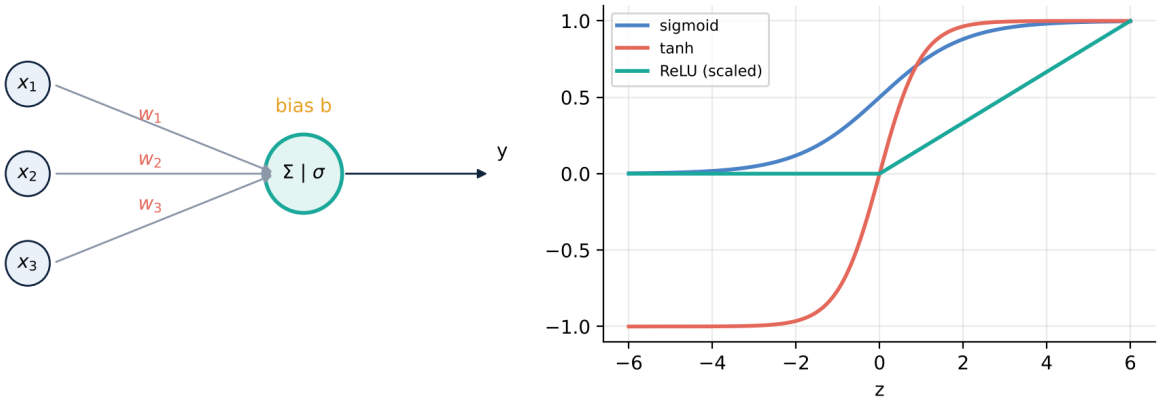


Figure 8.1: Left: the artificial neuron. Right: three common activation functions.

A neuron is a line. With two inputs and a step output, the neuron says "class 1" whenever $w_1x_1 + w_2x_2 + b > 0$. That equation describes a straight line dividing the plane into two sides. A single neuron can therefore only separate data that a straight line can separate. This limitation will matter in a minute.

Derivation: the neuron's boundary, an AND gate, and XOR built by hand

Setting $w_1x_1 + w_2x_2 + b = 0$ and solving for x_2 gives $x_2 = -\dfrac{w_1}{w_2}x_1 - \dfrac{b}{w_2}$, a straight line with that slope and intercept. Points on one side make the sum positive, on the other side negative; the weight vector (w_1, w_2) points toward the positive side.

An AND gate. Take $w_1 = w_2 = 1$, $b = -1.5$ and a step output (1 if $z > 0$). For $(0,0)$: $z = -1.5$; for $(1,0)$ or $(0,1)$: $z = -0.5$; for $(1,1)$: $z = 0.5$. Only $(1,1)$ gives a positive sum, so the neuron computes AND. Changing b to -0.5 gives OR.

XOR with two layers. Let hidden neuron $h_1 = \text{step}(x_1 + x_2 - 0.5)$ (it is OR) and $h_2 = \text{step}(x_1 + x_2 - 1.5)$ (it is AND). Let the output be $\text{step}(h_1 - h_2 - 0.5)$.

x_1	x_2	h_1	h_2	$h_1 - h_2 - 0.5$	output
0	0	0	0	-0.5	0
0	1	1	0	0.5	1
1	0	1	0	0.5	1
1	1	1	1	-0.5	0

The network says "OR but not AND", which is exactly XOR. Training does nothing mysterious: it searches for weights like these automatically.

8.5 How a neuron learns: gradient descent

How do we find good weights? Define a **loss** that measures how wrong the neuron is on the training examples, for instance the mean squared error $L = \frac{1}{N}\sum_n (\hat{y}_n - y_n)^2$. Learning is the search for weights that make L small. The simplest method is to look at the slope of L with respect to each weight and take a small step downhill:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \tag{8.2}$$

where η is the **learning rate**. The gradient points to the steepest ascent, so subtracting it descends.

Worked example: Learning one weight

A single linear neuron $\hat{y} = wx$ must learn $y = 3x$ from one example $x = 2$, $y = 6$. Loss $L = (wx - y)^2 = (2w - 6)^2$. The gradient is $dL/dw = 2(2w - 6)(2) = 8w - 24$. Start with $w = 0$ and $\eta = 0.05$.

Step 1: gradient -24, new $w = 0 + 0.05 \times 24 = 1.2$. Step 2: gradient $8(1.2) - 24 = -14.4$, new $w = 1.2 + 0.72 = 1.92$. Step 3: gradient -8.64, new $w = 1.92 + 0.432 = 2.352$. Step 4: gradient -5.18, new $w = 2.611$.

Each step leaves the remaining gap multiplied by $(1 - 8\eta) = 0.6$. The weight approaches 3 geometrically. If η were 0.3, the factor would be $1 - 2.4 = -1.4$ and the weight would swing away from the answer in growing oscillations, the sign of too large a learning rate.

Derivation: why the gap shrinks by $(1 - \eta\lambda)$, and where the stability limit comes from

For one weight, $L = (wx - y)^2$ has the minimum at $w^* = y/x$ (here 3), and $\dfrac{dL}{dw} = 2x(wx - y) = 2x^2(w - w^*)$. The update is therefore

$$w_{k+1} - w^* = (w_k - w^*) - \eta\, 2x^2(w_k - w^*) = (1 - 2\eta x^2)(w_k - w^*).$$

The gap to the answer is multiplied by $1 - 2\eta x^2$ at every step; with $x = 2$ and $\eta = 0.05$ this is $1 - 0.4 = 0.6$. The initial gap is 3, so after four steps it is $3(0.6)^4 = 0.389$ and $w = 2.611$, matching the table. The quantity $2x^2$ is the curvature λ of the loss. The gap shrinks only if $|1 - \eta\lambda| < 1$, that is $0 < \eta < 2/\lambda$. For the valley $f = \frac{1}{2}(x^2 + 10y^2)$ the second derivatives are 1 and 10; the steepest direction has $\lambda = 10$, so $\eta < 2/10 = 0.2$, and the path in the figure for $\eta = 0.215$ is just beyond it.

This effect has a picture. Figure 8.2 shows descent on an elongated valley, a bad but common situation: the slope is steep across the valley and shallow along it. A small rate crawls; a good rate zig-zags a little but arrives; a rate a hair too large diverges. For this valley, the stability limit is $\eta < 2/10 = 0.2$ because the steepest direction has curvature 10.

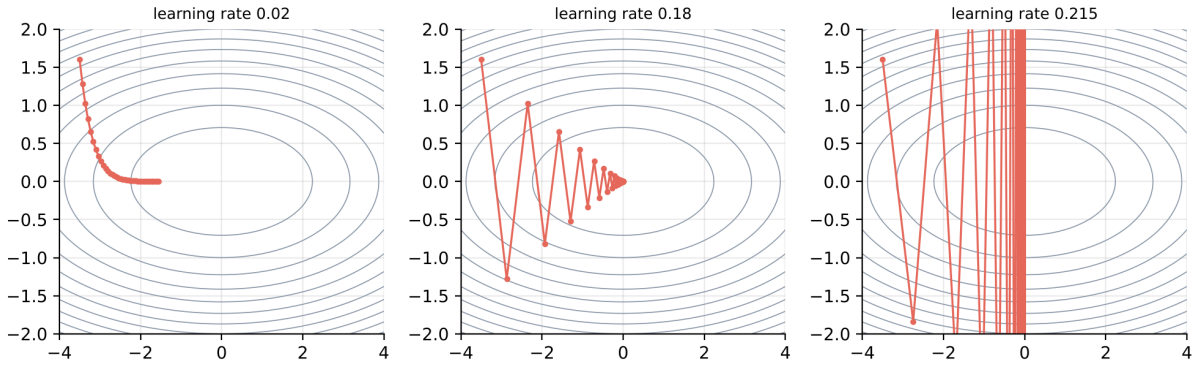


Figure 8.2: Gradient descent on $f(x, y) = \frac{1}{2}(x^2 + 10y^2)$ for three learning rates. The last, 0.215, is just past the stability limit and the path blows up.

8.6 Layers of neurons

If one neuron is a line, a layer of neurons followed by another layer can draw curved and disconnected regions. The famous test is XOR: the output must be 1 when exactly one of two inputs is 1. The four points $(0,0)$, $(1,1)$ (output 0) and $(0,1)$, $(1,0)$ (output 1) cannot be separated by a single line, but a network with one hidden layer can do it, because the hidden neurons each draw a line and the output neuron combines them.

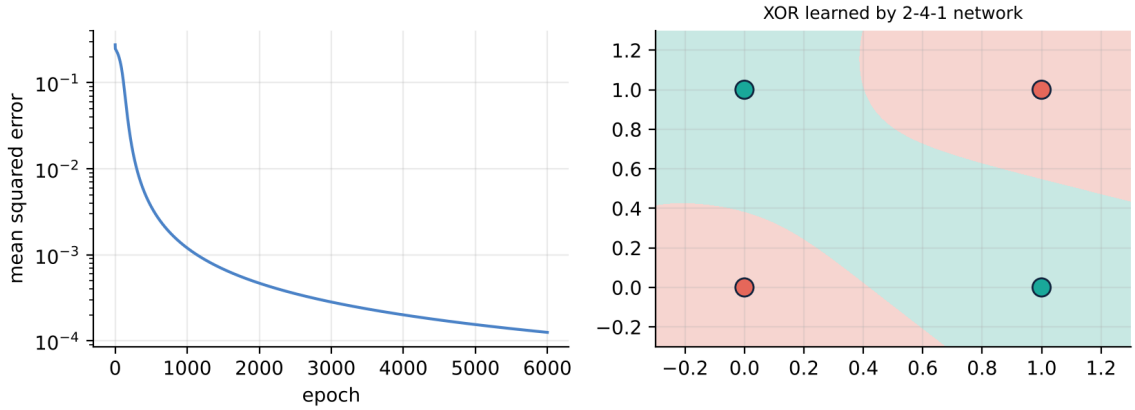


Figure 8.3: A network with 2 inputs, 4 hidden sigmoid neurons and 1 output trained on XOR. Left: the loss falls over 6000 epochs. Right: the learned decision regions separate the two classes with a bent boundary.

Training such a network needs the gradient of the loss with respect to *every* weight, including those in early layers. The **backpropagation** algorithm does this by applying the chain rule layer by layer from the output backward. If $h = \sigma(W_1\mathbf{x} + \mathbf{b}_1)$ and $\hat{y} = \sigma(W_2 h + \mathbf{b}_2)$, then for squared error $L = \frac{1}{2}(\hat{y} - y)^2$ the output-layer error signal is

$$\delta_2 = (\hat{y} - y)\,\hat{y}(1 - \hat{y}),$$

and it is passed back as

$$\delta_1 = \left(W_2^T \delta_2\right) \odot h \odot (1 - h),$$

where $\odot$ means elementwise product. The weight updates are $\Delta W_2 = -\eta\,\delta_2 h^T$ and $\Delta W_1 = -\eta\,\delta_1 \mathbf{x}^T$. The code that produced the figure is only a dozen lines; Appendix B has a version you can run.

Derivation: backpropagation from the chain rule, with a numerical example

Write $z_1 = W_1\mathbf{x} + \mathbf{b}_1$, $h = \sigma(z_1)$, $z_2 = W_2 h + \mathbf{b}_2$, $\hat{y} = \sigma(z_2)$ and $L = \frac{1}{2}(\hat{y} - y)^2$. First, the derivative of the sigmoid: with $\sigma(z) = (1 + e^{-z})^{-1}$,

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)\,\frac{e^{-z}}{1 + e^{-z}} = \sigma(z)\big(1 - \sigma(z)\big).$$

Now go backward, one link of the chain at a time. $\partial L/\partial \hat{y} = \hat{y} - y$ and $\partial \hat{y}/\partial z_2 = \hat{y}(1 - \hat{y})$, so $\delta_2 = \partial L/\partial z_2 = (\hat{y} - y)\hat{y}(1 - \hat{y})$. Because $z_2 = W_2 h + \mathbf{b}_2$, we have $\partial z_2/\partial W_2 = h$, so $\partial L/\partial W_2 = \delta_2 h^T$. The hidden layer feels the error through W_2: $\partial L/\partial h = W_2^T \delta_2$, and passing through the sigmoid, $\delta_1 = \partial L/\partial z_1 = (W_2^T \delta_2) \odot h \odot (1 - h)$. Finally $\partial L/\partial W_1 = \delta_1 \mathbf{x}^T$. Each weight is moved by $-\eta$ times its gradient.

A one-weight-per-layer example. Input $x = 1$, hidden weight $w_1 = 0.5$, output weight

$w_2 = 0.8$, biases zero, target $y = 1$, $\eta = 0.5$.

$$h = \sigma(0.5) = 0.6225, \qquad \hat{y} = \sigma(0.8 \times 0.6225) = \sigma(0.4980) = 0.6220,$$

$$L = \tfrac{1}{2}(0.6220 - 1)^2 = 0.0714,$$

$$\delta_2 = (0.6220 - 1)(0.6220)(0.3780) = -0.0889, \qquad \partial L/\partial w_2 = \delta_2 h = -0.0553,$$

$$\delta_1 = (0.8)(-0.0889)(0.6225)(0.3775) = -0.0167, \qquad \partial L/\partial w_1 = \delta_1 x = -0.0167.$$

Update: $w_2 = 0.8 + 0.5(0.0553) = 0.8277$ and $w_1 = 0.5 + 0.5(0.0167) = 0.5084$. The new output is $\hat{y} = 0.6264$ and the loss has fallen from 0.0714 to 0.0698. One small step, in the right direction; training is just thousands of these.

8.7 Generalisation: the real goal

A network that memorises its training examples is useless. What we want is performance on *new* data. Two failure modes cover most cases.

- **Underfitting**: the model is too simple to capture the pattern; error is high on both training and new data.
- **Overfitting**: the model is so flexible that it fits the noise in the training set; error on training data is low but error on new data is high.

The standard defence is to split the data into a **training set** used to fit the weights and a **test set** that is never touched until the end. A **validation set** is used for choosing settings. If the test error is much worse than the training error, the model has overfitted. Remedies include more data, a smaller model, early stopping and regularisation (a penalty on large weights).

A word from the classroom

A confession from the lab: the most common mistake I see in student projects is testing on the same data used for training and then reporting 99% accuracy. A robot that was "perfect" in the lab then fails on the first day outdoors, because lighting, floor and objects were different. Always keep some data aside, and make it as different from the training set as the real world will be.

8.8 Where learning enters a robot

- **Perception**: recognising objects, people, lanes and obstacles in images and point clouds.
- **Grasping**: predicting from an image where a gripper should close.
- **Control**: learning a controller or tuning parameters from experience, as in reinforcement learning of walking.
- **Modelling**: learning friction or payload effects that are hard to write by hand.

Learning is not a replacement for the physics of the earlier chapters; the best systems use both. A learned vision system tells the arm where the part is, and inverse kinematics and a well-tuned controller then take the hand there.

8.9 Chapter summary

- Rational agents choose actions expected to achieve the best outcome; PEAS describes any agent problem.
- Environments differ in observability, determinism, dynamics and more; design follows from them.
- An artificial neuron computes $\sigma(\mathbf{w}^T\mathbf{x} + b)$; a single neuron draws a line.
- Gradient descent updates weights against the slope of the loss; the learning rate must be small enough for stability.
- Hidden layers allow curved boundaries (XOR); backpropagation provides the gradients.
- Use separate training and test data to measure generalisation.

Chapter 9

Search, Games and Evolution

Whenever a robot must decide among many possible sequences of actions, it faces a search problem. Which route through the warehouse? Which order of moves in a game against an opponent? Which design among millions of candidates? This chapter gives you three families of methods: search through a state space, search against an opponent, and search by imitation of evolution.

9.1 Formulating a search problem

A search problem has five ingredients:

1. a set of **states** (the situations the world can be in),
2. an **initial state**,
3. the available **actions** in each state,
4. a **transition model** (what state results from an action),
5. a **goal test**, and a **path cost** that adds up the cost of the steps.

For a mobile robot on a grid map the state is its cell (r, c); the actions are moves to the four neighbours that are not walls; every step costs 1; the goal test is "am I in the target cell?". A **solution** is a sequence of actions from the start to a goal, and we usually want one of least cost.

We organise the exploration as a **search tree**. The root is the initial state; its children are the states reachable by one action. The set of nodes waiting to be explored is called the **frontier**. Every search algorithm is the same loop,

```
frontier = [start]
explored = {}
while frontier:
    node = choose_and_remove(frontier) # <- the only thing that differs
    if is_goal(node): return path_to(node)
    explored.add(node)
    for child in expand(node):
        if child not in explored: frontier.add(child)
```

and the algorithms differ only in how they choose the next node.

9.2 Uninformed search

Breadth-first search (BFS) takes the oldest node in the frontier (a queue). It explores all states at distance 1, then 2, and so on. For equal step costs it always finds a shortest path, but it must remember an entire layer of the tree. If each state has branching factor b and the shallowest goal is at depth d, BFS visits about b^d nodes.

Depth-first search (DFS) takes the newest node (a stack), so it dives deep before backing up. It needs very little memory, about $b\,m$ nodes for a maximum depth m, but it can go down a long useless branch and does not guarantee the shortest path.

Uniform-cost search (Dijkstra's algorithm) takes the node with the smallest path cost so far, $g(n)$. It handles unequal step costs and is optimal.

Table 9.1: Properties of basic search methods (b branching factor, d solution depth, m maximum depth).

Method	Finds shortest?	Time	Memory	Note
BFS	yes (equal costs)	b^d	b^d	memory-hungry
DFS	no	b^m	bm	may wander
Uniform cost	yes	$\approx b^{1+C^*/\epsilon}$	same	uses $g(n)$
A*	yes (admissible h)	depends on h	large	uses $g+h$

The numbers b^d explain why blind search collapses on real problems. With $b = 4$ and $d = 20$, we would be looking at about $4^{20} \approx 10^{12}$ nodes. We need a way to use knowledge of the goal.

Derivation: counting nodes

A tree with branching factor b has 1 node at depth 0, b at depth 1, b^2 at depth 2, and so on. The total up to depth d is the geometric sum

$$1 + b + b^2 + \cdots + b^d = \frac{b^{d+1} - 1}{b - 1} \approx b^d \cdot \frac{b}{b-1},$$

which for $b \geq 2$ is within a small factor of the last term b^d. The last layer is by far the largest, which is why BFS memory grows as b^d. For $b = 4$, $d = 20$: $4^{20} = 1\,099\,511\,627\,776 \approx 1.1 \times 10^{12}$. At a million nodes per second that is about 13 days of computing.

9.3 Informed search and A*

Suppose we have a **heuristic** $h(n)$, an estimate of the remaining cost from node n to the goal. **A*** chooses the node with the smallest

$$f(n) = g(n) + h(n), \tag{9.1}$$

that is, the cheapest path known so far plus the estimated cheapest remaining. The method is as careful as uniform cost (it counts the cost already paid) and as eager as a greedy walker (it prefers nodes that look close to the goal).

Admissible heuristics. A heuristic is *admissible* if it never overestimates the true remaining cost: $h(n) \leq h^*(n)$. If h is admissible, A* returns an optimal path. The reasoning is short: the first

time the goal is removed from the frontier, its f equals its true cost g; any other unexplored node on a better path would have $f \leq$ that path's true cost, so it would have been taken first. For grid navigation with four-way moves, the **Manhattan distance** $h = |r - r_g| + |c - c_g|$ is admissible, since you cannot get there faster than by walking the row and column differences with no walls. If diagonal moves are allowed, the straight-line (Euclidean) distance is the safe choice.

Derivation: why an admissible heuristic gives an optimal path

Let C^* be the cost of the best path. Suppose A* is about to remove from the frontier a goal node G_2 reached by a worse path, so $f(G_2) = g(G_2) > C^*$ (at a goal $h = 0$). Because the true best path exists, at least one node n on it is still in the frontier, and for that node

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*,$$

where we used admissibility, $h \leq h^*$, and the fact that $g(n)$ plus the true remaining cost on the best path is C^*. So $f(n) \leq C^* < f(G_2)$. A* always removes the node of smallest f, so it would have taken n, not G_2. The assumption leads to a contradiction, so the first goal removed is optimal.

Why Manhattan distance is admissible. Each four-way move changes the row or the column by exactly one, so it can reduce $|r - r_g| + |c - c_g|$ by at most 1. To bring this quantity from h to zero needs at least h moves, so the true cost is never less than h. Walls only add moves.

Worked example: Computing f

Start $(2, 3)$, goal $(7, 9)$ on a four-connected grid. The Manhattan heuristic is $h = |7 - 2| + |9 - 3| = 5 + 6 = 11$ at the start. After four steps we stand at $(4, 5)$ with $g = 4$: $h = |7 - 4| + |9 - 5| = 3 + 4 = 7$, so $f = 4 + 7 = 11$. Notice that f stays at 11 along any path that heads straight toward the goal, and it rises only when the robot is forced to go around a wall. That behaviour is the heart of why A* ignores most of the map. Compare a cell $(3, 4)$ that was reached only after a detour, with $g = 4$: it has $h = 4 + 5 = 9$ and $f = 13$, a poorer choice, so it waits in the frontier while the better nodes are explored first.

The saving here is moderate because walls force a long detour, which cancels some of the heuristic's guidance. In open spaces, A* saves a lot more. The quality of the heuristic is everything: $h = 0$ turns A* into uniform-cost search, and an h equal to the true cost makes it walk straight to the goal.

A word from the classroom

If a student's A* returns a path that is not the shortest, I first check the heuristic. A heuristic that overestimates, such as Manhattan distance multiplied by a weight of 2 "to go faster", makes the search quick but gives up optimality. That can be a perfectly good decision, called weighted A*, as long as it is made on purpose.

9.4 Local search and the idea of optimisation

Sometimes the path does not matter, only the final state: the best arrangement of components, the best gain settings for a controller, the best shape of a gripper finger. In these problems

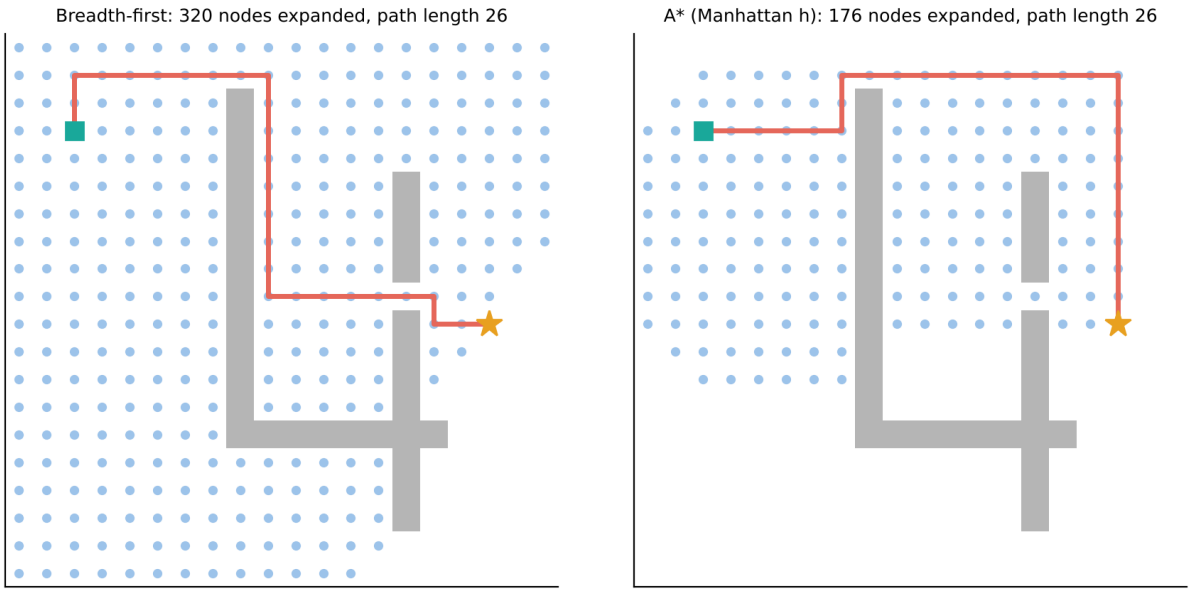


Figure 9.1: Planning on a 20×20 map with walls. Both methods find a shortest path of length 26, but breadth-first search expands 320 cells while A* with a Manhattan heuristic expands 176 (light blue dots).

we search a landscape of candidate solutions and try to find the highest point of an objective function f. **Hill climbing** repeatedly moves to the best neighbour and stops when none is better. It is quick, and it gets stuck on small peaks (local maxima). Adding randomness, for example occasionally accepting a worse move in simulated annealing, or running many searches in parallel, helps escape. Evolutionary methods, below, are one such family.

9.5 Adversarial search: minimax

Now an opponent enters. In a two-player game where we choose a move, then the opponent, then we again, our best move depends on what the opponent will reply. The opponent wants us to lose. **Minimax** assumes the opponent always plays its best reply. Two kinds of nodes alternate: at a MAX node it is our turn and we take the *largest* value among the children; at a MIN node it is the opponent's turn and it takes the *smallest*. Values at the leaves come from an evaluation function (win $= +1$, loss $= -1$, or any score). Working up from the leaves gives the value of the root, the best score we can guarantee against best play.

Minimax is not only for board games. A drone that must avoid a moving obstacle, or two robots negotiating a doorway, can be modelled by the same logic whenever the other party is assumed to act against us or we want a worst-case guarantee.

9.5.1 Alpha–beta pruning

The game tree is exponential, but we do not need to visit all of it. Keep two numbers while searching: α, the best value MAX can already guarantee on the path to the root, and β, the best value MIN can already guarantee. If at some node $\alpha \geq \beta$, the remaining children cannot change the final decision and are skipped (pruned). The result is *the same* as minimax with often far less work. With perfect ordering the number of leaves examined falls from b^d to roughly $b^{d/2}$, which lets the search go twice as deep in the same time.

Derivation: where $b^{d/2}$ comes from

With the best possible move ordering, at a node where one side must prove that its move is good, *all* b replies must be examined; at the next level only *one* reply (the best one) is needed to show that the branch can be cut. Full levels and single-child levels alternate, so the number of leaves examined is the product of b for every second level: about $b^{d/2}$. The exact count for a tree of depth d is $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$. Example: $b = 3$, $d = 4$. Plain minimax visits $3^4 = 81$ leaves; perfect alpha–beta visits $3^2 + 3^2 - 1 = 17$. For $b = 4$ and $d = 20$ the figures are 1.1×10^{12} against about 1×10^6.

Worked example: A small tree

MAX root, two MIN children. The left child has leaves $3, 5, 2$; the right child has leaves $4, 1, 6$.
Left child: $\min(3, 5, 2) = 2$. MAX now has $\alpha = 2$.
Right child: look at the first leaf, 4: current MIN value ≤ 4. Next leaf, 1: MIN value ≤ 1. Since $1 < \alpha = 2$, MAX will never choose this branch, so the last leaf (6) is pruned.
Root value $= \max(2, \leq 1) = 2$. One leaf of six was never examined.

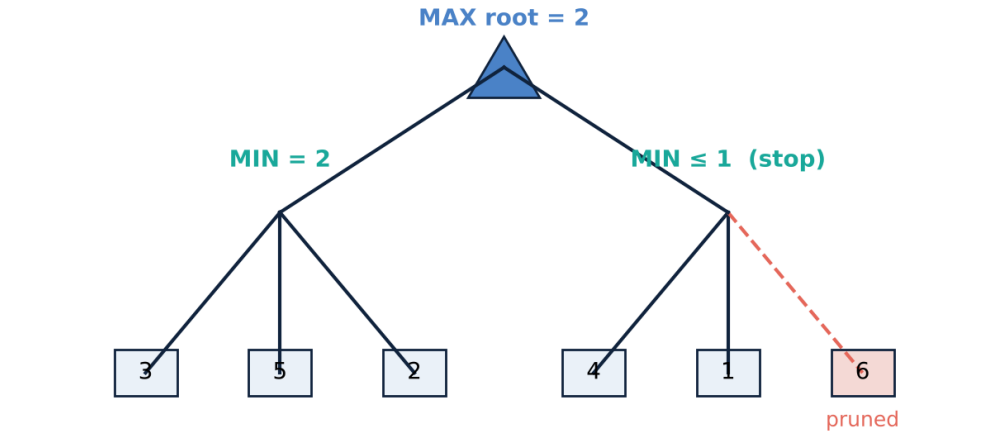


Figure 9.2: Alpha–beta pruning on the example tree. After the left branch guarantees 2, a single leaf below 2 in the right branch makes the rest irrelevant.

Move ordering matters. If the best moves are examined first, pruning is maximal; if the worst are examined first, nothing is pruned. Good programs sort moves with a quick heuristic before searching. And since the whole tree can never be searched to the end, the search stops at a fixed depth and applies an evaluation function there. Designing a good evaluation function is where domain knowledge enters.

9.6 Genetic algorithms

Evolution is itself an optimiser, and we can borrow it. A **genetic algorithm** (GA) keeps a population of candidate solutions, each encoded as a chromosome, a string of bits or numbers. It repeats these steps.

1. **Evaluate** the fitness of each individual with an objective function.
2. **Select** parents, favouring the fitter ones (for example by tournament: pick three at random and keep the best).
3. **Crossover**: pair parents and swap parts of their chromosomes to produce children.
4. **Mutate**: flip each bit with a small probability, say 1% to 3%.
5. Form the new generation and repeat. Often the best individual is carried over unchanged (elitism).

Encoding. Suppose we want the $x \in [0, 4]$ that maximises some fitness. We use an 8-bit chromosome, a number from 0 to 255, and decode it as $x = \dfrac{\text{integer}}{255} \times 4$.

Worked example: Decoding a chromosome

The chromosome 10110010 has the value $1 \cdot 128 + 0 \cdot 64 + 1 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 178$. Then $x = 178/255 \times 4 = 2.792$. The resolution of this encoding is $4/255 = 0.0157$ per bit; more bits give a finer search.
Crossover of $1011 \,|\, 0010$ and $0100 \,|\, 1101$ at the cut after the fourth bit produces 10111101 and 01000010.

Derivation: the decoding rule and the pressure of tournament selection

An 8-bit string can hold the integers $0, 1, \ldots, 255$, which are 255 equal steps between the smallest and largest values. Mapping $0 \mapsto 0$ and $255 \mapsto 4$ gives $x = \text{integer} \times 4/255$. Check: $178 \times 4/255 = 2.792$, and the step between neighbouring codes is $4/255 = 0.0157$.
Selection pressure. In a population of $N = 20$, a tournament draws 3 individuals at random. The chance that a particular individual (say the best) is *not* among them is $(1 - 1/20)^3 = 0.857$, so it is among them with probability 0.143, and then it always wins. With 20 tournaments to fill the next parent pool the best individual expects about $20 \times 0.143 = 2.9$ places, compared with 1 place if parents were picked at random. The median individual wins only when both rivals are worse, probability about $0.5^3 = 0.125$ (and is drawn with 0.143), so it expects well under one place. This is how selection favours the fit without excluding the weak completely, which keeps variety in the population.
Mutation rate. With a chromosome of $L = 8$ bits and a flip probability $p = 0.02$, the expected number of flipped bits per child is $Lp = 0.16$: most children are unchanged, and about one in six has a single change.

In the figure the fitness landscape has several bumps, which defeats a simple hill climber started in the wrong place, but the population spreads over the landscape and finds the highest peak. GAs are not the fastest method when you have gradients, but they handle problems where you do not: designing robot gaits, scheduling, evolving controller parameters in a simulator.

9.7 Choosing among methods

- Known map, one agent, want the shortest path: A*.

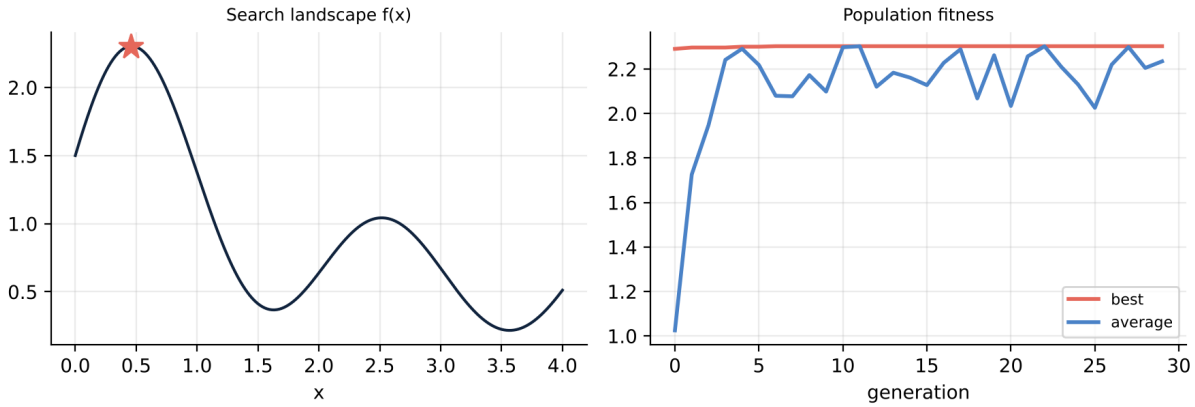


Figure 9.3: A GA with 20 individuals optimising the function on the left. The best solution found (red star, $x = 0.455$) is within 0.006 of the true peak ($x = 0.461$) after about ten generations.

- Opponent whose choices matter: minimax with alpha–beta.
- Large, bumpy design space with no gradient: genetic algorithm or other stochastic search.
- Continuous smooth problem with gradient: gradient-based optimisation (Chapter 8).

9.8 Chapter summary

- Search problems have states, actions, a transition model, a goal test and path cost; the algorithms differ in how they choose the next node.
- BFS is optimal for equal costs but memory-hungry; A* uses $f = g + h$ and is optimal with an admissible heuristic.
- Minimax assumes a best-playing opponent; alpha–beta pruning gives the same answer with less work.
- Genetic algorithms evolve a population through selection, crossover and mutation.

Chapter 10

Knowledge and Reasoning Under Uncertainty

Search needs a model of the world. Where does the model come from, and what do we do when the information in it is incomplete or wrong? This chapter covers how a robot can store knowledge, reason from it with logic, and, crucially, reason when it is not certain, which is its normal condition.

10.1 Representing knowledge

A knowledge-based agent keeps a **knowledge base** (KB), a collection of sentences in some formal language, together with an **inference** procedure that derives new sentences. The pleasant feature is that we tell the agent facts and rules, and it works out their consequences. The common representations are:

- **Propositional logic**: sentences built from true/false statements with $\wedge$ (and), $\vee$ (or), $\neg$ (not), $\Rightarrow$ (implies), $\Leftrightarrow$ (if and only if).
- **First-order logic**: adds objects, relations and quantifiers: "for all" $\forall$ and "there exists" $\exists$.
- **Production rules**: IF–THEN rules, popular in expert systems and easy for people to read.
- **Semantic networks and frames**: graphs of concepts and relations; ancestors of today's knowledge graphs.
- **Ontologies**: shared vocabularies that let different robots and programs agree on what "cup" or "table" means.

10.2 Propositional logic in practice

Let us write a small KB for a robot in a warehouse. Use the symbols D: "door is open", P: "a person is in the aisle", S: "safe to move", B: "battery is above the threshold". The rules are:

$$R_1: \; P \Rightarrow \neg S \qquad \text{(never move with a person in the aisle)}$$
$$R_2: \; (D \wedge \neg P \wedge B) \Rightarrow S \qquad \text{(open door, no person, enough battery)}$$
$$R_3: \; \neg B \Rightarrow \text{GoCharge}.$$

Facts: D and B and $\neg P$. From R_2 we conclude S. If the sensors later report P, then by R_1 we get $\neg S$ and the robot stops. The logic is plain, but it demonstrates something important: with a small set of rules we get behaviour that responds to combinations of facts, and it is easy to audit.

10.2.1 Inference rules

The most useful rule is **modus ponens**: from A and $A \Rightarrow B$ infer B. **Modus tollens** is its mirror: from $\neg B$ and $A \Rightarrow B$ infer $\neg A$. Another is **resolution**, which is complete for propositional logic when the KB is written in conjunctive normal form: from $(A \vee B)$ and $(\neg B \vee C)$ infer $(A \vee C)$.

10.2.2 Forward and backward chaining

When the KB is a set of rules of the form "conditions $\Rightarrow$ conclusion", two simple strategies work.

- **Forward chaining** starts from known facts, fires every rule whose conditions are all satisfied, adds the conclusions and repeats until nothing new is derived. It is *data-driven* and suits monitoring.
- **Backward chaining** starts from a goal and looks for rules that conclude it, then tries to prove their conditions in turn. It is *goal-driven* and suits diagnosis: "why is the arm not moving?".

Worked example: Diagnosing a stalled arm

Rules: (1) motor current high and no movement $\Rightarrow$ mechanical jam. (2) motor current zero $\Rightarrow$ electrical fault. (3) electrical fault and fuse blown $\Rightarrow$ replace fuse. (4) mechanical jam $\Rightarrow$ check the gears.
Observations: current is high, arm does not move. Forward chaining: rule 1 fires $\rightarrow$ mechanical jam; rule 4 fires $\rightarrow$ check the gears. Rules 2 and 3 never fire. A backward chain for the goal "replace fuse" asks for "electrical fault" and "fuse blown", then for "current zero"; the current is high, so the goal fails and the fuse is not the problem.

10.3 The trouble with certainty

Logic deals with true and false. Sensors deal with "probably". The sonar says there is an obstacle 0.5 m ahead. Is there? Most likely, but a ghost echo off a side wall is possible. If our agent demanded certainty, it could never act. The tool for decisions with partial information is probability theory.

10.4 Probability in one page

For an event A, $P(A)$ is a number in $[0,1]$. The **conditional probability** $P(A \mid B)$ is the probability of A given that B is known to be true:

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}. \tag{10.1}$$

Two events are **independent** if $P(A \wedge B) = P(A)P(B)$. The **law of total probability** says that if $B_1, \dots, B_k$ are exclusive and cover all possibilities, then $P(A) = \sum_i P(A \mid B_i)P(B_i)$.

10.5 Bayes' rule

Combining the definition of conditional probability in both directions gives the engine of probabilistic reasoning:

$$P(H \mid E) = \frac{P(E \mid H)\,P(H)}{P(E)}, \qquad P(E) = P(E \mid H)P(H) + P(E \mid \neg H)P(\neg H). \tag{10.2}$$

Read it as a recipe for updating belief. $P(H)$ is the **prior**, what we believed before seeing evidence E. $P(E \mid H)$ is the **likelihood**, how probable the evidence is if the hypothesis is true. $P(H \mid E)$ is the **posterior**, our belief after the evidence.

Worked example: Is something really ahead?

A corridor has a protruding obstacle only 10% of the time: $P(O) = 0.1$. The sonar reports an echo with probability 0.9 when an obstacle is present (hit rate) and with probability 0.2 when nothing is there (false alarm rate). The sonar reports an echo. What is the probability that an obstacle is present?
$P(\text{echo}) = 0.9(0.1) + 0.2(0.9) = 0.09 + 0.18 = 0.27.$
$P(O \mid \text{echo}) = \dfrac{0.9 \times 0.1}{0.27} = 0.333.$
Despite a 90% sensitive sensor, one echo only makes the obstacle 33% likely, because obstacles are rare and false alarms from the large number of empty cases outweigh the true ones. This is the base-rate effect, and it surprises almost everyone the first time.

Repeated evidence. If echoes are independent from reading to reading, the posterior after one echo becomes the prior for the next. Applying the update again with $P(O) = 0.333$:

$$P(\text{echo}) = 0.9(0.333) + 0.2(0.667) = 0.433, \qquad P(O \mid \text{echo}) = \frac{0.3}{0.433} = 0.692.$$

A third echo raises it to 0.910, a fourth to 0.979, a fifth to 0.995 (Figure 10.1). Each new observation strengthens belief, and a sequence of weak, cheap observations can make a decision confident.

10.5.1 A neat way to see it: odds form

Many people find the update easier in odds. The prior odds are $P(H)/P(\neg H)$ and each piece of evidence multiplies the odds by the **likelihood ratio** $P(E \mid H)/P(E \mid \neg H)$. In our example

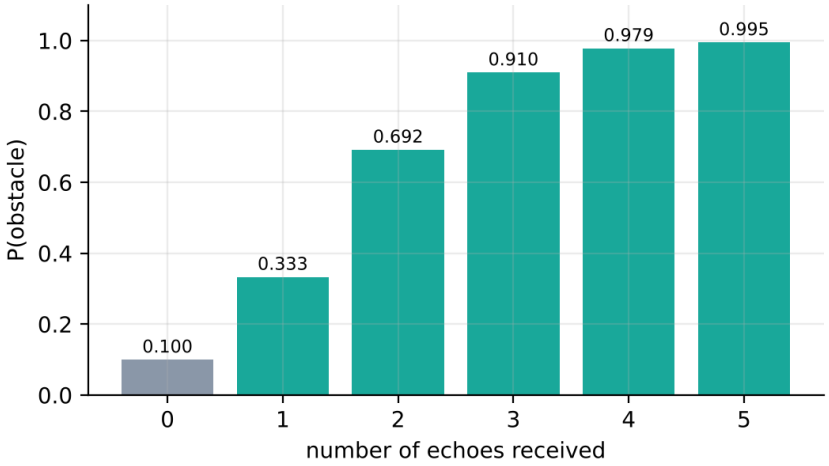


Figure 10.1: Belief that an obstacle is present after $0, 1, \ldots, 5$ consecutive echoes.

the ratio is $0.9/0.2 = 4.5$. Starting at odds $0.1/0.9 = 0.111$: after one echo 0.5 (probability 0.333), after two 2.25 (0.692), after three 10.1 (0.910). Each echo multiplies the odds by 4.5. The calculation is a one-line multiplication.

Derivation: Bayes' rule and its odds form, step by step

The rule. The probability that both H and E occur can be written in two ways: $P(H \wedge E) = P(H \mid E)P(E)$ and $P(H \wedge E) = P(E \mid H)P(H)$. Equate them and divide by $P(E)$:

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}.$$

The denominator comes from the law of total probability: evidence occurs either with H true or with H false, so $P(E) = P(E \mid H)P(H) + P(E \mid \neg H)P(\neg H)$.
Counting picture for the sonar. Imagine 1000 corridor trips. In 100 of them there is an obstacle, and 90 of those give an echo. In the other 900 nothing is there, yet 20% of them, that is 180, give an echo. Of the $90 + 180 = 270$ trips with an echo, 90 have a real obstacle: $90/270 = 1/3$. The rule has just formalised this count.
Odds form. Write the rule for H and for $\neg H$ and divide one by the other. The common $P(E)$ cancels:

$$\frac{P(H \mid E)}{P(\neg H \mid E)} = \frac{P(E \mid H)}{P(E \mid \neg H)} \times \frac{P(H)}{P(\neg H)}.$$

So the posterior odds equal the likelihood ratio times the prior odds. With prior odds $0.1/0.9 = 0.111$ and ratio 4.5: $0.111 \to 0.500 \to 2.25 \to 10.1$. To convert odds back, $P = \text{odds}/(1 + \text{odds})$: $2.25/3.25 = 0.692$ and $10.1/11.1 = 0.910$, which agree with the repeated-update numbers above.

10.6 Localisation with a discrete Bayes filter

A robot in a corridor with doors wants to know where it is. Divide the corridor into n cells. Keep a probability $b(i)$ for each cell; at the start the robot has no idea, so $b(i) = 1/n$ for all i. Two steps then repeat.

Measurement update. The robot senses "door". For each cell, multiply $b(i)$ by the probability of sensing a door there (high if cell i really has a door, low otherwise) and normalise so that the beliefs sum to 1.

Motion update. The robot moves one cell right. Shift the beliefs one cell, and blur them slightly because motion is imperfect: a fraction stays, a fraction jumps too far.

Worked example: Five cells

A corridor has five cells; doors at cells 1 and 4 (cells numbered from 0). Sensor: $P(\text{door seen} \mid \text{door}) = 0.8$, $P(\text{door seen} \mid \text{wall}) = 0.2$. Start with $b = (0.2, 0.2, 0.2, 0.2, 0.2)$. The robot senses a door.

Cells 1 and 4 are doors, so they are multiplied by 0.8 and the others by 0.2, giving $(0.04, 0.16, 0.04, 0.04, 0.16)$ for cells 0 to 4. The sum is 0.44, and after normalising we get $(0.091, 0.364, 0.091, 0.091, 0.364)$.

The belief now has two peaks, at the two doors. The robot does not know which door it is facing. It moves one cell right (assume perfect motion). Beliefs shift: cell $0 \to 1$, $1 \to 2$, $2 \to 3$, $3 \to 4$, $4 \to$ outside (or wraps). The corridor ends, so the mass at the last cell leaves; renormalising what remains: $(0, 0.091, 0.364, 0.091, 0.091)$ sums to 0.637; it becomes $(0, 0.143, 0.571, 0.143, 0.143)$. It now senses a wall, which has probability 0.8 at wall cells and 0.2 at door cells. Multiplying: $(0,\ 0.143 \times 0.2,\ 0.571 \times 0.8,\ 0.143 \times 0.8,\ 0.143 \times 0.2) = (0,\ 0.0286,\ 0.457,\ 0.114,\ 0.0286)$, sum 0.629; normalised $(0,\ 0.045,\ 0.727,\ 0.182,\ 0.045)$. The robot is now 73% sure it is in cell 2, the one right after the first door. Two noisy observations and one motion localised it.

Derivation: the two steps of the Bayes filter

Measurement update. By Bayes' rule, the belief in cell i after sensing z is $b'(i) = P(z \mid i)\, b(i) / P(z)$. The denominator is the same for every cell, so we compute the products $P(z \mid i) b(i)$ and divide each by their total. In the example the total is $2(0.2 \times 0.8) + 3(0.2 \times 0.2) = 0.32 + 0.12 = 0.44$, which is exactly $P(\text{door seen})$, as the law of total probability says.

Motion update. The robot ends in cell i if it started in some cell j and the motion took it to i. Summing over the possible starting cells (total probability again): $b'(i) = \sum_j P(i \mid j, u)\, b(j)$. For imperfect motion suppose the command "one cell right" moves the robot exactly one cell with probability 0.8, leaves it in place with 0.1 and moves it two cells with 0.1. If the robot is certainly in cell 2 before the move, then afterwards $b(3) = 0.8$, $b(2) = 0.1$, $b(4) = 0.1$. A sharp belief has become blurred. Motion always spreads belief, and measurement always sharpens it, which is why the two steps are alternated.

This discrete Bayes filter is the grid version of the Kalman filter of Chapter 4: both predict from a motion model and update from a measurement. The Kalman filter assumes Gaussian beliefs and linear models and is very efficient; the grid or particle versions can represent any belief, including the two-peaked ones above.

10.7 Deciding with probabilities: expected utility

Once beliefs are probabilities, the agent still has to act. The rational rule is to choose the action with the highest **expected utility**,

$$EU(a) = \sum_{s'} P(s' \mid a)\, U(s'). \tag{10.3}$$

Worked example: Go around or push through?

A robot must cross a doorway. Option A (go through): with probability 0.9 it succeeds and gets utility $+10$; with probability 0.1 it jams, utility -20. Option B (go around): sure success but with a long detour, utility $+6$.
$EU(A) = 0.9(10) + 0.1(-20) = 9 - 2 = 7$; $EU(B) = 6$. Choose A. But if the jam probability were 0.2, $EU(A) = 8 - 4 = 4 < 6$ and B wins. The decision flips at a jam probability p where $10(1-p) - 20p = 6$, i.e. $10 - 30p = 6$, $p = 0.133$.

10.8 Putting probabilities to work

There is a family of more elaborate models built from these pieces: Bayesian networks, which draw the dependence between many variables as a graph; hidden Markov models for sequences; Markov decision processes (MDPs) for planning when actions have random results. You do not need them today, but they are built by the very steps in this chapter, a prior, a likelihood, an update, an expected utility. If you grasp the odds form of Bayes' rule you already hold the key.

10.9 Chapter summary

- Knowledge bases store facts and rules; modus ponens, forward chaining and backward chaining derive conclusions.
- Sensors are uncertain, so robots reason with probabilities.
- Bayes' rule updates belief: posterior $\propto$ likelihood $\times$ prior; in odds form each observation multiplies the odds by a likelihood ratio.
- A Bayes filter alternates a motion update and a measurement update; the Kalman filter is its Gaussian special case.
- Choose the action with maximum expected utility.

Chapter 11

Putting It Together: A Mobile Robot

Every chapter so far looked at one organ. Now we assemble a complete animal. I have chosen a small wheeled robot because it is cheap, because it exercises every idea in the book, and because you can build one on a table. We will model how it moves, work out how it knows where it has been, size its motors and sensors with real numbers, and decide how to organise its software.

11.1 Kinematics of a differential-drive robot

Two powered wheels on a common axis, plus a caster for balance, make a **differential-drive** robot, the layout of most educational and indoor robots. The two wheels can run at different speeds. Let v_L and v_R be the ground speeds of the left and right wheel contact points, and b the distance between the wheels (the track).

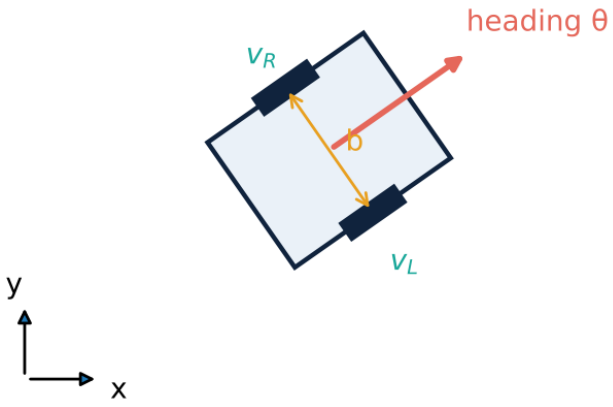


Figure 11.1: Geometry of a differential-drive robot. The body has heading θ; the wheels are a distance b apart.

The centre of the axle moves at the average of the two wheel speeds and the body turns at the

difference divided by the track:

$$v = \frac{v_R + v_L}{2}, \qquad \omega = \frac{v_R - v_L}{b}. \tag{11.1}$$

So equal speeds give straight motion, opposite speeds give a spin on the spot ($v = 0$), and anything else gives an arc. The pose (x, y, θ) then changes as

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega. \tag{11.2}$$

These three equations are the whole motion model. Notice a limitation: the robot cannot move sideways. It is a **non-holonomic** machine, which means it has fewer directly controllable motions (two: v and ω) than coordinates in its pose (three). To park next to a kerb a car, or this robot, needs to manoeuvre: forward, back, forward.

Worked example: An arc

$b = 0.2$ m, $v_L = 0.30$ m/s, $v_R = 0.40$ m/s. $v = 0.35$ m/s, $\omega = (0.40 - 0.30)/0.2 = 0.5$ rad/s. The radius of the circle followed by the axle centre is $\rho = v/\omega = 0.7$ m. One complete circle takes $2\pi/\omega = 12.6$ s. In general $\rho = \dfrac{b}{2}\cdot\dfrac{v_R + v_L}{v_R - v_L}$.

Derivation: the differential-drive equations from rigid-body motion

Treat the robot body as a rigid object. Its centre moves with speed v along the heading and the body spins with angular rate ω. A point at distance $b/2$ to the right of the centre has, in addition to v, a sideways-turning velocity $\omega\, b/2$ along the heading direction, and the left wheel has the opposite contribution:

$$v_R = v + \omega\frac{b}{2}, \qquad v_L = v - \omega\frac{b}{2}.$$

Add the two equations: $v_R + v_L = 2v$, so $v = (v_R + v_L)/2$. Subtract them: $v_R - v_L = \omega b$, so $\omega = (v_R - v_L)/b$. The same two equations used in the other direction convert a command (v, ω) to wheel speeds, which is what the path follower does later.
The centre moves with speed v in the direction θ, so the components of velocity along the fixed axes are $\dot{x} = v\cos\theta$ and $\dot{y} = v\sin\theta$, and the heading changes at $\dot{\theta} = \omega$. For motion on a circle, speed equals radius times angular rate, $v = \rho\omega$, so $\rho = v/\omega = \dfrac{(v_R + v_L)/2}{(v_R - v_L)/b} = \dfrac{b}{2}\cdot\dfrac{v_R + v_L}{v_R - v_L}$. Check with the example: $\frac{0.2}{2}\cdot\frac{0.70}{0.10} = 0.7$ m.

11.2 Odometry: where have I been?

With encoders on the wheels we can integrate the motion model. In a short interval the encoders return counts n_L and n_R. If there are C counts per wheel revolution and the wheel radius is r, each wheel's travel is

$$\Delta s_{L,R} = \frac{2\pi r}{C}\, n_{L,R}. \tag{11.3}$$

Then

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \qquad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{b}, \tag{11.4}$$

and the pose is updated by

$$x \leftarrow x + \Delta s \cos\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad y \leftarrow y + \Delta s \sin\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad \theta \leftarrow \theta + \Delta\theta. \tag{11.5}$$

Using the mid-point heading $\theta + \Delta\theta/2$ makes the straight-line approximation to the arc more accurate.

Derivation: distance per count, and why the mid-point heading

One wheel revolution is C counts and covers one circumference $2\pi r$, so one count is $2\pi r/C$ of travel, and n counts give $\Delta s = 2\pi r n/C$. For the example: $2\pi(0.05)/960 = 0.000327$ m per count, and 1920 counts give 0.628 m.
Now suppose the robot drives a true circular arc of length Δs while turning by $\Delta\theta$. The radius is $\rho = \Delta s/\Delta\theta$. The straight line joining the start and end points (the chord) has length $2\rho\sin(\Delta\theta/2)$ and points exactly halfway between the starting and final headings, that is along $\theta + \Delta\theta/2$. So the exact update is a move of

$$2\rho \sin\frac{\Delta\theta}{2} = \Delta s \,\frac{\sin(\Delta\theta/2)}{\Delta\theta/2}$$

along the mid-point heading. The fraction $\sin(u)/u$ is close to 1 for small angles: $1 - u^2/6$ approximately. For the example $u = \pi/4$ and $\sin u/u = 0.9003$, giving a chord $0.785 \times 0.9003 = 0.707$ m, and $x = 0.707\cos 45^\circ = 0.5$ m, the exact answer. For a 10 ms update turning at 0.5 rad/s, $u = 0.0025$ rad and the factor differs from 1 by only 10^{-6}, which is why the simple formula is used.

Worked example: One odometry update

Wheel radius $r = 0.05$ m, $C = 960$ counts per revolution, $b = 0.2$ m. In one interval, $n_L = 1920$ and $n_R = 2880$ counts.
$\Delta s_L = 2\pi(0.05)(1920)/960 = 0.628$ m, $\Delta s_R = 2\pi(0.05)(2880)/960 = 0.942$ m.
$\Delta s = 0.785$ m, $\Delta\theta = (0.942 - 0.628)/0.2 = 1.571$ rad $= 90^\circ$. The robot drove a quarter circle of radius $\Delta s/\Delta\theta = 0.5$ m. (The mid-point formula applied to such a large turn in one step lands at $(0.555, 0.555)$ m while the exact arc ends at $(0.5, 0.5)$ m. Real controllers update every few milliseconds, so each turn is tiny and the approximation is excellent.)

Why odometry drifts. Every update is a little wrong: wheels slip, the wheel radius is not exactly r, the track is not exactly b, and counts are quantised. These errors *accumulate*. Worse, a small heading error turns into a large position error later, because the robot keeps driving along the wrong direction. Figure 11.2 shows the effect with only 3% random error in each wheel speed: six runs of the same "straight line" command end in different places, spread more and more with distance.

Derivation: how fast the drift grows

A fixed heading error. If the heading is wrong by δ and the robot drives a distance d, the sideways error is $d\sin\delta \approx d\,\delta$ for a small angle. A mere 1° (0.0175 rad) error over 10 m gives 0.17 m.
Random heading errors accumulate. Let each of N steps of length Δ add an independent heading error of standard deviation σ. After k steps the heading error has variance $k\sigma^2$

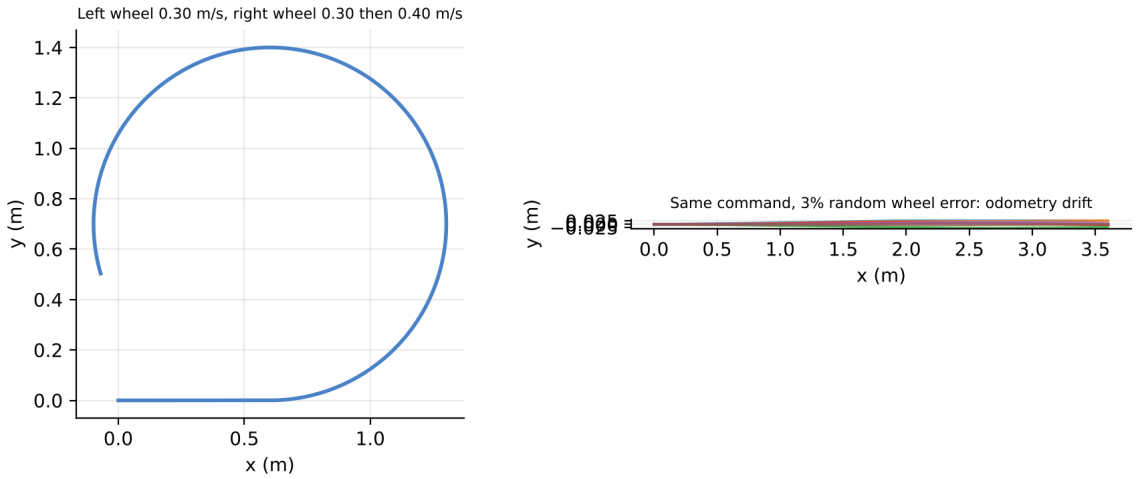


Figure 11.2: Left: a path with a speed change. Right: six simulated runs of the same straight command with 3% random wheel error show the drift of dead reckoning.

(variances of independent errors add). The sideways error is the sum of Δ times the heading error at each step, whose variance is

$$\Delta^2\sigma^2 \sum_{k=1}^{N}\sum_{l=1}^{N} \min(k,l) = \Delta^2\sigma^2\,\frac{N(N+1)(2N+1)}{6} \approx \Delta^2\sigma^2\frac{N^3}{3}.$$

With the total distance $d = N\Delta$ the standard deviation is $\sigma_y \approx d\,\sigma\sqrt{N/3}$. It grows faster than the distance. Example: steps of 1 cm, $\sigma = 0.1^\circ = 0.00175$ rad, $d = 10$ m so $N = 1000$: $\sigma_y = 10 \times 0.00175 \times \sqrt{333} = 0.32$ m. Ten metres of travel with a tiny 0.1° error per step leaves a third of a metre of uncertainty, and doubling the distance multiplies it by $2^{1.5} = 2.8$.

This is why serious robots never trust odometry alone. They correct it with absolute information (a LiDAR scan matched against a map, a beacon, a camera), usually through the filters of Chapters 4 and 10.

11.3 Sizing a delivery robot, step by step

Let me walk through a complete design calculation. The task: a small robot carrying a tray at up to 1 m/s along a corridor that includes a gentle ramp. This is the kind of reasoning I would like you to be able to do on a blank sheet.

Requirements. Total mass $m = 5$ kg. Top speed $v_{\max} = 1$ m/s. Acceleration $a = 0.5$ m/s^2. Ramp slope 5°. Rolling resistance coefficient $C_{rr} \approx 0.02$. Two driven wheels of radius $r = 0.05$ m.

Step 1: force at the wheels.

$$F = m\big(a + g\sin 5^\circ + C_{rr}\,g\big) = 5\,(0.5 + 9.81 \times 0.0872 + 0.02 \times 9.81)$$

$$= 5\,(0.5 + 0.855 + 0.196) = 7.76\ \text{N}.$$

Each wheel carries half: 3.88 N. Wheel torque $\tau_w = 3.88 \times 0.05 = 0.194$ N m.

Step 2: wheel speed. $\omega_w = v/r = 1/0.05 = 20$ rad/s $= 191$ rpm.

Step 3: choose a motor and gear ratio. Take a small motor rated 0.03 N m continuous at 3000 rpm (314 rad/s). A reduction $N = 15$ gives a wheel speed $3000/15 = 200$ rpm, just above the need. With efficiency $\eta = 0.8$ the wheel torque is $0.03 \times 15 \times 0.8 = 0.36$ N m, about 1.9 times the requirement of 0.194 N m: a healthy margin for friction, battery sag and a bit of extra payload.

Step 4: choose the encoder. The motor carries a magnetic encoder with 16 pulses per revolution; with quadrature decoding this is 64 counts per motor revolution. After the 15:1 gearbox, $C = 64 \times 15 = 960$ counts per wheel revolution. Each count is $2\pi(0.05)/960 = 0.327$ mm of travel. At 1 m/s the encoder produces $1/0.000327 \approx 3056$ counts per second per wheel, which is about 30 counts in a 10 ms control period. That is enough to estimate the speed to about 3% per sample, so a 100 Hz speed loop will work.

Step 5: obstacle sensing and stopping. Braking at 1 m/s^2 from 1 m/s needs a distance $v^2/(2a_b) = 0.5$ m. The software takes about 0.1 s from measurement to brake command, during which the robot covers another 0.1 m. So 0.6 m is the minimum detection distance, and a safety factor of 2 suggests the sonar should reliably see obstacles at 1.2 m. A module with a range of several metres is comfortable.

Derivation: checking each step of the design

Step 1. Along a slope of angle α the wheels must overcome three things: the part of the weight along the slope, $mg\sin\alpha$; rolling resistance, $C_{rr}\,mg\cos\alpha$; and the push needed for acceleration, ma. Newton's law gives $F = m(a + g\sin\alpha + C_{rr}\,g\cos\alpha)$. We replaced $\cos 5^\circ = 0.996$ by 1, an error of 0.4% in a term that is itself small. The numbers: $\sin 5^\circ = 0.0872$, $9.81 \times 0.0872 = 0.855$, $0.02 \times 9.81 = 0.196$, sum $0.5 + 0.855 + 0.196 = 1.551$, times 5 kg gives 7.76 N. A force F at radius r is a torque Fr, so two wheels each need $3.88 \times 0.05 = 0.194$ N m.
Steps 2 and 3. A wheel of radius r rolling at speed v turns at $\omega_w = v/r$; $1/0.05 = 20$ rad/s and $20 \times 60/(2\pi) = 191$ rpm. A gearbox of ratio 15 divides motor speed and multiplies torque (Chapter 6): $3000/15 = 200$ rpm, and $0.03 \times 15 \times 0.8 = 0.36$ N m. The margin is $0.36/0.194 = 1.86$.
Step 4. At 1 m/s each wheel travels 1000 mm per second, and at 0.327 mm per count that is $1000/0.327 = 3058$ counts/s, so 30.6 counts in 10 ms. Counting is accurate to ± 1 count, so the speed error is $1/30.6 = 3.3\%$.
Step 5. Braking with a constant deceleration a_b from speed v obeys $v^2 = 2a_b d$, so $d = v^2/(2a_b) = 1/(2 \times 1) = 0.5$ m. During the 0.1 s of software delay the robot still moves at full speed: $1 \times 0.1 = 0.1$ m. The total is 0.6 m, and doubling for safety gives 1.2 m.

The idea to keep

The path from a requirement to a purchase order is: forces → torque → speed → gear ratio → motor → resolution → sample rate → sensor range. Each step is a line of arithmetic you already know. Do it on paper before ordering parts.

11.4 Software architecture: layers of behaviour

A robot has several jobs at different time scales, and mixing them in one loop is the road to a mess. A widely used layout stacks three levels.

1. **Reactive (milliseconds).** Motor control loops and safety reflexes: stop if the bumper is hit, limit the current, keep wheel speed at the commanded value. This layer must always work, even if everything above it crashes.
2. **Navigation (tenths of seconds).** Localisation, obstacle avoidance and path following. A* or another planner produces a path; a follower converts it into v and ω.
3. **Mission (seconds to minutes).** Decides what the robot is doing: pick up a tray, deliver to room 4, return to charge.

Higher layers give goals; lower layers keep the robot alive. A useful rule is that a lower layer may override a higher one (the bumper beats the mission planner), but never the reverse. When several behaviours want control at once, an *arbiter* chooses, by fixed priority or by blending commands.

Worked example: Behaviour priorities

Three behaviours: AVOID (steer away if a sonar reading is below 0.4 m), GO_TO (steer toward the next waypoint), CHARGE (head for the dock when the battery is low). Priorities: AVOID > CHARGE > GO_TO. At every cycle the highest-priority behaviour that is active supplies (v, ω). With a clear path and a full battery, GO_TO drives. When the battery falls, CHARGE takes over, until AVOID fires for a chair in the way, then the robot steers around it and returns to CHARGE. No behaviour needs to know about the others; the priorities do the work. It is exactly the structure of the three laws in Chapter 1.

11.5 A control loop for following a line or a path

A simple and effective path follower uses the heading error. If the desired heading is θ_d and the actual heading is θ, command

$$\omega = K_\theta(\theta_d - \theta), \qquad v = v_0\,(1 - \min(1, |\theta_d - \theta| / \theta_{\max})), \tag{11.6}$$

which is a proportional controller on angle, combined with slowing down when the robot is badly misaligned. Then convert to wheel speeds by $v_R = v + \frac{b}{2}\omega$ and $v_L = v - \frac{b}{2}\omega$. This tiny controller, together with a sonar-based avoid behaviour, is already enough to navigate many corridors.

11.6 Testing: simulate, then trust

Before the robot leaves the bench, run its software in simulation. The code in Appendix B includes a differential-drive simulator that you can extend with a controller and noisy sensors. A discipline I recommend: log everything (commands, sensor values, estimated pose), plot it after each run, and look for the first moment the plot disagrees with your expectation. That moment is nearly always where the bug is.

11.7 Chapter summary

- A differential-drive robot has $v = (v_R + v_L)/2$ and $\omega = (v_R - v_L)/b$, and cannot move sideways.
- Odometry integrates encoder counts into pose but drifts without bound; correct it with absolute sensing.
- A design flows from requirement to force, torque, gear ratio, encoder resolution and sensor range.
- Layered software with priority arbitration keeps a robot safe and understandable.

Chapter 12

Practice Problems with Worked Solutions

This is the part of the book I would like you to use the most. The first fifteen problems are the homework set from the lecture; I have worked each one in full. The rest are extra practice at the same level. Try every problem before you read the solution. If your answer differs, do not just copy the correct one: find the line where you and the solution diverged. Constants used: $g = 9.81$ m/s^2, speed of sound 343 m/s.

12.1 The lecture homework

Problem 1 (kinematics). A two-link arm has $l_1 = 0.5$ m, $l_2 = 0.3$ m, $\theta_1 = 30^\circ$, $\theta_2 = 45^\circ$. Find the tip position.

Solution. $x = 0.5\cos 30^\circ + 0.3\cos 75^\circ = 0.4330 + 0.0776 = 0.511$ m; $y = 0.5\sin 30^\circ + 0.3\sin 75^\circ = 0.2500 + 0.2898 = 0.540$ m. Tip at $(0.511,\ 0.540)$ m.

Problem 2 (workspace). Can the same arm reach a point 0.9 m from its base? State its maximum and minimum reach.

Solution. Maximum reach $l_1 + l_2 = 0.8$ m, minimum $|l_1 - l_2| = 0.2$ m. A point at 0.9 m is beyond 0.8 m, so it cannot be reached. (If you tried the inverse kinematics anyway, $\cos\theta_2$ would come out larger than 1.)

Problem 3 (encoder). A disc has 500 slots and all four edges are decoded. What is the resolution per count and the angle for 3500 counts?

Solution. Counts per turn $= 500 \times 4 = 2000$. Resolution $360^\circ/2000 = 0.18^\circ$. $3500 \times 0.18^\circ = 630^\circ = 1.75$ turns.

Problem 4 (ultrasonic). An echo returns after 11.7 ms. How far is the object?

Solution. $d = vt/2 = 343 \times 0.0117/2 = 2.0$ m (the factor 2 because the pulse goes there and back).

Problem 5 (ADC). A 10-bit ADC has $V_{\text{ref}} = 3.3$ V. Find the LSB and the code for 1.65 V.

Solution. LSB $= 3.3/1024 = 3.223$ mV. Code $= 1.65/0.003223 = 512$.

Problem 6 (control). A plant of gain 1 is under P control with $K_p = 4$. Find the final value for a unit step.

Solution. Closed-loop gain $K_p/(1 + K_p) = 4/5 = 0.8$. The final value is 0.8 and the steady-state error is 0.2. An integral term would remove it.

Problem 7 (gearbox). A motor gives 3000 rpm and 0.1 N m. A gearbox with ratio 50 and efficiency 0.8 follows it. Find output speed and torque.

Solution. Speed $3000/50 = 60$ rpm. Torque $0.1 \times 50 \times 0.8 = 4.0$ N m. Check by power: in, $0.1 \times 314 = 31.4$ W; out, $4.0 \times 6.28 = 25.1$ W, and $25.1/31.4 = 0.8$, the efficiency.

Problem 8 (hydraulics). $A_1 = 4$ cm^2, $A_2 = 100$ cm^2, $F_1 = 50$ N, pump stroke 50 mm. Find F_2 and d_2.

Solution. $F_2 = 50 \times 100/4 = 1250$ N. $d_2 = 50 \times 4/100 = 2$ mm. Check work: $50 \times 0.05 = 2.5$ J in, $1250 \times 0.002 = 2.5$ J out.

Problem 9 (gripper). A 2 kg part, $\mu = 0.4$, two fingers, safety factor 2. Find the normal force per finger.

Solution. Required friction $= 2 \times 2 \times 9.81 = 39.24$ N shared by two fingers: $2\mu N = 39.24 \Rightarrow N = 39.24/0.8 = 49.05$ N each.

Problem 10 (neuron). $\mathbf{x} = (2, -1)$, $\mathbf{w} = (0.5, 1.2)$, $b = 0.3$. Find z and the sigmoid output.

Solution. $z = 1.0 - 1.2 + 0.3 = 0.1$; $\sigma(0.1) = 1/(1 + e^{-0.1}) = 0.525$.

Problem 11 (A*). On a four-connected grid find h from $(2, 3)$ to $(7, 9)$ with the Manhattan heuristic, and f if $g = 4$.

Solution. $h = 5 + 6 = 11$. $f = g + h = 4 + 11 = 15$.

Problem 12 (design). Why do robots use a gear reducer rather than a bigger motor? Give three reasons.

Solution. (i) A small, light, fast motor with a reducer delivers the same output power as a large slow motor at lower weight and cost. (ii) Reflected inertia of the load falls by N^2, so the motor accelerates the load easily and the control is less sensitive to load changes. (iii) The encoder on the motor shaft effectively has N times more resolution at the joint. Other acceptable answers: the motor runs near its most efficient speed; a self-locking gearbox can hold a load without power.

Problem 13 (alpha–beta). MAX root, two MIN children with leaves $[3, 5, 2]$ and $[4, 1, 6]$. Find the root value and the number of pruned leaves.

Solution. Left MIN node $= \min(3, 5, 2) = 2$. In the right node, after the leaf 4 the bound is ≤ 4, and after the leaf 1 it is $\leq 1 < 2$. MAX will never prefer this branch, so the leaf 6 is pruned. Root $= \max(2, \leq 1) = 2$. One leaf is pruned.

Problem 14 (Bayes). Prior $P(O) = 0.1$, hit rate 0.9, false alarm rate 0.2. Find $P(O \mid \text{echo})$.

Solution. $P(\text{echo}) = 0.9 \times 0.1 + 0.2 \times 0.9 = 0.27$. $P(O \mid \text{echo}) = 0.09/0.27 = 0.333$.

Problem 15 (genetic algorithm). Decode 10110010 to $x \in [0, 4]$ with 8-bit scaling.

Solution. $10110010_2 = 178$. $x = 178/255 \times 4 = 2.79$.

12.2 Extra practice

Problem 16 (inverse kinematics). For the same arm find both joint solutions for the target $(0.6,\ 0.2)$ m.

Solution. $r^2 = 0.36 + 0.04 = 0.40$. $\cos\theta_2 = (0.40 - 0.25 - 0.09)/0.30 = 0.20$, so $\theta_2 = \pm 78.46^\circ$. The target direction is $\text{atan2}(0.2, 0.6) = 18.43^\circ$. For $\theta_2 = +78.46^\circ$ the correction is $\text{atan2}(0.3 \sin 78.46^\circ,\ 0.5 + 0.3 \times 0.2) = \text{atan2}(0.2939,\ 0.56) = 27.69^\circ$, so $\theta_1 = 18.43 - 27.69 = -9.26^\circ$. For $\theta_2 = -78.46^\circ$, $\theta_1 = 18.43 + 27.69 = 46.13^\circ$. Check the second solution: $x = 0.5 \cos 46.13^\circ + 0.3 \cos(-32.33^\circ) = 0.3469 + 0.2534 = 0.600$; $y = 0.5 \sin 46.13^\circ + 0.3 \sin(-32.33^\circ) = 0.3600 - 0.1604 = 0.200$. Both solutions are valid.

Problem 17 (frames). A point has coordinates $(2, 0)$ in frame 1, which is rotated 90° from frame 0 about the same origin. Where is it in frame 0?

Solution. $R(90^\circ) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, so $\mathbf{p}^0 = (0 \cdot 2 - 1 \cdot 0,\ 1 \cdot 2 + 0 \cdot 0) = (0,\ 2)$. Interpretation: the x_1 axis points along y_0, so a point two units along x_1 is two units up in frame 0.

Problem 18 (Jacobian). At $\theta_1 = 0$, $\theta_2 = 90^\circ$ with the same arm, write J and find $\det J$. Is this pose singular?

Solution. $s_1 = 0, c_1 = 1, s_{12} = 1, c_{12} = 0$. $J = \begin{pmatrix} -0.3 & -0.3 \\ 0.5 & 0 \end{pmatrix}$, $\det J = (-0.3)(0) - (-0.3)(0.5) = 0.15$, which equals $l_1 l_2 \sin\theta_2 = 0.15$. It is not singular; in fact, $\sin\theta_2 = 1$ is the best-conditioned elbow angle.

Problem 19 (filter). An exponential filter with $\alpha = 0.2$ starts at $y_0 = 0$ and receives a constant input of 1. Find y_1, y_2, y_3 and the approximate time constant in samples.

Solution. $y_1 = 0.8(0) + 0.2(1) = 0.2$; $y_2 = 0.8(0.2) + 0.2 = 0.36$; $y_3 = 0.8(0.36) + 0.2 = 0.488$. The pattern is that the gap to 1 shrinks by the factor 0.8 at each step: $1 - y_k = 0.8^k$, so $y_k = 1 - 0.8^k$ (check: $1 - 0.8^3 = 1 - 0.512 = 0.488$). The time constant is about $1/\alpha = 5$ samples; after 5

samples $y_5 = 1 - 0.8^5 = 0.672$, close to the 63% of a true time constant. The exact value is $-1/\ln 0.8 = 4.48$ samples.

Problem 20 (second-order spec). A positioning loop must have no more than 5% overshoot and a 2% settling time of 0.5 s. Find the required ζ and ω_n.

Solution. $\%OS = 100e^{-\pi\zeta/\sqrt{1-\zeta^2}}$, so $0.05 = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$. Take the natural logarithm and write $L = -\ln 0.05 = 2.996$: $L = \pi\zeta/\sqrt{1-\zeta^2}$. Square both sides, $L^2(1-\zeta^2) = \pi^2\zeta^2$, collect ζ^2: $L^2 = \zeta^2(\pi^2 + L^2)$. Hence $\zeta = L/\sqrt{\pi^2 + L^2} = 2.996/\sqrt{9.870 + 8.976} = 2.996/4.341 = 0.690$. Settling $4/(\zeta\omega_n) \le 0.5 \Rightarrow \omega_n \ge 4/(0.690 \times 0.5) = 11.6$ rad/s.

Problem 21 (DC motor). A motor has $V = 24$ V, $K_t = K_e = 0.04$ N m/A, $R = 2\ \Omega$. Find stall torque, no-load speed and the speed at a load of 0.1 N m.

Solution. Stall torque $= K_t V/R = 0.04 \times 24/2 = 0.48$ N m. No-load speed $= V/K_e = 600$ rad/s $= 5730$ rpm. At 0.1 N m the current is $i = 0.1/0.04 = 2.5$ A; $\omega = (24 - 2 \times 2.5)/0.04 = 475$ rad/s $= 4536$ rpm.

Problem 22 (stepper). A 1.8° stepper is driven in half-step mode at 800 half-steps per second. Find the speed in rpm.

Solution. Half-stepping gives 400 steps per revolution. $800/400 = 2$ rev/s $= 120$ rpm.

Problem 23 (vacuum). A 5 kg box is lifted with acceleration 1 m/s^2 using cups of 50 mm diameter at a vacuum of 50 kPa below atmosphere. Safety factor 2. How many cups are needed?

Solution. Required force $= 2 \times 5 \times (9.81 + 1) = 108.1$ N. Area of one cup $= \pi(0.025)^2 = 1.963 \times 10^{-3}$ m^2; force $= 50\,000 \times 1.963 \times 10^{-3} = 98.2$ N. One cup is not enough ($98.2 < 108.1$); two cups give 196 N. Use 2 (or 3 if the surface leaks).

Problem 24 (gradient descent). Minimise $f(w) = (w-4)^2$ starting at $w = 0$ with learning rate 0.1. Give the first three iterates.

Solution. $f'(w) = 2(w-4)$. $w_1 = 0 - 0.1(-8) = 0.8$; $w_2 = 0.8 - 0.1(2)(0.8-4) = 0.8 + 0.64 = 1.44$; $w_3 = 1.44 + 0.512 = 1.952$. Each step shrinks the gap to 4 by the factor $1 - 2\eta = 0.8$ (because $w_{k+1} - 4 = (w_k - 4) - 0.2(w_k - 4)$). After k steps $w_k = 4(1 - 0.8^k)$; check $k = 3$: $4(1 - 0.512) = 1.952$.

Problem 25 (odometry). Wheel radius 0.04 m, 600 counts per revolution, track 0.16 m. Counts in one interval: left 300, right 450. Find Δs and $\Delta\theta$.

Solution. Distance per count $= 2\pi(0.04)/600 = 4.189 \times 10^{-4}$ m. $\Delta s_L = 0.1257$ m, $\Delta s_R = 0.1885$ m. $\Delta s = 0.1571$ m. $\Delta\theta = (0.1885 - 0.1257)/0.16 = 0.3927$ rad $= 22.5^\circ$.

Problem 26 (Kalman gain). Predicted variance $P^- = 0.20$, measurement variance $R = 0.05$. Find K and the updated variance.

Solution. $K = 0.20/0.25 = 0.8$. $P = (1 - 0.8)(0.20) = 0.04$. The estimate leans 80% on the measurement, which is more certain than the prediction.

Problem 27 (trajectory). A joint must turn 60° in 1.5 s with a cubic profile. Find the peak speed and the time needed if the joint can do at most $50^\circ/\text{s}$.

Solution. Peak speed $= \frac{3}{2}\Delta q/T = 1.5 \times 60/1.5 = 60^\circ/\text{s}$ (the derivation is in Chapter 3). This exceeds $50^\circ/\text{s}$, so $T \geq 1.5 \times 60/50 = 1.8$ s.

Problem 28 (logic). Rules: $A \Rightarrow B$, $B \Rightarrow C$, $(C \wedge D) \Rightarrow E$. Facts: A and D. Use forward chaining to list everything that can be derived.

Solution. Start with $\{A, D\}$. Rule 1 fires: add B. Rule 2 fires: add C. Now C and D are both known, so rule 3 fires: add E. Final set $\{A, B, C, D, E\}$.

12.3 Challenge projects

1. **Rebuild an animation.** Pick one of: the two-link arm with sliders for the angles, a PID loop with sliders for the gains, or an A* search on a grid you can draw. Write it in Python (with a plotting library) or JavaScript (with a canvas). It must let the user change one parameter live.

2. **Measure a real sensor.** Take an ultrasonic module, record 200 readings of a fixed target at 1 m, and compute the mean and standard deviation. Then try a 10-sample moving average. Does the standard deviation fall by $\sqrt{10}$? If not, why not?

3. **Tune by experiment.** Simulate the motor of Problem 21 with a P controller and a fixed load. Plot step responses for several K_p values and find the one where the response begins to oscillate.

Appendix A

A Maths Toolkit: What the Equations Assume

Every equation in this book uses a small set of mathematical tools. If a step ever looked like magic, come back to this appendix. It is the short course I would give a student who has forgotten school mathematics over a holiday, and each tool is followed by a small check you can do with a calculator.

A.1 Angles and trigonometry

Degrees and radians. A full turn is $360^\circ = 2\pi$ rad, so 1 rad $= 57.296^\circ$ and $\theta_{\text{rad}} = \theta_{\text{deg}} \times \pi/180$. Calculators and programs usually want radians. Speeds in rpm convert by 1 rpm $= 2\pi/60 =$ 0.1047 rad/s; for example 3000 rpm $=$ 314 rad/s.

Right triangle. For an angle θ in a right triangle, $\sin\theta$ = opposite/hypotenuse, $\cos\theta$ = adjacent/hypotenuse and $\tan\theta = \sin\theta/\cos\theta$. A link of length l at angle θ therefore reaches $l\cos\theta$ across and $l\sin\theta$ up. This one sentence is the basis of all arm kinematics.

Identities used in the book.

$$\begin{aligned} \sin^2\theta + \cos^2\theta &= 1, \\ \cos(a \pm b) &= \cos a\cos b \mp \sin a\sin b, \\ \sin(a \pm b) &= \sin a\cos b \pm \cos a\sin b. \end{aligned}$$

Check: $a = 60^\circ$, $b = 30^\circ$: $\cos(90^\circ) = 0.5(0.866) - 0.866(0.5) = 0$. ✓

The two-argument arctangent. $\arctan(y/x)$ cannot tell $(1,1)$ from $(-1,-1)$, because the ratio is the same. $\text{atan2}(y,x)$ looks at the signs of both and returns the correct angle in $(-180^\circ, 180^\circ]$. For the point $(-1,1)$ it gives 135°, while $\arctan(-1)$ gives -45°.

A.2 Vectors and matrices

Vectors. A vector such as $\mathbf{p} = (x,y)$ has a length $|\mathbf{p}| = \sqrt{x^2+y^2}$. Vectors add component by component, which is why the tip of an arm is the sum of its link vectors. The dot product is $\mathbf{a}\cdot\mathbf{b} = a_1b_1 + a_2b_2 = |\mathbf{a}||\mathbf{b}|\cos\varphi$.

Matrix times vector. Each entry of the result is a row of the matrix multiplied element by element with the vector, then added:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax + by \\ cx + dy \end{pmatrix}.$$

Matrix times matrix. Entry (i, j) is row i of the first matrix against column j of the second. Order matters. Take $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. Then

$$AB = \begin{pmatrix} 1\cdot 0 + 2\cdot 1 & 1\cdot 1 + 2\cdot 0 \\ 3\cdot 0 + 4\cdot 1 & 3\cdot 1 + 4\cdot 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}, \qquad BA = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}.$$

The two answers differ, which is the algebraic root of the fact that the order of rotations matters.

Transpose, determinant, inverse. The transpose A^T swaps rows and columns. For a 2×2 matrix the determinant is $ad - bc$ and the inverse exists when it is not zero:

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

For the matrix A above, $\det A = 1\cdot 4 - 2\cdot 3 = -2$ and $A^{-1} = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix}$. Check: the first row times the first column is $1(-2) + 2(1.5) = 1$, and the first row times the second column is $1(1) + 2(-0.5) = 0$. ✓

A.3 Calculus in five lines

The **derivative** $\dot{q} = dq/dt$ is the slope of the graph of q against t, that is the rate of change; for a position it is a speed. The **integral** $\int_0^t e\, d\tau$ is the area under the graph of e, that is an accumulated total. They undo each other.

- Power rule: $\dfrac{d}{dt} t^n = n\, t^{n-1}$. So $\dfrac{d}{dt}(a_3 t^3) = 3a_3 t^2$.
- Sine and cosine: $\dfrac{d}{dt} \sin t = \cos t$ and $\dfrac{d}{dt} \cos t = -\sin t$.
- Exponential: $\dfrac{d}{dt} e^{at} = a\, e^{at}$.
- Chain rule: the derivative of $f(g(t))$ is $f'(g(t))\, g'(t)$. Example: $\dfrac{d}{dt} \sin(3t^2) = \cos(3t^2) \cdot 6t$. In the arm chapter, $g = \theta_1 + \theta_2$ and $g' = \dot{\theta}_1 + \dot{\theta}_2$.
- Partial derivative $\partial f / \partial x$: differentiate with respect to x and treat every other variable as a constant. The **gradient** ∇f is the vector of all partial derivatives; for $f = \frac{1}{2}(x^2 + 10y^2)$ it is $(x,\ 10y)$ and it points uphill, so gradient descent subtracts it.

A.4 Exponentials and logarithms

The number $e = 2.71828\ldots$ is the base of natural growth and decay, and ln is its inverse: $\ln e^x = x$. Because physical systems relax toward their final state, the function $e^{-t/\tau}$ appears throughout the book. It has handy values:

time	τ	2τ	3τ	4τ
$e^{-t/\tau}$ (what is left)	0.368	0.135	0.050	0.018
$1 - e^{-t/\tau}$ (what is done)	63.2%	86.5%	95.0%	98.2%

That table is the reason for the rules of thumb "63% in one time constant", "95% in three" and "settled in four".

A.5 Averages, variance and noise

For readings $z_1, \ldots, z_M$ the **mean** is $\bar{z} = \frac{1}{M}\sum z_i$, the **variance** is the mean of the squared deviations, $\sigma^2 = \frac{1}{M}\sum (z_i - \bar{z})^2$, and the **standard deviation** is σ. Example: $2, 4, 4, 4, 5, 5, 7, 9$ have mean 5, squared deviations $9, 1, 1, 1, 0, 0, 4, 16$ summing to 32, variance $32/8 = 4$, and $\sigma = 2$.

For *independent* quantities the variances add: $\mathrm{Var}(a + b) = \mathrm{Var}(a) + \mathrm{Var}(b)$, and multiplying a quantity by a constant c multiplies its variance by c^2. These two facts are all that is needed for the averaging law $\sigma/\sqrt{M}$ and for the Kalman gain.

A.6 Complex numbers, in one paragraph

Engineers write j for the number with $j^2 = -1$. A pair $a + jb$ is a point in a plane, with a its real part. When a quadratic equation has a negative number under its square root, the roots are complex, $-\sigma \pm j\omega_d$. In a response, the real part $-\sigma$ sets how fast the motion dies away ($e^{-\sigma t}$) and the imaginary part ω_d sets how fast it rings. A root with a positive real part would give $e^{+\sigma t}$, growth without limit: an unstable system.

Appendix B

Formula Sheet

Robot geometry and kinematics

Grübler (planar DOF)	$\text{DOF} = 3(n-1) - 2j_1 - j_2$
Planar rotation	$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$, $R^{-1} = R^T$
Homogeneous transform	$T = \begin{pmatrix} R & \mathbf{d} \\ \mathbf{0}^T & 1 \end{pmatrix}$, $T^{-1} = \begin{pmatrix} R^T & -R^T\mathbf{d} \\ \mathbf{0}^T & 1 \end{pmatrix}$
Two-link forward kinematics	$x = l_1 c_1 + l_2 c_{12}$, $y = l_1 s_1 + l_2 s_{12}$
Two-link inverse kinematics	$\cos\theta_2 = \dfrac{x^2 + y^2 - l_1^2 - l_2^2}{2 l_1 l_2}$, $\theta_1 = \text{atan2}(y, x) - \text{atan2}(l_2 s_2,\ l_1 + l_2 c_2)$
Workspace (two-link)	$\lvert l_1 - l_2 \rvert \le r \le l_1 + l_2$
Jacobian (two-link)	$J = \begin{pmatrix} -l_1 s_1 - l_2 s_{12} & -l_2 s_{12} \\ l_1 c_1 + l_2 c_{12} & l_2 c_{12} \end{pmatrix}$, $\det J = l_1 l_2 \sin\theta_2$
Cubic trajectory	$q = q_0 + \Delta q\,[3s^2 - 2s^3]$, $s = t/T$; peak speed $1.5\,\Delta q / T$

Sensors and signals

Encoder resolution	$360^\circ / (4N)$ with quadrature decoding
Ultrasonic distance	$d = vt/2$, $v \approx 331.3 + 0.606\,T_C$ m/s
ADC	$\text{LSB} = V_{\text{ref}} / 2^n$, error $\le$ LSB/2
Sampling	$f_s > 2 f_{\max}$
Moving average	noise reduced by $\sqrt{M}$
Exponential filter	$y_k = (1-\alpha) y_{k-1} + \alpha z_k$
Kalman (1D)	$K = P^- / (P^- + R)$, $\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$, $P = (1 - K)P^-$

Control

PID	$u = K_p e + K_i \int e\,dt + K_d\,\dot{e}$
P control on first-order plant	$y_\infty = \dfrac{K K_p}{1 + K K_p}\, r$
Second-order overshoot	$\%OS = 100 \exp\!\left(-\pi\zeta / \sqrt{1-\zeta^2}\right)$
Settling time (2%)	$T_s \approx 4/(\zeta\omega_n)$
Ziegler–Nichols	$K_p = 0.6K_u$, $K_i = 2K_p/T_u$, $K_d = K_p T_u/8$

Actuators and grippers

DC motor	$\tau = K_t i$, $e_b = K_e\omega$, $\tau = \dfrac{K_t}{R}(V - K_e\omega)$
Stall torque, no-load speed	$\tau_s = K_t V/R$, $\omega_0 = V/K_e$, $P_{\max} = \tau_s\omega_0/4$
Stepper	$\omega = 2\pi f\,\theta_{\text{step}}/360^\circ$
Gearbox	$\omega_{out} = \omega_{in}/N$, $\tau_{out} = N\eta\tau_{in}$, reflected inertia J_L/N^2
Hydraulics	$p = F_1/A_1 = F_2/A_2$, $F_1 d_1 = F_2 d_2$
Friction grip	$N = Sm(g+a)/(n\mu)$; friction cone half-angle $\arctan\mu$
Suction	$F = \Delta p\, A$ per cup

Intelligence

Neuron	$y = \sigma(\mathbf{w}^T\mathbf{x} + b)$, $\sigma(z) = 1/(1+e^{-z})$
Gradient descent	$w \leftarrow w - \eta\,\partial L/\partial w$
A*	$f(n) = g(n) + h(n)$, admissible if $h \le h^*$
Manhattan distance	$\lvert r - r_g\rvert + \lvert c - c_g\rvert$
Bayes	$P(H \mid E) = P(E \mid H)P(H)/P(E)$
Odds form	posterior odds = prior odds $\times$ likelihood ratio
Expected utility	$EU(a) = \sum_{s'} P(s' \mid a)U(s')$

Mobile robots

Differential drive	$v = (v_R + v_L)/2$, $\omega = (v_R - v_L)/b$
Turning radius	$\rho = \dfrac{b}{2}\dfrac{v_R + v_L}{v_R - v_L}$
Odometry	$\Delta s = (\Delta s_R + \Delta s_L)/2$, $\Delta\theta = (\Delta s_R - \Delta s_L)/b$

Appendix C

Programs to Run and Change

Each program is short on purpose. Type them in rather than copying; typing slows you down just enough to notice what each line does. They need only Python 3 and the libraries `numpy` and `matplotlib`.

C.1 Forward and inverse kinematics

```
import numpy as np

l1, l2 = 0.5, 0.3

def fk(t1, t2):
    x = l1*np.cos(t1) + l2*np.cos(t1 + t2)
    y = l1*np.sin(t1) + l2*np.sin(t1 + t2)
    return x, y

def ik(x, y, elbow=+1):
    c2 = (x*x + y*y - l1*l1 - l2*l2) / (2*l1*l2)
    if abs(c2) > 1:
        raise ValueError("target out of reach")
    t2 = elbow * np.arccos(c2)
    t1 = np.arctan2(y, x) - np.arctan2(l2*np.sin(t2), l1 + l2*np.cos(t2))
    return t1, t2

t1, t2 = ik(0.5, 0.4)
print(np.degrees([t1, t2]), fk(t1, t2)) # check: should print (0.5, 0.4)
```

C.2 A PID loop on a simulated motor

```
import numpy as np
import matplotlib.pyplot as plt

def simulate(kp, ki, kd, T=3.0, dt=0.001):
    # first-order motor: tau*dy/dt + y = K*u
    tau, K = 0.4, 2.0
```

```
    y, I, prev_e = 0.0, 0.0, 1.0
    t_list, y_list = [], []
    for k in range(int(T/dt)):
        e = 1.0 - y # set-point is 1
        I += e*dt
        d = (e - prev_e)/dt
        prev_e = e
        u = np.clip(kp*e + ki*I + kd*d, -12, 12)
        y += dt*(K*u - y)/tau
        t_list.append(k*dt); y_list.append(y)
    return t_list, y_list

for gains in [(1, 0, 0), (3, 0, 0), (3, 4, 0.05)]:
    t, y = simulate(*gains)
    plt.plot(t, y, label=str(gains))
plt.legend(); plt.xlabel("time (s)"); plt.show()
```

C.3 A neuron network that learns XOR

```
import numpy as np
rng = np.random.default_rng(4)
X = np.array([[0,0],[0,1],[1,0],[1,1]], float)
Y = np.array([[0],[1],[1],[0]], float)
W1 = rng.normal(0, 1, (2, 4)); b1 = np.zeros(4)
W2 = rng.normal(0, 1, (4, 1)); b2 = np.zeros(1)
sig = lambda z: 1/(1+np.exp(-z))

for epoch in range(6000):
    h = sig(X @ W1 + b1) # forward pass
    out = sig(h @ W2 + b2)
    d2 = (out - Y) * out * (1 - out) # backward pass
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 -= 1.5 * h.T @ d2; b2 -= 1.5 * d2.sum(0)
    W1 -= 1.5 * X.T @ d1; b1 -= 1.5 * d1.sum(0)

print(np.round(out.ravel(), 3)) # close to 0 1 1 0
```

C.4 A* on a grid

```
import heapq

def astar(grid, start, goal):
    R, C = len(grid), len(grid[0])
    h = lambda p: abs(p[0]-goal[0]) + abs(p[1]-goal[1])
    frontier = [(h(start), 0, start)]
    came, best = {start: None}, {start: 0}
    while frontier:
        f, g, p = heapq.heappop(frontier)
        if p == goal:
            path = []
```

```
            while p is not None:
                path.append(p); p = came[p]
            return path[::-1]
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            q = (p[0]+dr, p[1]+dc)
            if 0 <= q[0] < R and 0 <= q[1] < C and grid[q[0]][q[1]] == 0:
                if q not in best or g + 1 < best[q]:
                    best[q] = g + 1
                    came[q] = p
                    heapq.heappush(frontier, (g + 1 + h(q), g + 1, q))
    return None

grid = [[0,0,0,0],[1,1,0,1],[0,0,0,0]]
print(astar(grid, (0,0), (2,0)))
```

C.5 Differential-drive odometry

```
import numpy as np

def update(pose, nL, nR, r=0.05, C=960, b=0.2):
    x, y, th = pose
    sL = 2*np.pi*r*nL/C
    sR = 2*np.pi*r*nR/C
    ds, dth = (sR + sL)/2, (sR - sL)/b
    x += ds*np.cos(th + dth/2)
    y += ds*np.sin(th + dth/2)
    return (x, y, th + dth)

print(update((0, 0, 0), 1920, 2880)) # one big update; the exact arc ends at (0.5,
    0.5), so use small intervals
```

Appendix D

Glossary

Accuracy How close a robot gets to a commanded point.

Actuator A device that produces motion or force from energy.

Admissible heuristic A heuristic that never overestimates the remaining cost.

ADC Analog-to-digital converter.

Agent Anything that perceives through sensors and acts through actuators.

Backlash Free play between meshing gear teeth.

Back-EMF Voltage generated by a spinning motor that opposes the supply.

Bayes' rule The formula that updates a belief from evidence.

Closed loop Control that uses a measurement of the output.

Degrees of freedom Number of independent motions a mechanism can make.

End effector The tool or gripper at the end of an arm.

Encoder A sensor that reports shaft rotation.

Forward kinematics Hand position computed from joint values.

Friction cone Set of contact force directions that do not cause slipping.

Heuristic An estimate that guides a search.

Homogeneous transform A matrix combining rotation and translation.

Inverse kinematics Joint values computed from a desired hand pose.

Jacobian Matrix relating joint rates to hand velocity.

Kalman filter Estimator that blends prediction and measurement by their uncertainties.

LiDAR Laser range sensor.

LSB Least significant bit; the step size of an ADC.

Minimax Search that assumes an opponent plays best.

Non-holonomic A system that cannot move in every direction instantaneously.

Odometry Estimating pose by integrating wheel motion.

Overfitting A model that fits training noise and fails on new data.

PID Proportional–integral–derivative control.

PWM Pulse-width modulation: average power by fast switching.

Quadrature Two encoder channels a quarter cycle apart that give direction.

Repeatability How closely a robot returns to the same point.

Resolution Smallest detectable or commandable change.

Singularity A pose where the Jacobian loses rank.

Stall torque Torque of a motor at zero speed.

Workspace The set of points the tool can reach.

Appendix E

Where to Go Next

E.1 Books I recommend

These are standard references that students and teachers have used for many years. Read them in small pieces alongside a program or a build.

- *Introduction to Robotics: Mechanics and Control*, John J. Craig. A clear treatment of frames, kinematics and dynamics of arms.
- *Robotics: Modelling, Planning and Control*, Siciliano, Sciavicco, Villani and Oriolo. A thorough, mathematical survey.
- *Modern Robotics: Mechanics, Planning, and Control*, Kevin Lynch and Frank Park. Uses a unified geometric approach.
- *Introduction to Autonomous Mobile Robots*, Siegwart, Nourbakhsh and Scaramuzza. Wheeled robots, perception and navigation.
- *Artificial Intelligence: A Modern Approach*, Stuart Russell and Peter Norvig. The standard AI textbook; search, logic, probability and learning.
- *Probabilistic Robotics*, Thrun, Burgard and Fox. Filters, localisation and mapping.
- *Feedback Systems*, Karl Åström and Richard Murray. Control with a physical feel.
- *Deep Learning*, Goodfellow, Bengio and Courville. When you want to go beyond the single neuron.

E.2 Things to build

1. A line follower with the four-rule reflex agent of Chapter 8, then give it memory to survive a gap in the line.
2. A two-link arm from cardboard and two hobby servos; compare the angles you command with a protractor and measure its repeatability.
3. A distance-keeping cart: sonar, a P controller on distance error, and a plot of how it behaves as you raise the gain.

4. A map-and-plan robot: a grid map in software, A* for the route, and odometry for the following; measure the drift and correct it with a landmark.
5. A small evolutionary search that tunes the PID gains of your simulated motor.

E.3 A closing word

When I started teaching this subject, I believed the main job was to explain equations. I now think the main job is to make students unafraid of them. An equation in robotics is only a sentence about something you can touch: how long the arm is, how fast the wheel turns, how sure the sensor is. Keep asking what the symbols are in the real machine, and keep testing your answers against it. Build things, break things, and write down what you learned. I hope this book helps you take the next step, and I would be glad to hear where it leads.

Mr. Sourbh Kumar

ABOUT THIS BOOK

Robots are built from ideas you can learn: a sensor that measures, a controller that corrects, a motor that moves and an algorithm that decides. This book walks through each of them with full calculations, worked examples, figures, practice problems with solutions and short programs to run.

It was written for students who want to understand how robots work, and to keep exploring once the lecture is over.



Mr. Sourbh Kumar

teaches robotics and artificial intelligence and works on STEM robotics learning tools at Livid Design.

Livid Design Inc.
The way we build, makes difference.

Livid Design LLP (Livid Design Inc.)

Web www.lividdesign.in
Email info@lividdesign.in
Phone +91-8348827197

© 2026 Livid Design LLP. All rights reserved.

The way we build, makes difference.